SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

VARAŽDIN

Nikola Kihas

RAZVOJ WEB KORISNIČKOG SUČELJA PREMA NAČELIMA ODRŽIVOSTI KODA

ZAVRŠNI RAD

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU

FAKULTET ORGANIZACIJE I INFORMATIKE

V A R A Ž D I N

Nikola Kihas

Matični broj: 35918/07-R

Studij: Informacijski i poslovni sustavi

RAZVOJ WEB KORISNIČKOG SUČELJA PREMA NAČELIMA ODRŽIVOSTI KODA

ZAVRŠNI RAD

Mentor :

Prof. dr. sc. Dragutin Kermek

Varaždin, rujan 2026.

Nikola Kihas

Izjava o izvornosti

Izjavljujem da je moj završni rad izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvaćanjem odredbi u sustavu FOI-radovi

Sažetak

Rad istražuje načela razvoja modularnog web korisničkog sučelja s naglaskom na iskoristivost, skalabilnost i responzivnost koda. Analiziraju se koncepti u okviru Angular okvira koji omogućuje učinkovitiju organizaciju i održavanje elemenata što rezultira manjom količinom koda koji je pregledniji i jednostavniji za održavanje. U pozadinskom dijelu primjenjuju se klase i načela objektno orijentiranog programiranja radi postizanja čitljivosti i mogućnosti jednostavnog proširenja koda. U korisničkom dijelu koriste se suvremene CSS tehnike za prilagodbu sadržaja različitim širinama zaslona. U praktičnom dijelu rada biti će izrađena web aplikacija u skladu s načelima održivosti koda.

Ključne riječi: web; sučelje; održivost; Angular; SCSS; OOP; responzivan dizajn

Sadržaj

1. Uvod 1

2. Održivost i pristupi izradi održivih rješenja 2
2.1. Povijest i tehnologije web razvoja 2
2.1.1. Web razvoj kroz povijest 2
2.1.2. Tehnologije u web razvoju 3
2.2. Održivost programskih rješenja 4
2.3. Metodologije i pristupi oblikovanju arhitekture rješenja 7
2.3.1. O metodologijama i njihovoj važnosti 7
2.3.2. Načini pristupa prema projektu 7
2.3.3. Suvremeni pristup razvoju programskog koda 10

3. Načela održivog programskog koda 12
3.1. Principi pisanja održivog programskog koda 12
3.1.1. Principi pisanja funkcija 12
3.1.2. Načela ponavljanja i apstrakcije 13
3.1.3. Načela jednostavnosti 13
3.1.4. Razdvajanje odgovornosti 15
3.2. Objektno usmjereno oblikovanje programskog koda 16
3.2.1. Princip jedne odgovornosti - SRP 17
3.2.2. Princip otvorenosti/zatvorenosti - OCP 18
3.2.3. Princip Liskovljeve zamjene - LSP 18
3.2.4. Princip odvajanja sučelja - ISP 18
3.2.5. Princip inverzije odgovornosti - DIP 18

4. Arhitektura programskog rješenja 20
4.1. Modularna arhitektura korisničkog sučelja 20
4.2. Ciljana struktura korisničkog sučelja 22
4.2.1. Slojevi i njihove uloge 23
4.2.2. Tok komunikacije 26
4.2.3. Povezanost s ostalim arhitekturalnim obrascima 26
4.3. Angular koncepti za održivost korisničkog sučelja 27
4.3.1. Komponente kao temelj održive izgradnje sučelja 27
4.3.2. Angular servisi 30
4.3.3. Spajanje dijelova sustava 30
4.3.4. Jednostranične aplikacije i navigacija 31

4.3.5. Grupiranje elemenata . . . 32
4.4. Arhitektura i organizacija koda na strani poslužitelja . . . 32
4.4.1. Izvođenje JavaScript koda van preglednika . . . 32
4.4.2. NestJS . . . 33

5. Oblikovanje i organizacija stilova . . . 34
5.1. Principi pisanja održivih stilova . . . 35
5.1.1. Problemi kaskade . . . 35
5.1.2. Objektno orijentirani CSS: OOCSS . . . 36
5.1.3. BEM konvencija imenovanja . . . 39
5.1.4. Pretprocesori . . . 41
5.2. Responzivnost primjenom suvremenog CSS-a . . . 43
5.2.1. Upiti o mediju . . . 44
5.2.2. CSS jedinice . . . 45
5.2.3. Fluidne vrijednosti i CSS funkcije . . . 45
5.2.4. Fluidni raspored elemenata . . . 47
5.2.5. Tokeni i varijable . . . 49
5.3. Organizacija stilskih datoteka . . . 51
5.3.1. Slojevi ITCSS-a . . . 52

6. Praktični dio rada: Web aplikacija Kroz Hrvatsku . . . 55
6.1. Opis teme aplikacije . . . 55
6.2. Arhitektura poslužiteljskog dijela aplikacije . . . 56
6.2.1. Opća strukutra aplikacije . . . 56
6.2.2. Struktura poslužiteljskog dijela aplikacije . . . 56
6.2.3. Komunikacija bazom podataka . . . 58
6.3. Arhitektura korisničkog dijela aplikacije . . . 59
6.3.1. Funkcionalnosti i njihova podjela . . . 60
6.3.2. Dohvat i prikaz podataka . . . 61
6.3.3. Stilovi . . . 62
6.4. Analiza implementiranih stranica . . . 66
6.4.1. Analize navigacije . . . 66
6.4.2. Analiza početne stranice . . . 71
6.4.3. Analiza stranice popisa lokacija . . . 79
6.4.4. Analiza stranice detalja . . . 85
6.4.5. Analiza stranice trgovine . . . 89
6.4.6. Analiza stranice profil . . . 90
6.4.7. Analiza stranice uređivanja lokacije . . . 92

7. Zaključak . . . 94

Popis literature . . . 96

Popis slika 98

Popis programskih kodova 100

1. Uvod

Razvoj web aplikacija danas se u velikoj mjeri oslanja na brzu isporuku funkcionalnosti, česte promjene zahtjeva, korisničkih očekivanja i zakonskih regulativa. Korisničko sučelje (eng. User Interface, UI) je dodirna točka između korisnika i usluge koju neka organizacija nudi, te je također dio sustava koji se često mijenja, jer izravno predstavlja osnovne funkcije koje aplikacija nudi, različite poslovne potrebe i način na koji korisnici koriste aplikaciju. Zbog tih razloga, kvaliteta implementacije korisničkog sučelja ne utječe samo na izgled i iskustvo korisnika kod koristenja (eng. User Experience, UX), nego i na dugoročnu upotrebljivost aplikacije.

Održavanje programskog koda obuhvaća sve promjene unutar programskog koda koje se provode nakon što je softver pušten u produkciju. Jedan od ključnih čimbenika koji omogućuju lakše ispravljanje pogrešaka i nadogradnju sustava jest održivost koda. Održiv kod omogućuje da se sustav kroz vrijeme jednostavnije razumije, održava i proširuje, odnosno da nastavi zadovoljavati korisničke zahtjeve uz smanjenje tehničkog duga [1, str. 7-3]. U kontekstu web korisničkog sučelja to se posebno odnosi na modularnost, organizaciju komponenti, jasnu strukturu projekta, pravilno razdvajanje odgovornosti te dosljedan pristup dizajnu i responzivnosti sučelja. Zanemarivanjem ovih načela često dovodi do dupliciranja logike i stilova, velikih komponenti koje su zadužene za više zadataka, što otežava razvoj i održavanje sustava.

Cilj ovog završnog rada jest istražiti načela razvoja modularnog web korisničkog sučelja prema principa održivosti koda, s naglaskom na ponovnu iskoristivost, skalabilnost i responzivnost. U teorijskom dijelu rada obrađuju se ključni principi i prakse održivog razvoja, pristupi dizajnu i metodologije razvoja softvera te njihov utjecaj na održivost sustava. Također se analiziraju specifični pristupi pisanju održivog koda unutar Angular okvira, kao i načela pisanja kvalitetnog i preglednog CSS koda.

U praktičnom dijelu rada izrađuje se web aplikacija koja primjenjuje analizirana načela. Implementacija obuhvaća modularnu strukturu projekta, ponovnu iskoristivost komponenti korisničkog sučelja te primjenu suvremene CSS tehnike za prikaz elemenata na različitim širinama zaslona. Kod će biti oblikovan prema načelima objektno orijentiranog programiranja s ciljem postizanja veće čitljivosti i lakšeg održavanja. Dodatno će se izraditi pripadni dijagrami kako bi se prikazala struktura i ponašanje sustava te osigurala jasna dokumentacija dizajna.

2. Održivost i pristupi izradi održivih rješenja

2.1. Povijest i tehnologije web razvoja

2.1.1. Web razvoj kroz povijest

Razvoj interneta i Web-a započeo je početkom 1990-ih godina, kada je Tim Berners-Lee htio riješiti problem komunikacije između znanstvenika te im omogućiti jednostavan pristup i dijeljenje dokumenata pomoću hiperpoveznica. Kao rješenje razvio je World Wide Web, koji je danas poznat kao Web, odnosno sustav koji koristi internetom kako bi pristupio dokumentima putem pretraživača. Važno je razlikovati internet koji predstavlja globalnu mrežu međusobno povezanih računala i Web-a koji je jedna od usluga koja se tom mrežom koristi kako bi se pristupilo određenom sadržaju. Berners-Lee pritom je razvio temeljne koncepte poput hiperteksta, URL adresa (eng. Uniform Resource Locator), HTTP protokola (eng. Hypertext Transfer Protocol) i jezik HTML (eng. HyperText Markup Language). U svojim počecima web stranice bile su statične i sastojale su se pretežito od tekstualnog sadržaja, koji je bio povezan hiperpoveznicama. Svrha stranica bila je prvenstveno prikaz informacija, bez značajne interakcije s korisnicima.[2]

Komercijalizacijom Web-a njegova se početna namjena proširila s razmjene znanstvenih informacija na širu svakodnevnu uporabu. Rastom korisničkih zahtjeva HTML je nadopunjen mogućnostima oblikovanja sadržaja primjenom CSS-a. Takve stranice podigle su interaktivnost i korisničko iskustvo, no stranice su i dalje bile daleko od onih koje danas poznajemo. Web stranice su bile oblikovane za jednu ciljanu rezoluciju, dok se prilagodba različitim veličinama zaslona uglavnom zanemarivala.

S vremenom se pojavila potreba za razvojem dinamičnih i funkcionalnih web aplikacija, pri čemu je važnu ulogu imao razvoj poslužiteljskih tehnologija. Zahvaljujući tome danas se koriste tehnologije koje omogućuju izradu aplikacija usmjerenih na korisničko iskustvo, potiči dulje zadržavanje korisnika na stranici i lakšu navigaciju. Isto tako, moderne tehnologije omogućuju razvoj složenih informacijskih sustava, poput ERP (eng. Enterprise Resource Planning) rješenja, administracijskih sučelja, poslovnih portala te javnih usluga poput e-Građani. Takvi sustavi često integriraju podatke iz više izvora, obrađuju ih i prikazuju unutar jedinstvenog sučelja, pri čemu rade s osjetljivim osobnim i poslovnim podacima.

S ovim izazovima složenost sustava raste i broj scenarija koje web aplikacija mora podržati. Korisnik se stavlja u središte procesa dizajna, što uključuje responzivan prikaz na različitim

uređajima i veličinama zaslona, poštivanje korisničkih preferenci poput svijetle ili tamne teme. Uz to, važnu ulogu ima i pristupačnost, odnosno prilagodba sučelja korisnicima s različitim potrebama, poput osobama slabijeg vida ili osobama s poteškoćama u raspoznavanju boja. Takav pristup podrazumijeva da se sučelje gradi tako da bude prilagodljivo, robusno i lako proširivo.

2.1.2. Tehnologije u web razvoju

Suvremeni web razvoj temelji se na nekoliko ključnih tehnologija koje služe za rješavanje prethodno navedenih izazova. HTML (eng. HyperText Markup Language) je deklarativni jezik koji služi za definiranje strukture i sadržaja web stranice. Drugi spomenuti deklarativni jezik, CSS (Cascading Style Sheets) koristi se za oblikovanje izgleda elementa, uključujući boje, raspored, tipografiju i prilagodbu različitim veličinama zaslona. Za interakciju i dinamičko ponašanje korisničkog sučelja koristi se skriptni jezik JavaScript koji omogućuje manipulaciju sadržajem definiranim u HTML-u te prikaz novog, odnosno dinamički generiranog sadržaja. Ove tri tehnologije čine temelj razvoja web korisničkog sučelja te se ove tehnologije međusobno nadopunjuju u izradi suvremenih web aplikacija.

S porastom složenosti web aplikacija razvijeni su okviri i biblioteke koje olakšavaju njihov razvoj. Njihova svrha je pojednostaviti izgradnju složenih korisničkih sučelja primjenom komponentnog pristupa, upravljanja stanjem aplikacije i jasnijeg strukturiranja programskog koda. U kontekstu održivosti koda, komponentni pristup ima veliku važnost jer omogućuje ponovnu iskoristivost dijelova sučelja, lakše testiranje i pregledniju organizaciju projekta. Najpopularniji okviri na klijentskoj strani su React, Vue i Angular[3]. Angular se ističe po tome što pruža cjelovito razvojno okruženje s jasno definiranim obrascima te ga to čini okvirnom koji u prvi plan stavlja skalabilnost što olakšava razvoj velikih i modularnih aplikacija. Osim klijentskih rješenja, postoje i odgovarajući okviri na poslužiteljskoj strani. Njihova uloga je obrada poslovne logike, komunikacija s bazom podataka te upravljanjem autentifikacijom i autorizacijom korisnika. Putem njih se najčešće izlažu programska sučelja (eng. Application Programming Interface, API) koja omogućuju komunikaciju između poslužiteljskog i klijentskog dijela aplikacije.

Razvoj web tehnologija pratio je i rast zahtjeva za prilagodbu na različitim uređajima. Pojavom pametnih telefona i tableta responzivni web dizajn postao je jedno od temeljnih načela suvremenog web razvoja,a često se preporučuje i razvoj web stranica prvo za mobilne uređaje, poznatiji kao *mobile-first* pristup. Suvremene CSS tehnike poput Flexbox-a, CSS rešetke (eng. grid) i korištenja medijskih upita omogućuju izradu korisničkih sučelja koja se prilagođavaju različitim veličinama zaslona, odnosno ostvaruju responzivnost. Time se osigurava pristupačnost

i upotrebljivost aplikacija neovisno o uređaju.

2.2. Održivost programskih rješenja

Zbog navedenih zahtjeva održivost programskog koda u razvoju web korisničkog sučelja postaje posebno važna. Responzivnost pristupačnost i kontinuirano proširivanje funkcionalnosti uvode dodatne slojeve logike i stilizacije, a bez jasne strukture i unaprijed definiranih pravila takva složenost brzo dovodi do dupliciranja koda, nekonzistentnosti i u konačnici stvaranju tehničkog duga. Prema normi ISO/IEC/IEEE 14764, održivost softvera definira se kao stupanj učinkovitosti u kojem se sustav može modificirati [4]. Održivost se ne odnosi samo na ispravljanje pogrešaka nakon isporuke, nego na širi skup aktivnosti kojima se osigurava da programsko rješenje tijekom svojeg životnog vijeka i dalje zadovoljava korisničke zahtjeve. Prema istraživanjima koje navodi SWEBOK, više od 80% aktivnosti održavanja softvera odnosi se na njegovo unapređenje i prilagodbu, dok se manji dio odnosi na ispravljanje pograšaka u programskom kodu [1, str. 7-3].

Prava kvaliteta programskog rješenja se ne nalazi samo u tome što zadovoljava trenutne zahtjeve i probleme, nego se kvaliteta očituje u mogućnosti da rješenje zadovoljava buduće zahtjeve. Pri rješavanju određenog problema najčešće postoji više mogućih načina za rješavanje. Iako različita rješenja rješavaju problem, način da koji je problem riješen može znatno utjecati na kvalitetu strukture projekta, čitljivost koda, mogućnost nadogradnje, testiranja i općeg razumijevanja sustava. Upravo se u tim aspektima razlikuje kod koji je više održiv od onoga koji je manje održiv. Održiv kod jest onaj koji je jednostavan za izmjenu i nadogradnju, a da se prethodno nije morao mijenjati i nije prouzrokovao nove pogreške. Kod manjih projekata posljedice lošije strukturiranog koda često nisu odmah vidljive niti prouzrokuju prevelik problem. Međutim u većim sustavima isti problemi postaju uočljiviji i izraženiji, te zahtijevaju otklanjanje. Male pogreške i loše odluke koje su donesene u ranim fazama razvoja mogu se akumulirati i prerasti u znatne probleme, što je poznato kao efekt grude snijega (eng. snowball effect).

Čisti kod usko je povezan s održivošću programskog rješenja jer teži čitljivosti, jasnoći i razumijevanju. Čist kod je oblikovan tako da njegova struktura, imenovanje i organizacija što jasnije prenose namjeru programera, bez složenosti, nejasnoća ili velikih komentara. Kada je kod pregledan i dosljedan, lakše je uočiti pogreške, uvoditi izmjene i nadograđivati postojeće funkcionalnosti. Čist kod rijetko je moguće napisati iz prvog pokušaja, jer je u početnim faza rješavanja problema teško unaprijed predvidjeti sve njegove aspekte i buduće zahtjeve. Zbog toga, nakon što je problem uspješno riješen, važno je provesti refaktoriranje kako bi kod zado-

voljavao kriterije čistog koda [5, str. 267-284]. Proces refaktoriranja osnovno je sredstvo kojim se osigurava preglednost i razumijevanje koda, a takav kod kasnije je lakše nadograditi i refaktorirati. Refaktoriranje je tehnika promjene unutarnje strukture programskog koda bez da se promijeni njegovo vanjsko ponašanje. Prema SWEBOK-u, u poglavlju *Software Maintenance*, odjeljku *Software Reengineering*, refaktoriranje se opisuje kao tehnika reorganizacije programa bez promjene njegova ponašanja, s ciljem poboljšanja unutarnje strukture i održivosti softvera [1, str. 7-13].

Čisti kod, osim refaktoriranjem, postiže se primjenom konvencija imenovanja varijabli, funkcija, datoteka i drugih programskih elemenata. Dosljedno korištenje donesenih pravila oko imenovanja olakšava identifikaciju varijable u odnosu na funkciju i obratno, može se zaključiti svrha i vidljivost varijable i sl. Na primjer, ako naziv varijable počinje povlakom (donja crta), može se zaključiti da je ona privatni član klase ili ako je pisana svim velikim slovima znači da je varijabla konstanta. Isto tako postoje notacije koje olakšavaju prepoznavanje varijabli poput: tzv. devinog zapisa (camel case), zmijskog zapisa (snake case) ili mađarske notacija (Hungarian notation). U devnoj notaciji prva riječ se piše malim slovom, dok se svaka sljedeća piše velikim slovom.

```
1    imeVarijable
```

Programski kod 2.1: Devina notacija

Unutar zmijekse notacije sve se riječi pišu malim slovom, ali između njih stavlja se povlaka (donja crta)

```
1    ime_varijable
```

Programski kod 2.2: Zmijina notacija

Pri korištenju mađarske notacije koristi se devina notacija s time da prefiks označava tip podataka, npr. *string*, *integer* i sl.

```
1    strImeVarijable
```

Programski kod 2.3: Notacija - mađarska notacija

Također, bitno je davati smislena imena varijablama i funkcijama. Ona imena koja opisuju namjenu, izbjegavaju dezinformacije i zbunjenost te ne sadrže kodna imena i abrevijacije. [6, *Chapter 2: Meaningful Names*] Pridržavanjem ovih pravila kod postaje samodokumentirajući te u većini slučajeva komentari neće biti potrebni. Autor knjige *Clean code*, Martin kaže da ispravna uporaba komentara služi da nadoknadi neuspjeh u izražavanju kroz kod, no isto tako

napominje da postoje opravdani razlozi za korištenje komentara. [6, *Chapter 4: Comments*].

Preporučuje se dosljedno oblikovati kod, odnosno primjenjivati ujednačena pravila pisanja, uključujući uvlačenja i načini pisanja uglatih zagrada pri pisanju petlji i uvjeta. Primjerice, stavlja li se otvorena uglata zagrada u isti red kao i petlja ili će se staviti u novi red. Radi bolje preglednosti poželjno je da pojedini redak koda ne prelazi 120 znakova, čime se olakšava čitanje i izbjegava potreba za horizontalnim klizačem. Za svaki blok koda trebali bi biti moguće identificirati svrhu i opseg, zbog čega funkcije ne bi trebale biti prevelike. Martin pritom ističe da bi funkcije trebale biti što kraće, odnosno ne veće od 5 redaka. Iako u pojedinim situacijama veće funkcije nije moguće u potpunosti izbjeći, njihovu složenost moguće je smanjiti primjenom tehnika refaktoriranja, poput izdvajanje dijela logike u zasebnu funkciju koju izvorna funkcija poziva [5, str. 61]. Time se postiže da inicijalna funkcija nije velika i dio koda poPrisma ima, odnosno opis koji opisuje njegovu funkciju. Također, preporučuje se da funkcije ne primaju više od 3 argumenta. Kada je veći broj argumenata ipak nužan, korisno ih je grupirati u klase ili druge prikladne strukture podatka, kako bi bilo jasnije koje je podatke potrebno proslijediti i kako bi se povećala čitljivost koda [6, *Chapter 3: Functions*].

Važan koncept koji se nastoji ublažiti primjenom načela čistog koda jest tehnički dug. Martin u knjizi *Clean Architecture* [5, str. 139] spominje kako je održavanje najskuplji aspekt programskog rješenja, zbog beskrajnih zahtjeva, neizbježnih nedostataka i pogrešaka te takve stvari zahtijevaju mnogo ljudskih resursa. SWEBOK definira tehnički dug kao dodatni napor potreban za ispravljanje problema koji ostaju u programskom kodu nakon početne isporuke softvera [1, str. 7-9]. On nastaje kada se kod razvija s ciljem brže isporuke funkcionalnosti, pri čemu se zanemaruju kvaliteta strukture i dugoročna održivost. Takav dug se može usporediti s kreditom, u smislu da kratkoročno donosi određenu korist jer omogućuje bržu isporuku funkcionalnosti, no dugoročno povećava složenost sustava i otežava njegov daljnji razvoj. S vremenom svaka nova izmjena ili nadogradnja zahtjeva više vremena i truda, a istodobno raste i rizik od uvođenja novih pogrešaka. Posljedično, razvoj postaje skuplji, sporiji i zahtjevniji za održavanje. U manjim i kratkotrajnijim projektima, tehnički dug ne igra preveliku ulogu, odnosno određena razina je prihvatljiva, no u sustavima koji su predviđeni za dugotrajnu uporabu on često uzrokuje ozbiljne poteškoće u održavanju i daljnjem razvoju aplikacije.

2.3. Metodologije i pristupi oblikovanju arhitekture rješenja

2.3.1. O metodologijama i njihovoj važnosti

Održivost koda u velikoj mjeri ovisi o sposobnosti predviđanja i pripremi u organizaciji koda. Kod nastaje relativno brzo, dok se on koristi kroz dulje vremensko razdoblje te je zbog toga važno unaprijed razmišljati o strukturi rješenja i o dijelovima koji mogu više puta iskoristiti. Što se ranije uoče elementi koji se mogu više puta iskoristiti to je lakše izgraditi modularan sustav s manje dupliciranja, s jasnijim granicama odgovornosti te u konačnici s manje tehničkog duga. U duhu ovoga, metodologija razvoja i pristup dizajnu sustava ne određuje samo način rada, nego izravno utječe na kvalitetu i održivost programskog koda.

Metodologije razvoja softvera predstavlja okvir koji usmjerava rad tima kroz faze životnog ciklusa projekta. Odabir metodologije može varirati ovisno o zadatku i očekivanjima klijenta, odnosno o tome radi li se o zadatku koji je moguće precizno definirati i unaprijed isplanirati ili radi li se o zadatku s većom razinom neizvjesnosti. Zadaci koji se mogu lakše definirati najčešće imaju stabilne zahtjeve, jasnu strukturu ili su slični prijašnjim projektima s kojima je tim upoznati. Na drugoj strani nalaze se zadaci veće kompleksnosti, kod kojih su promjene česte, a rizik pogrešaka veći. U kontekstu web korisničkog sučelja to je čest slučaj, jer se zahtjevi sučelja mogu mijenjati radi promijene želja korisnika, promjene poslovnih pravila ili novih uređaja i standarda.

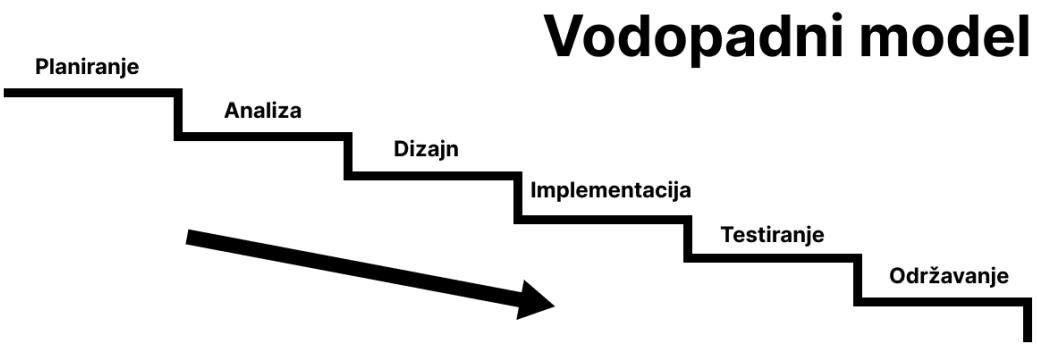
2.3.2. Načini pristupa prema projektu

Za odabir metodologije utječu preferencije timova koji rade na projektu kao i odnos učestalosti isporuke i učestalosti promjena koje sustav ili klijent zahtijevaju. U slučaju rijetkih isporuka i rijetkih zahtjeva za promjenama, često se koristi vodopadni (endl. *waterfall*) model (Slika 1). Ova se metodologija bazira na stvaranju i upravljanju artefaktima u fazama životnog ciklusa. Artefakti su rezultati procesa prijašnjih faza razvoja kako bi se proizveli novi. Projekt počinje fazom planiranja u kojoj se određuju ciljevi projekta, opseg rada, potrebni resursi i vremenski okvir izvedbe. Zatim dolazi faza analize u kojoj se prikupljaju i precizno opisuju funkcionalni i nefunkcionalni zahtjevi sustava, te kao artefakt iz te faze izlazi specifikacija zahtjeva. Iz specifikacije zahtjeva, u fazi dizajna, oblikuje se arhitektura sustava, baza podataka i način komunikacije komponentni, te se stvaraju UML dijagrami koji su dio tehničke dokumentacije. U fazi implementacije dijagrami se sa zahtjevima pretvaraju u programski kod, te je kao posljedica ove faze izvorni kod aplikacije. U fazi testiranja provjerava se ispravnost aplikacije kao

i usklađenost sa zadanim zahtjevima. U konačnici projekt ulazi u fazu održavanja u kojoj se sustav nakon puštanja u produkciju ispravlja, prilagođava i nadograđuje u skladu s novim potrebama. Ovakav pristup rješavanju problem još se naziva pristupom odozgo prema dolje (eng. *Top-Down*). Prvo se sustav promatra na najvišoj razini, a zatim se on razlaže na manje i detaljnije cjeline pa sve do pojedinih modula i funkcija. Martin u kontekstu pristupa odozgo prema dolje napominje da bi se trebalo oslanjati u smjeru stabilnosti [5, str. 120].

Cilj takvog pristupa jest u svakoj fazi što detaljnije pokriti svaki dio projekta kako bi se razvoj mogao kvalitetno organizirati i kako bi se sustav izgradio na dobrim temeljima. Prednost ovog pristupa jest detaljno planiranje, analiza zahtjeva i oblikovanje dizajna sustava kako bi sustav odgovarao korisničkim zahtjevima. Rezultat takvog pristupa je skup dizajnerskih odluka, model sustava i detaljna dokumentacija koja pomaže u razumijevanju softvera u fazi održavanja. To je posebno važno jer održavanje softvera često započinje ograničenim razumijevanjem sustava, osobito kada ga održava programer koji nije sudjelovao u njegovu izvornom razvoju. Jasna dokumentacija, modeli sustava i dosljedna organizacija koda mogu smanjiti vrijeme potrebno za razumijevanje mjesta na kojem je potrebno izvršiti izmjenu ili ispravak [1, str. 7-5].

Nedostatak vodopadnog pristupa je njegova tromost i slaba prilagodljivost promjenama. Budući da se razvoj odvija kroz unaprijed definirane faze koje su strogo odvojene, izmjene zahtjeva teško je uklopiti jer zahtijevaju promjene u različitim dijelovima projekta. Kada se završi određena faza, na nju se u pravilo ne vraća, te zbog toga naknadno uočena pogreška vuče za sobom dodatne troškove, kašnjenja i doradama. Martin ističe da je komponenti pristup teško dizajnirati pristupom odozgo prema dolje, jer se komponenti pristup zasniva na kontinuiranom proširenju klasa i drugih objekata, a pristup odozgo prema dolje ima za cilj sve te objekte definirati na početku razvoja [5, str. 188]. Ovaj model se smatra sporijim i manje prikladnim za dinamične projekte, no u određenim situacijama njegova detaljnost i jasno definirana struktura mogu donijeti više koristi nego ograničenja.

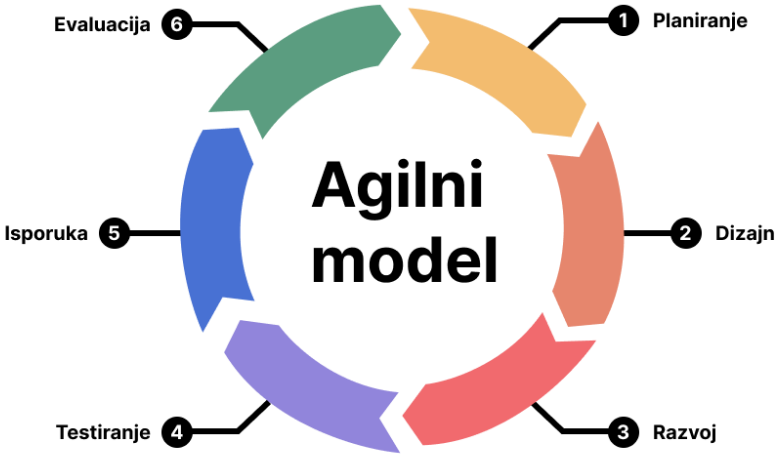


Slika 1: Vodopadni model (Izvor: slika je rekreirana prema slici sa `https://artoftesting.com/waterfall-model`, pristupljeno: 16.6.2026.)

Na drugoj strani spektra nalazi se agilni pristup razvoju (Slika 2). On je najprikladniji za projekte u kojima se očekuju promjene zahtjeva, zahtijevaju se česte isporuke te postoji aktivna suradnja s korisnicima koji daju povratne informacije za daljnji razvoj sustava. Za razliku od vodopadnog modela, koji se temelji na strogo definiranim i jasno odvojenim fazama, agilni se pristup sastoji od malih iterativnih koraka. To znači da se aktivnosti poput analize, dizajna, implementacije i testiranja ponavljaju kroz kraće faze razvojnog ciklusa, te se tako sustav postupno nadograđuje novim funkcionalnostima. Sustav se gradi izgradnjom manjih dijelova programskog koda (komponenta, klasa), te se zatim povezuju u veće cjeline i module, te takav se pristup često povezuje s pristupom odozdo prema gore (eng. *Bottom-Up*).

Fokus se stavlja na programski proizvod i ono što donosi vrijednost klijentu [1, str. 7.5], dok se dokumentacija i ostale dizajnerske odluke ne izostavljaju, ali se stavljaju u drugi plan. Zbog toga se dodatna važnost pridodaje čitljivosti, razumljivosti i dobro strukturiranog programskom kodu, koji olakšava suradnju između članova tima i kasnije održavanje sustava.

Glavna prednost agilnog pristupa u odnosu na vodopadni model jest njegova prilagodljivost promjenama i mogućnosti da korisnik brzo validira rješenje. Međutim, u konteksti održivosti takav pristup zahtjeva visoku razinu discipline, jer bez jasno definiranih pravila, organizacije i odgovarajuće dokumentacija lako dolazi do neusklađenosti, ponavljanja koda i otežanog održavanja sustava. To je posebno važno u razvoju web korisničkog sučelja u kojem se komponente i stilovi brzo šire, te s vremenom postaje sve složenija za održavanje. Agilni pristup unutar sebe sadrži više načela i metoda, od kojih su neke Scrum, Kanban, ekstremno programiranje (eng. *Extreme Programming*, *XP*).



Slika 2: Agilni model (Izvor: slika je rekreirana prema slici sa `https://www.code-brew.com/agile-methodology/`, pristupljeno: 16.6.2026.)

2.3.3. Suvremeni pristup razvoju programskog koda

U posljednje vrijeme, u kontekstu pristupa prema razvoju projekta, sve se češće spominje paradigma programiranja bez pisanja programskog koda (eng. *Low-Code/No-Code*, *LC-NC*). Takvi se pristupi tradicionalno zasnivaju na razvoju aplikacija pomoću prilagođenih razvojnih grafičkih sučelja, no razvojem alata umjetne inteligencije (eng. *artificial intelligence*, *AI*) sve se više povezuje i s programiranjem pomoći prirodnog jezika. Takav se način rada opisuje engleskim pojmom *vibe coding*. Autor članka *Vibe coding: Revolucija u razvoju softvera ili prolazni trend*[7] Boris Plavljenić ističe da se popularnost ovog načina pisanja koda temelja na njegovoj pristupačnosti i činjenici da ga je lako razumjeti, nije potrebno učiti nove jezike i sintakse te se brzina razvoja drastično povećava. S druge strane, razvoj usmjeren na brzo postizanje rezultata koji radi bez unaprijed jasnog dugoročnog plana razvoja dovodi do eksponencijalnog povećanja tehničkog duga. Takav način rada može kratkoročno ubrzati implementaciju pojedinih funkcionalnosti, no istodobno može otežati dugoročnu održivost sustava [7]. Iako takav način rada ne mora nužno rezultirati nekvalitetnim kodom, pogotovo kada je riječ o suvremenim agentima umjetne inteligencije koji mogu uzeti u obzir širi kontekst sustava, održiv rezultat, kako Plavljanić u svom članku piše, i dalje zahtijeva visoku razinu discipline i duboko razumijevanje arhitekture i odluka donesenih tijekom razvoja te u konačnici napominje da ga je pogrešno promatrati kao zamjenu za tradicionalno programiranje [7] .

Metodologije razvoja i pristupi dizajnu predstavljaju okvir koji utječe na sve faze razvoja softvera, od početne analize do održavanja. U ovom radu promatraju se kao opći kontekst unutar kojeg nastaje i unutar kojeg se razvija programski kod. Ipak, naglasak završnog rada nije na

usporedbi metodologija, nego na primjeni načela održivosti unutar web korisničkog sučelja, no njihova je važnost u tome što izravno utječu na način planiranja, organizacije rada, donošenja odluka, te njihova pogrešna primjena i zanemarivanje aspekta održivosti može utjecati na lošu arhitekturu i manjak dokumentacije što dovodi do pošteškoća u razumijevanju programskog koda i mogućnosti njegovog unaprjeđenja [1, str. 7-6].

3. Načela održivog programskog koda

Do sada naglasak je bio na općim aktivnostima koje prethode pisanju koda, odnosno programiranju. U te aktivnosti spadaju odabir metodologije razvoja, načini pristupa dizajnu sustava te kako planiranje i organizacija rada smanjuju rizik kasnijih problema i tehničkog duga. U nastavku se fokus prebacuje na opće principe i prakse koje se primjenjuju tijekom pisanja programskog koda, a koje izravno utječu na njegovu čitljivost, razumijevanje, konzistentnost i dugoročnu održivost.

Načela održivog programskog koda su skup smjernica i praksi koje pomažu programerima u pisanju koda tako da je on lako razumljiv, održiv i prilagodljiv promjenama. Ova načela su univerzalna i primjenjiva u mnogim programskim jezicima i okruženjima, te zato što su univerzalno poznata ostalim programerima olakšavaju razumijevanje i rad na projektu.

3.1. Principi pisanja održivog programskog koda

3.1.1. Principi pisanja funkcija

Jedno od osnovnih programskih koncepata su funkcije. One su temeljni gradivni blokovi programskog koda, a njihova je osnovna funkcija organizacija i strukturiranje koda. Po svojoj definiciji, funkcije su imenovani blokovi koda koji izvršavaju određeni zadatak na poziv. Funkcije su ključne za održivost koda jer omogućuju ponovnu iskoristivost, posljedično smanjuju dupliciranje koda, te olakšavaju razumijevanje i preglednost. Funkcije bi trebale biti jednostavne, pregledne i usmjerene na rješavanje jednog zadatka pri čemu su sve 3 karakteristike povezane. Usmjerenost funkcije na jedan zadatak ima najveći utjecaj na ove 3 karakteristike, a pogotovo na jednostavnost. Određene funkcije znaju biti postavljene da rješavaju više zadataka do te mjere da same postanu program u programu te se kolokvijalno nazivaju božanske funkcije, odnosno na engleskom *god functions*. Takve funkcije postaju izvor tehničkog duga, jer se teško razumiju, traže greške unutar nje, te je postavljanje testa za njih gotovo pa nemoguće.

Zbog toga, Martin naglašava da funkcije trebaju biti male i usmjere na jednu jasno određenu zadaću [5, str. 34-36]. Poštivanjem ovih načela funkcije postaju stvarno jednostavne jer im je cilj riješiti jedan problem, te ako taj problem zahtjeva više koraka, oni se mogu organizirati u zasebne funkcije koje se pozivaju unutar glavne funkcije. Time programer više ne mora tumačiti kod koji rješava neki problem, nego mu naziv funkcije jasno govori koji problem on rješava. Do sada su načela održivog i čistog koda opisana kroz opće preporuke za oblikovanje funkcija i

organizaciju programskog koda. U nastavku se ta razmišljanja povezuju s konkretnim načelima i nazivima koji se često koriste u praksi i literaturi, poput DRY, AHA, KISS, YAGNI i razdvajanje odgovornosti.

3.1.2. Načela ponavljanja i apstrakcije

Prvo načelo, a kojoj realizaciji pomažu i same funkcije, jest načelo koje naglašava "nemoj se ponavljati" ili poznatije po akronimu DRY (eng. *Don't Repeat Yourself*) [8, str. 10] [6, str.43]. Ovo načelo naglašava da se ista logika ili struktura ne bi trebala ponavljati na više mjesta u kodu, zato što ponavljanje povećava mogućnost pogreške i otežava promjene, jer se ista promjena mora napraviti na više mjesta. U web korisničkom sučelja ovaj problem se javlja kada postoji ista HTML struktura definirana ispočetka na više mjesta, ili kada se isti CSS stilovi ponavljaju na više mjesta. Primjenom ovog načela, programer se usmjerava da izdvaja zajedničke dijelove u ponovno iskoristive komponente, pomoćne funkcije ili zajedničke stilove. Glavni princip koji se koristi u primjeni ovog načela je apstrakcija. Apstrakcija je proces izdvajanja zajedničkih karakteristika i funkcionalnosti u zasebne module, funkcije, komponente ili klase. Glavni cilj apstrakcije je učiti strukturu generičkom, odnosno što više općenitom tako da može pokriti više slučajeva. Tom se generičkom kodu kasnije dodaju, kroz atribute funkcije ili komponente, specifičnosti koje rješavaju konkretan problem.

Uz DRY načelo, potrebno je spomenuti i načelo nepotrebne ili suvišne apstrakcije poznato po akronimu AHA (eng. *Avoid Hasty Apstraction*). Ono upozorava na drugu krajnost DRY načela, a to je uvođenje apstrakcije "za svaki slučaj". Ovo načelo ne negira važnost ponovne iskoristivosti, nego naglašava da apstrakcije trebaju nastati tek kada su one stvarno potrebne. Ponekad dvije funkcije imaju sliučnu implementaciju, ali rješavaju različite probleme. U takvim slučajevima prerana apstrakcija može otežati razumijevanje i održavanje, jer programer mora prilagoditi ponašanje iste funkcije za različite slučajeve. Zbog toga je bolje pričekati dok se ne pojavi jasan i stabilan obrazac ponavljanja. To je obrazac u kojem se logika zaista ponavlja i ima istu svrhu u više dijelova sustava. Na taj način AHA načelo dopunjuju DRY načelo.

3.1.3. Načela jednostavnosti

Sljedeće načelo govori da kod bude što jednostavniji, a načelo ima akronim KISS (eng. *Keep It Simple, Stupid*). Kao što je već više puta bilo spomenuto, jednostavan kod lakše je razumjeti, testirati i mijenjati, a istovremeno smanjuje vjerojatnost grešaka. To bi značilo izbjegavanje kompliciranih i "pametnih rješenja" koja trenutno djeluju elegantno, ali u budućnosti ne

doprinose ničime te kod samo čine težim za razumijevanje. Primjeri gdje bi bilo dobro koristiti ovo načelo je pri spajanju više ternarnih operatora, lambda izraza ili predubokodog ugnježđivanja. Na primjernu spajanje više ternarnih operatora prikazano kodom 3.1, može se vidjeti kako spajanje više više ternarnih operatora u jednu izrazito dugu naredbu smanjuje čitljivost i razumijevanje. Alternativa je da se taj dio koda izdvoji u zasebnu funkciju koja kroz više provjera određuje povratnu vrijednost. Takav način pisanja lako je razumjeti jer kod postaje strukturiran unutar *if* blokova, te izdvajanjem koda u funkciju postaje samodokumentirajući u smislu da naziv funkcije opisuje njem zadatak. Na primjeru koda 2 mogu se vidjeti ugniježđeni upiti provjere (*if* upiti) kojima je teško odrediti mjesto završetka i koji *else* blok odgovora kojem *if* bloku. Taj se problem može riješiti korištenjem tzv. zaštitih klauzula (eng. *guard clauses*), koje ako su zadovoljene bacaju pogrešku u istoj liniji koda, čime se završava izvršavanje funkcije.

```
// Spajanje ternarnih operatora
const dohvatiUloguKorisnika = korisnik
    ? (korisnik.jeAdmin
        ? (korisnik.jeBlokiran ? "Admin(blokiran)" : "Admin")
        : korisnik.jeBlokiran ? "Korisnik(blokiran)" : "Korisnik")
    : "Gost";

// Alternativa
const dohvatiUloguKorisnika = dohvatiUloguKorisnika(korisnik)

function dohvatiUloguKorisnika(korisnik) {
  if (!korisnik) return "Gost";
  if (korisnik.jeBlokiran) {
    return korisnik.jeAdmin ? "Admin (blokiran)" : "Korisnik (blokiran)";
  }
  return korisnik.jeAdmin ? "Admin" : "Korisnik";
}
```

Programski kod 3.1: KISS primjer - grupiranje ternarnih operatora

```
// Duboko ugnježđivanje
function potvrdiNarudzbu(korisnik, kosarica) {
    if (korisnik) {
        if (cart) {
            if (kosarica.length > 0) {
                // Potvrdi narudžbu
            } else {
```

```
                console.log("Košarica je prazna.");
            }
        } else {
        console.log("Košarica ne postoji.");
        }
    } else {
        console.log("Korisnik nije prijavljen.");
    }
}

// Alternativa
function potvrdiNarudzbu(korisnik, kosarica) {
    if (!korisnik) throw new Error("Korisnik nije prijavljen.");
    if (!kosarica) throw new Error("Košarica ne postoji.");
    if (kosarica.length === 0) throw new Error("Košarica je prazna.");
    // Potvrdi narudžbu
}
```

Programski kod 3.2: KISS primjer - preduboko ognježđivanje

Sličan cilj ima i načelo YAGNI (eng. *You Aren't Gonna Need It*) [8, str. 11] koje upozorava da se ne rade stvari "za svaki slučaj", nego tek kada za njih postoji stvarna potreba. Primjer kršenja ovog načela bio bi kod izrade REST servisa koji mora preko svojeg sučelja nuditi rute za dohvaćanje (*GET*), dodavanje (*POST*) i brisanje (*DELETE*) elemenata, no programer za svaki slučaj rezervira rutu za ažuriranje (*PUT*) sadržaja koja nije dio zahtjeva i ne zna se hoće li ona u budućnosti biti potrebna.

3.1.4. Razdvajanje odgovornosti

Posebno važan princip u razvoju programskog koda kao i razvoju web korisničkih sučelja jest razdvajanje odgovornosti, odnosno na engleskom *Separation of Concerns (SoC)*. Martin u poglavlju o *Chapter 22: The Clean Architecture* napominje da je cilj svih arhitektura razdvajanje odgovornosti tako što se softver prelomi u slojeve [5, str. 202]. Ovaj princip ima svoje polazište u ideju da je jako korisnio da se fokusirano na jedan po jedan aspekt problema. Autori u knjizi *Architecting CSS - The Programmer's Guide to Effective Style* [8] ističu da je ovo jedan od osnovnih principa u razvoj arhitekture web sučelja u toj mjeri da se koriste različite tehnologije za svaku. Web arhitektura dijeli se na 3 osnovna dijela: sadržaj, izgled i ponašanje. Ovi dijelovi mogli su se vidjeti u poglavlju *2.1.2 Tehnologije u web razvoju* u kojem se spominju

3 osnovne tehnologije web razvoja [8, str. 6]. U HTML-u definira se sadržaj koji se oblikuje u CSS-u, te se ponašanje ubacuje pomoću JavaScript-a. Razdvajanje ne prestaje tu, nego se dalje proteže. Komponente su temelj modularnog web korisničkog sučelja, no ne postoji jedna vrsta komponenti. Komponente se mogu odvojiti na one koje služe za prikaz od onih koje služe za rješavanje poslovne logike. Osnovne komponente prikaza su komponente korisničkog sučelja (*UI components*), pa dalje dolaze kontejneri, moduli, servisi, pomoćni elementi i slično. Uloga i svrha ovih jedinica detaljnije će se objasniti u poglavlju *4.1 Modularna arhitektura korisničkog sučelja*.

Razdvajanje odgovornosti smanjuje ovisnost i povezanost između dijelova sustava, povećava mogućnost ponovne upotrebe, a testiranje postaje jednostavnije jer se pojedine cjeline mogu provjeravati zasebno. Ovaj kontekst može se povezati s pojmovima povezanosti (eng. *Coupling*) i kohezije (eng. *Cohesion*). Povezanost opisuje mjeru u kojoj jedan dio sustava ovisi o drugom dijelu sustava. Što je povezanost veća, to je promjena jedne komponente rizičnija jer može zahtijevati izmjene drugih komponenti. Zato autori u knjizi *Architecting CSS* [8, str. 7] preporučuju slabu povezanost jer je takav modul ponovno iskoristiv i promjene unutar njega neće imati utjecaja na ostale komponente, te autori ističu da je to važan atribut robusnog, odnosno izdržljivog i fleksibilnog sustava. S druge strane, kohezija opisuje koliko je jedna cjelina koda usmjerena na jednu odgovornost, odnosno koliko su zadaci koje obavlja međusobno povezani [8, str. 6]. Autori nisku povezanost povezuju sa dobrom, odnosno visokom kohezijom.

Prema tome, slaba povezanost i visoka kohezija izravno se nadovezuju na načelo jedne odgovornosti, odnosno eng. *Single Responsibility Principle (SRP)*, koje se je u prijašnjim poglavljima puno spominjalo. Uz načelo jedne odgovornosti, sljedećem poglavlju objasnit će se još 4 načela koja zajedno čine tzv. SOLID principe.

3.2. Objektno usmjereno oblikovanje programskog koda

SOLID principi su osnovni principi dizajn programskog koda unutar objektno orijentiranog programiranja (eng. *Object-Oriented Programming, OOP*). Njihove temelje postavio je Robert C. Martin, autor mnogih knjiga na temu programiranja, od kojih se neka citiraju u ovom radu: *Clean code* [6], *Clean architecture* [5]. Ovi principi služe kao smjernice u stvaranju softvera koji je održiv, robustan i lak za razumijevanje. Objektno orijentirano programiranje predstavlja paradigmu programiranja koja se temelji na klasama i objektima. Klasa opisuje strukturu i ponašanje određenog tipa objekata, odnosno definira atribute i metode koje će njezine instance sadržavati. Instanca klase naziva se objekt, te on posjeduje vlastite podatke i nad njima izvr-

šava unaprijed definirane operacije. Time se srodni podaci i funkcionalnosti grupiraju u logičke cjeline, što olakšava organizaciju i razumijevanje zato što je bliže stvarnom svijetu. Martin napominje da je objektno orijentirani dizajn temelj dobre arhitekture [5, str. 34], što posljednično dovodi do održivog i objektno usmjerenog oblikovanja programskog koda.

Objektno orijentirano programiranje temelji se na 4 osnovne ideje: učahuriavnje, apstrakcija, nasljeđivanje i polimorfizam. Učahurivanje (eng. encapsulation) omogućuje da se unutarnja implementacija pojedinog objekta sakrije od krajnjeg korisnika, odnosno ostatku sustava izlaže se samo ono što mu je potrebno. To se realizira kroz koncept vidljivosti elemenata klase koje mogu biti: javni, zaštićeni i privatni . Javni elementi dostupni su cijelom sustavu, dok s privatnim elementima može raditi samo klasa unutar koje su definirani. Zaštićeni elementi vidljivi su unutar klase koja ju je definirala i svim klasama koje ta klasa nasljeđuje. Sljedeća ideja je apstrakcija, te je njezin cilj izdvajanje bitnih karakteristika određenog dijela sustava, bez nepotrebnog opterećenja detaljima implementacije [5, str. 25.-37.].

Nasljeđivanje predstavlja mehanizam kojim klasa dijete može preuzeti svojstva i metode klase roditelj, te ih dalje može proširit i prilagoditi. Martin to preuzimanje opisuje kao ponovno deklariranje (eng. *redeclaration*) grupa varijabli i funkcija. Time se omogućuje ponovno korištenje postojećeg koda, smanjuje dupliciranje funkcionalnosti te posljedično se olakšava održavanje sustava. Način na koji se metode mogu proširiti i prilagoditi zove se polimorfizam. To je sposobnost da se neki isti postupak ili sučelje (eng. *interface*) primijeni nad objektima različitih klasa, s time da svaki objekt može reagirati drugačije, odnosno onako kako baš njemu to odgovara. Polimorfizmom se postiže fleksibilnost sustava te on omogućuje inverziju ovisnosti (eng. *dependency inversion*) [8, str. 37-45] o čijoj važnosti će kasnije biti više riječi.

3.2.1. Princip jedne odgovornosti - SRP

Akronim SRP (eng. *Single Responsibility Principle*) označava načelo prema kojem svaka klasa ili mogul treba imati samo jednu odgovornost, odnosno samo jedan razlog za promjenu. Martin ističe da SRP nije isto kao i princip da funkcija treba raditi jednu i samo jednu stvar, zato što se SRP fokusira na klase i module dok se drugi na funkcije. Isto tako ističe da je ime principa nedovoljno precizno te se tada lako miješa sa spomenutim principom za funkcije. Riječ odgovornost se ne odnosi na za što je funkcija odgovorna ili što radi nego na to kome je onda odgovorna ili čije potrebe treba zadovoljiti. Martin je u svojoj knjizi *Clean architecture* [5] dao definiciju tako da modul treba biti odgovoran jednom i samo jednom korisniku (eng. "*A module should be responsible to one, and only one, actor*"), pri čemu modul definira kao izvornu dato-

teku [5, str. 61-68]. Cilj ovog načela je postići jasniju podjelu zaduženja unutar sustava kako on bio jasniji za razumijevanje.

3.2.2. Princip otvorenosti/zatvorenosti - OCP

Akronim OCP eng. *Open/Closed Principle* koji označava da klase, moduli i ostali programski entiteti trebaju biti otvoreni za proširenje, a zatvoreni za izmjenu. Nove funkcionalnosti trebaju biti uvedene dodavanjem novog koda (proširenje) umjesto izmjenom postojećeg. Cilj je, Martin kaže, da sistem bude lak za proširenje bez velikog utjecaja na promjene, odnosno da trenutne funkcionalnosti rade isto kao i prije. Cilj se postiže razdvajanjem sustava u komponente i njihovom organizacijom u tako da promjene komponenta nižih razine ne utječu na komponente viših razina [5, str. 69-75].

3.2.3. Princip Liskovljeve zamjene - LSP

LSP (eng. *Liskov Substitution Principle*) označava načelo prema kojem objekti izvedene klase (klase dijete) moraju moći zamijeniti objekte bazne klase (klasa roditelj), ali da pri tome ne narušavaju ispravnost programa. To znači da naslijeđena klasa mora zadržati očekivano ponašanje osnovne klase te ne smije uvoditi promjene koje bi uzrokovale nepredvidljive rezultate, odnosno mijenjale značenje metoda, bali neočekivane pogreške ili slično. [5, str. 77-82]

3.2.4. Princip odvajanja sučelja - ISP

Slovo `I` u akronimu SOLID označava ISP (eng. *Interface Segregation Principle*). Ovaj princip govori da se klijentima ne bi trebali imati pristup metodama koje ne koriste, odnosno da prema njima se izlažu samo metode koje su im potrebne. Posljedično, bolje je imati više manjih i specifičnijih sučelja nego jedno veliko. Kada je sučelje ide previše u širinu, klase koje ga implementiraju su prisiljene definirati metode koje im nisu stvarno potrebne što povećava rizik nepotrebnih pogrešaka, smanjuje preglednost i povećava složenost sustava [5, str. 83-86].

3.2.5. Princip inverzije odgovornosti - DIP

Posljednje slovo u akronimu predstavlja DIP (eng. *Dependency Inversion Principle*) te ono označava načelo prema kojem moduli visoke razine ne bi smjeli ovisiti o modulima niske razine, nego bi oba dva trebali ovisiti od apstrakcijama. Martin u svojoj knjizi *Clean architecture : a craftsman's guide to software structure and design* [5] piše da su najfleksibilniji sustavi

oni kojima se ovisnosti izvornog koda odnose samo na apstrakcije, a ne na konkretne implementacije. Ovisnost o apstrakcijama je bitna zato što su one stabilnije, odnosno ako se radi promjena u implementaciji, apstrakcija najčešće ne zahtijeva promijene što ću čini stabilnijom. Isto tako napominje da je ovo jedno od osnovnih načela, te da dobar dizajner i arhitekt softvera nastoji pronaći načine da dodaje funkcionalnost implementaciji bez promijene sučelja. Time se omogućuje da poslovna logika sustava ne ovisiti izravno o konkretnoj implementaciji, odnosno omogućuje da baza podataka i korisničko sučelje budu dodaci (eng. *plugins*) na poslovnu logiku [5, str. 46, 87–91].

4. Arhitektura programskog rješenja

4.1. Modularna arhitektura korisničkog sučelja

Do sada se govorilo općenito o dobrim praksama, tehnikama i pristupima kako pisati održivi programski kod, a u nastavku fokus se prebacuje kako to primijeniti i implementirati unutar web aplikacije, točnije unutar web korisničkog sučelja. Unutar web korisničkog sučelja održivost se može očitovati kroz organizaciju datoteka i modularnoj arhitekturi kojoj je cilj razdvajanje odgovornosti između dijelova sustava. Opisat će se odabrani okvir Angular za izradu aplikacije te kako njegove karakteristike utječu na pojedine aspekte izrade web korisničkog sučelja.

Modularna arhitektura korisničkog sučelja podrazumijeva podjelu aplikacije na manje cjeline, koje su logički povezane. Te cjeline mogu odgovarati funkcionalnostima aplikacije koje se određuju unutar faze definiranja zahtjeva, a u fazi razvoja aplikacije one se nazivaju moduli. Moduli u kontekstu web korisničkog sučelja podrazumijevaju samostalne jedinice koje obuhvaćaju strukturu (HTML), prikaz (CSS), logiku (JavaScript). Modularna arhitektura počiva na razdvajanju i jasnom definiranju odgovornosti. Umjesto da postoji jedna datoteka koja je zadužena za dohvaćanje svih podataka koje aplikacija koristi, ta se jedna datoteka razdvoji na više njih od kojih je jedna zadužena za dohvaćanje izazova, druga za dohvaćanje podataka o napretku korisnika, a treća za dohvaćanje iskustvenih bodova. Ovakav pristup olakšava održavanje jer dohvaćanje izazova nema nikakvu povezanost s dohvaćanjem napretka korisnika te se time smanjuje mogućnost da jedan dio utječe na drugi. Isto tako je lakše pronaći i razumjeti neku specifičnu datoteku nego li određenu funkciju u općoj datoteci od nekoliko tisuća linija koda.

Nakon što je utvrđeno da modularna arhitektura doprinosi boljoj održivosti, postavlja se pitanja na koji se način moduli mogu organizirati unutar strukture projekta. Podjela i organizacija datoteka unutar projekta može se realizirati kroz dva osnovna pristupa. Prvi pristup se zasniva na tome da se datoteke grupiraju prema vrsti, te se taj pristup na engleskom naziva *type-first* pristup. Ovakva organizacija podrazumijeva grupiranje datoteka u mape po njihovoj svrsi. Svi servisi, odnosno datoteke zadužene za komunikaciju s poslužiteljskim dijelom aplikacije (eng. back-end) smjestile bi se u zasebnu mapu "servisi", svi preslikavači (eng. mapper) u svoju mapu, svi kontejneri i komponente u svoju mapu i tako za svaku tip datoteke. Ovakav pristup je dobar za manje projekte u kojima nemu puno datoteke, no kada je riječ o velikim projektima s na primjer puno servisa, otvaranjem te mape teško je pronaći željeni servis između njih 30. Druga opcije je da se datoteke grupiraju prema funkcionalnosti i funkcionalnim cjelinama (eng.

feature-first pristup). Unutar ove organizacije postojale bi mape s imenima funkcionalnosti, npr. "izazov", "napredak", ili "profil-korisnika". Unutar mape izazova nalazio bi se servis za dohvaćanje izazova pomoću API-a, preslikavač koji bi pretvorio odgovor u format koji se koristi unutar korisničkog dijela aplikacije (eng. front-end), te bi sve komponente potrebne za prikaz tih podataka bile unutar te mape. Ovaj pristup može se činiti prirodnijim jer se datoteke grupiraju onako kako korisnik doživljava sustav, te se bolje skalira na veće sustav jer se za nove funkcionalnosti kreira nova mapa i nema potrebe da se postojeće mijenjaju.

No uz datoteke koje pripadaju funkcionalnostima sustava pojavljuju se i one koje se koriste u više funkcionalnosti, odnosno isti kod se ponavlja. Primjer takvog koda bio bi opće komponente, pomoćne funkcije, servisi i konfiguracije koje se koriste na više mjesta unutar aplikacije. Kako je u prethodnim poglavljima bilo navedeno, takav je kod vođen apstrakcijom i ciljem ponovne iskoristive što je karakteristika kvalitetno napisanog koda. Zbog toga je uz mapu u koju se spremaju funkcionalnosti (eng. features), uvode mape poput "dijeljeno" (eng. shared) i "jezgra" (eng. core). Mapa "dijeljeno" namijenjena je za zajedničke elemente koji se ponavljaju u više funkcionalnosti kao što je navigacija, skočni prozori, pomoćne funkcije i slično. U mapu "jezgra" spremaju se najvažniji/temeljni dijelovi sustava i poslovne logike koji se koriste u radu cijele aplikacije. Primjer bi bili osnovni servisi, konfiguracija, modeli podataka (klase) i slično. Time se omogućuje razdvajanje specifičnih elemenata za pojedinu funkcionalnost od onih koji se koristi kroz cijelu aplikaciju.

Aplikacija koja će biti izrađena u sklopu ovog završnog rada bit će napravljenu u razvojnom okviru Angular. Kako je bilo spomenuto u poglavlju *2.1.2 Tehnologije u web razvoju*, okviri su nastali kako bi arhitektura web aplikacija dobila jasniju strukturu i smjer razvoja čime se poboljšava skalabilnost i održivost. Angular se od drugih razvojnih okvira razlikuje po njegovoj strožoj strukturi kojoj se programer mora prilagoditi, dok na primjer razvojni okvir *React* nudi puno više slobode u razvoju, no stroža pravila Angulara doprinose konzistentnosti i održivosti. Angular se temelji na komponentnom pristupu. Komponenta u Angulara sastoji se od 3 datoteke:

- HTML datoteka - ona služi za definiranje strukture sadržaja (eng. template)
- CSS/SCSS datoteka - ona služi da definiranje stilova sadržaja
- TypeScript datoteka - ona je osnova svake komponente, te je to klasa anotirana sa `@Component`. Unutar anotacije se navode informacije o oznaci komponente preko koje joj se pristupa (definira se preko `selector` (Kod: 4.1)), strukturi koja joj pripada (definira se preko `templateURL` (Kod: 4.1)), te koji se stil za tu strukturu koristi (definira se pre-

ko `styleUrls` (Kod: 4.1)). Ova klasa služi za obavljanje radnji, npr. pozivanje servisa, prosljeđivanje sadržaja za prikaz. [9, Poglavlje 8. Angular: The Beginning]

```
1 import { Component } from '@angular/core';
2 @Component({
3   selector: 'app-root',
4   templateUrl: './app.coponent.html',
5   styleUrls: ['./app.component.css']
6 })
7 export class AppComponent {
8   title = 'Moja aplikacija';
9 }
```

Programski kod 4.1: Primjer osnovne Angular komponente

Osim komponenti, odnosno klasa anotiranih s `@Component`, postoje i klase anotirane s `@Injectable` koje označavaju servise. Servisi služe za izdvajanje logike koja nije izravno vezana uz prikaz na korisničko sučelje (npr. dohvaćanje podataka). Angular usklađuje korištenje servisa i komponenta kroz automatsko ubrizgavanje ovisnosti (eng. dependency injection) [9]. Također, važno je razlikovati dva različita pojma: Angular i AngularJS. AngularJS označava stariju verziju (`1.x`) Angulara koja se bazira na običnom JavaScriptu, dok je Angular predstavlja novu verziju (`2.x`) koja se temelji na TypeScriptu[9, str. 6-7]. Aplikacija koja je dio ovog rada bit će izrađena u novijoj verziji. TypeScript je nadogradnja na običan (eng. vanilla) JavaScript koji nudi statičku tipizaciju i naprednije mogućnosti rada s objektima, na primjer proširuje mogućnosti JavaScripta dodavanjem sučelja (eng. Interface) [9, str. 134-137, 140–141]. TypeScript doprinosi održivosti jer smanjuje broj pogrešaka definiranjem tipa varijable kao: običan tekst, broj, niz ili nešto drugo. Tim definiranjem vrijednosti smanjuje se mogućnost neočekivane vrijednosti koja bi se u običnom JavaScriptu utvrdila tek u vrijeme izvršavanja programa. Isto tako definirani tipovi podataka djeluju kao dodatna dokumentacija koja pomaže u komunikaciji i razumijevanju svrhe koda. U konačnici TypeScript dataoteka se kompajlira u običnu JavaScript datoteku. [9, Poglavlje 7. TypeScript]

4.2. Ciljana struktura korisničkog sučelja

U nastavku definirat će se ciljana arhitektura korisničkog sučelja koja će biti primijenjena unutar praktičnog dijela rada. Definirat će se slojevi koji će se koristiti kao i njihove odgovornosti tako da su unaprijed zadane i poznate granice odgovornosti, odnosno koje datoteke služe za

prikaz podataka na ekran korisniak, gdje se podaci dohvaćaju ili gdje se obrađuju i pripremaju za prikaz. Cilj je postići čistu arhitekturu koja je održiva te da njezini dijelovi budu zamjenjivi.

4.2.1. Slojevi i njihove uloge

Arhitektura korisničkog sučelja sadržat će sljedeće elemente: komponenta, kontejner (spremnik), spremište stanja, sučelje servisa, konkretna implementacija servisa i preslikavač (eng. mapper) (Slika 3).

Najmanja cjelina korisničkog sučelja su komponente, te su one glavni dio sučelja s kojim korisnik ima kontakt. Komponente bi trebale biti zadužene samo za prikaz sadržaja na ekran, što znači da ne bi treble imati nikakvu logiku unutar sebe. Ona bi trebala biti što "gluplja". Kao što je u prethodnom poglavlju bilo spomenuto, u Angularu je komponenta klasa anotirana s `@Component`. Klasa sadrži informacije o HTML predlošku i CSS/SCSS stilovima te su ta 2 osnovna za komponentu. Sama klasa (TypeScript datoteka) ima zadaću prikazati podatke koje je dobila od nadređene klase ili emitirati događaj prema nadređenoj klasi, u ovom slučaju kontejneru. Primjeri komponenata bili bi: kartica izazova, navigacija, podnožje (eng. footer), forma i slično. [9, str. 427-428, 433]

Više manjih komponenti grupiraju se u veću komponentnu koja se naziva kontejner ili spremnik. Naziva se još pametna komponenta te je njena uloga upravljati s više manjih i "glupih" komponenti kako bi one činile jednu složnu cjelinu koja se u konačnici zove web stranica. Za razliku od komponenta, kontejner ima što je manje moguće prezentacijske odgovornosti te puno više logike. Kao prezentacijsku odgovornost ima prikaz komponenata, a što se tiče logike mora uzimati podatke iz spremišta stanja, prosljeđivati ih komponentama, Prisma događaje koje su komponente emitirale koje u konačnici pokreću metode u spremištu. Služi za transformaciju i početnu validaciju podataka koje korisnik unese preko forme. Tako bi kontejner unutar sebe imao komponente navigacije, kartice izazova, formu i podnožje što u konačnici čini jednu stranicu unutar aplikacije. [9, str. 427-428]

Spremište stanja je zaduženo za čuvanje stanja podataka koji se trebaju prikazi korisniku. Ono određuje hoće li se prikazi informacija da dohvaćanje/učitavanje podataka još traje, je li se dogodila pogreška u tom procesu ili su podaci uspješno dohvaćeni i spremi za prikaz. Glavna ideja je da servisi ostanu funkcijski orijentirani, te da ne pamte stanja i ne spremaju podatke, nego da to radi spremište. Spremište stanja također osigurava načelo jednog mjesta istine (eng. Single Source of Truth). Prema tom načelu neki dio stanja aplikacije nalazi se na jasno definiranom mjesto te se time izbjegava dupliciranje istih podataka u više komponen-

ta/kontejnera i smanjuje se mogućnost da se na jednoj stranici pokazuje jedan podataka, a na drugoj drugi podataka. Ono dohvaća podatke tako da injektira sučelje servisa i preko njega dohvaća ili mijenja podatke. Time spremište ne ovisi o konkretnoj implementaciji servisa, nego o apstrakciji. Na primjer, ako se korisnikovi iskustveni bodovi (eng. Experience points, XP) koriste u navigaciji na počtenoj stranici, stranici profila korisnika i na stranici prikaza izazova, ti se podaci ne bi trebali dohvaćati i spremati za svaki kontjner zasebno. Rješenje je da se napravi spremište stanja na primjer `spremišteStanjaNapredak` unutar kojeg će biti spremljeni podaci o iskustvenim bodovima koji će kontejneri čitati. Ako korisnik uspješno izvrši neki izazov i dobi 100 iskustvenih bodova, oni se samo promijene u spremištu te svi kontejneri prikazuju promijenjeni podatak iz spremišta umjesto da rade novi zahtjev prema poslužiteljskom dijelu aplikacije.

Sučelje servisa (na Slici3 označeno kao `IServis`) predstavlja granicu između aplikacije i vanjskog izvora podataka. Može se predočiti kao ugovor koji definira koje operacije aplikacija treba moću izvesti. Na primjer dohvati iskustvene bodove korisnika ili povećaj iskustvene bodove korisnika za 100, no sučelje ne određuje način na koji će se to izvesti. Važno je da spremište stanja ovisi o sučelju, a ne o konkretnoj implementaciji koja komunicira s poslužiteljem. Time se ostvaruje načelo inverzije ovisnosti (*DIP*) iz SOLID načela i ostvaruje slaba povezanost(eng. *loose/low coupling*). Prema tom načelu viši slojevi aplikacije ne bi trebali ovisiti o nižim slojevima, nego o apstrakcijama. Na taj način servis ne mora znati kako i od gdje se dohvaćaju podaci o iskustvenim bodove. To može biti preko REST API-a ili GraphQL-a ili na neki treći način, njemu je samo bitno da dobije podatke.

Može se dogoditi da se način komunikacije s vanjskim izvorom želi promijeniti s REST API-a na GraphQL. U tom slučaju nije potrebno mijenjati kontejner, spremište stanja niti metode koje pozivaju, nego samo izmijeni konkretnu implementacija sučelja koja je na Slici3 označena kao `Servis`. Unutar njega nalazi se konkretan kod koji se izvršava pri pozivu metode, na primjer dohvaća podatka o iskustvenim bodovima. Ako se kasnije želi uvesti drugi način dohvaćanja podataka, potrebno je samo promijeniti koja klasa implementira sučelje, dok ostatak aplikacije poput servisa, spremišta stanja i komponenti ostaje nepromijenjen. Servis u ovoj arhitekturi predstavlja konkretnu implementaciju komunikacije s vanjskim izvorom podataka. On koristi Angularov `HttpClient` za slanje zahtjeva prema API-u te po potrebi koristi preslikavač kako bi podatke dobivene od poslužitelja pretvorio iz DTO oblika u model koji se koristi u aplikaciji. Servis ne čuva stanje aplikacije, nego vraća podatke spremištu stanja koje odlučuje kako će ih spremiti i izložiti ostatku korisničkog sučelja.

U konačnici može se dogoditi da podaci nisu u formatu u kojem se koriste unutar ar-

hitekture korisničkog sučelja. Na primjer, REST API vraća podatke o iskustvenim bodovima devinom notacijom.

```
{
  "id": 1,
  "iskustevniBodovi": 430,
  ...
}
```

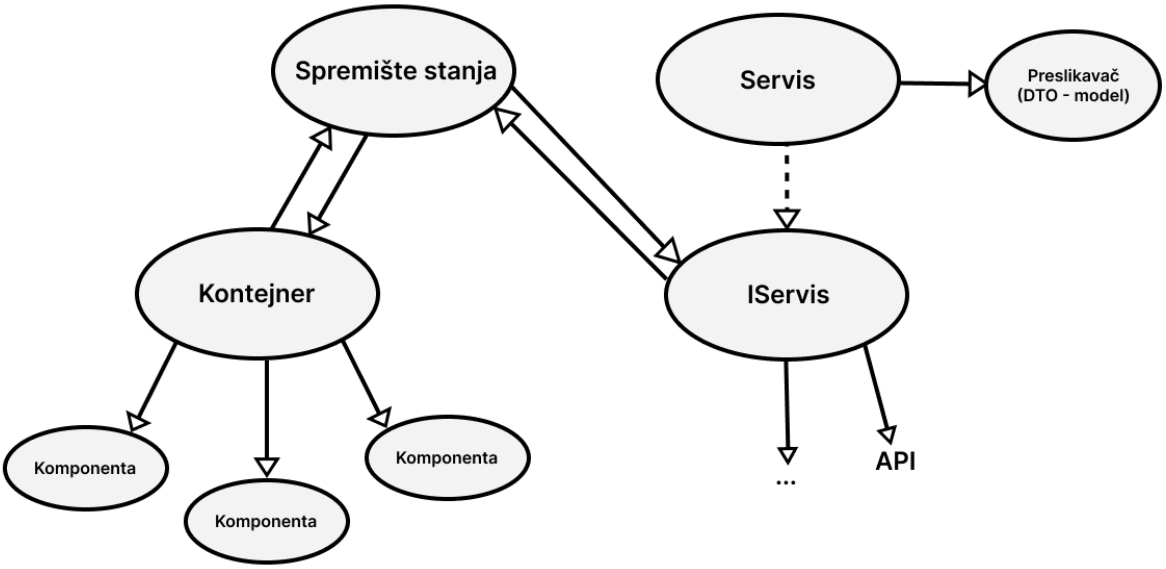
Programski kod 4.2: Primjer JSON objekta odgovora REST API-a

S druge strane, unutar aplikacije se isti podataka može prikazivati ili obrađivati na drugi način. Na primjer da se umjesto devine notacije koristi zmijina notacija ili se koristi drugi jezik ili je potrebno promijeniti format prikaza tako da se umjesto ukupnih 430 iskustvenih bodova, prikaže četvrta razina i 30 od 100 bodova za prelazak na sljedeću.

```
{
  "id": 1,
  "iskustevni_bodovi_razina": 4,
  "iskustevni_bodovi_trenutna_razina_bodovi": 30,
  "iskustevni_bodovi_sljedeca_razina_bodovi": 100,
  ...
}
```

Programski kod 4.3: Primjer modela podataka unutar aplikacije

U tom je slučaju potrebno uvesti preslikavač (eng. mapper) koji će prilagoditi dohvaćene podatke onima koji se koriste unutar aplikacije. Na ovom primjeru on je zadužen za pretvaranje zapisa iz devine u zmijsku notaciju, te prilagodbu podataka tako da izračuna trenutačnu razinu i trenutačno stanje bodova do sljedeće razine tako da ostatak aplikacija to ne mora računati.



Slika 3: Vizualni prikaz arhitekture korisničkog sučelja (Izvor: vlastita izrada)

4.2.2. Tok komunikacije

Tok komunikacije sustava kroz sve njegove dijelove izgledao bi ovako za primjer kada korisnik klikne na gumb za završetak izazova. Pritiskom na gumb, komponenta emitira događaj kontejneru koji poziva metodu spremišta stanja kojoj daje potrebne informacije o događaju. Pozivom metode u spremištu obrađuje se poziv te se poziva metoda za zabilježavanje izazova unutar sučelja servisa. Ovisno o implementaciji metode servus mijenja stanje u bazi te nakon promjene vraća odgovor. Ako je potrebno servis koristi preslikavač koji prilagođava odgovor modelu koji aplikacija koristi. Odgovor može biti broj iskustvenih bodova koji se sprema u spremište stanja. Kontejner dalje čita vrijednost varijable iskustvenih bodova korisnika i prosljeđuje ih komponentama koje taj podataka prikazuju.

4.2.3. Povezanost s ostalim arhitekturalnim obrascima

U kontekstu modeliranje arhitekture korisničkih sučelja često se spominju MVC (eng. Model-View-Controller) i MVVM (eng. Model-View-ViewModel) kao obrasci koji se baziraju na razdvajanju odgovornosti unutar aplikacije. Iako se moderni razvojni okviri poput Angulara i Reacta ne uklapaju strogo u jedan od njih, praćenje njihovih principa može služiti kao nit vodilja prema izgradnji kvalitetne arhitekture. U MVC obrascu pogled (eng. *View*) predstavlja prikaz sadržaja, odnosno ono što korisnik vidi, upravljač (eng. *Controller*) služi za obradu korisničkih akcija i upravljanja toka podataka, a model (eng. *Model*) predstavlja podatke i logiku. Prema ovim značajkama komponente su najsličnije pogledima, dok su kontejneri slični kontrolerima,

jer primaju događaje i određuju daljnji tok. MVVM obrazac posebno je primjenjiv u kontekstu web korisničkih sučelja, ističe Andreas Kagoshima u članku *UI Architecture Patterns. MVC and MVVM*[10]. MVVM naglašava pripremu podataka za prikaz i upravljanje stanjima kroz model pogleda (eng. *ViewModel*). On služi kao most koji izlaže podatke i nalazi se između pogleda i modela. Koncept *ViewModela* može se primijetiti kroz kombinaciju kontejnera i spremišta stanja. Spremište stanja čuva podatke, a kontejner ih prosljeđuje prema komponentama koje ih prikazuju. Servisi djelomično preuzimaju ulogu modela (eng. *Model*) zato što provode aplikacijsku logiku, zaduženi su za dohvaćanje, obradu i promjenu podataka, no ne mogu se u potpunosti usporediti s modelom u navedenim obrascima pošto modeli podataka nisu dio servisa i stanja se spremaju u spremište.

4.3. Angular koncepti za održivost korisničkog sučelja

Angular je okvir na strani klijenta koji unutar sebe sadrži kolekciju biblioteka i alata za pomoć u razvoju aplikacije. Autor knjige *A Journey to Angular Development* Sukesh Marla ga opisuje kao sveobuhvatan okvir za razvoj koji unutar sebe sadrži ponovno iskoristive biblioteke koje čine razvoj jednostavnijim, ali ga i otežavaju u drugim stvarima. [9, str. 13]. U ovom poglavlju će se obraditi aspekti Angular okvira koji služe kao pomoć u pogledu održivosti aplikacije.

4.3.1. Komponente kao temelj održive izgradnje sučelja

Angular nije pionir što se tiče komponentno orijentiranog dizajna, no ovaj novi način pristupa izgradnji korisničkog sučelja je temelj tog okvira. Kao što je već spomenuto, ovaj način pristupa dizajnu bazira se na tome da je korisničko sučelje kompozicija malih i ponovno iskoristivih komponenti [9, str. 450]. Osnovni koncpet za rad s komponenta su JavaScript moduli. Oni su standardizirani kao dio ES6 verzije JavaScripta te oni omogućuje izvoz (eng. export) varijable, funckije ili klase, te korištenje te vrijednosti u drugim datotekama kao uvezene (eng. import) varijable, funkcije ili klase. Koriste se tako da se ispred vrijednosti (varijabla, funckija, klasa) koja se želi izvesti stavi ključna riječ `export` (Programski kod 4.4) te se na početku datoteke u kojoj se želi napraviti uvoz preko ključne riječi `import` navede naziv izvedene vrijednosti. U nastavku potrebno je još samo navesti iz koje se datoteke vrijednost uzima (Programski kod 4.5) [9, Poglavlje 5. ES6 Features - Part 4].

```
1 // izracunajRazinuDatoteka.js
2 export function izracunajRazinu(iskustveniBodovi) {
3   return Math.floor(iskustveniBodovi / 100);
```

```
}
```

Programski kod 4.4: Primjer izvoza funkcije iz modula

```
import { izracunajRazinu } from './izracunajRazinuDatoteka.js';
const razina = izracunajRazinu(430);
```

Programski kod 4.5: Primjer uvoza funkcije iz modula

Angular aplikacija je strukturno gledano stablo komponenata (Slika 4). Ovo stablo počinje korijenskom (eng. root) komponentom koja uključuje druge komponente koje unutar sebe uključuje treće.



Slika 4: Stablo Angular komponenata (Izvor: vlastita izrada)

Takav se odnos komponenta može gledati kao odnos roditelj-dijete te u tom odnosu roditelj zna koju djecu on ima (uključuje), a djeca ne znaju koji ih roditelj uključuje. Sukesh Marla u knjizi *A Journey to Angular Development* komponentu roditelja naziva vanjska (eng. outer) komponenta, dok komponentu djeteta naziva unutarnja (eng. inner) komponenta. Vanjska komponenta prosljeđuje podatke unutarnjoj tako da unutarnja komponenta ima atribut klase označen/dekoriran kao ulazni atribut (eng. Input property) preko dekoratora `@Input()` (Programski kod 4.8) [9, str. 434]. Roditeljska komponenta zatim prosljeđuje vrijednost preko predloška pomoću povezivanja atributa (eng. property binding) (Programski kod 4.7). Unutar uglatih zagrada upisuje se atribut dekoriran kao ulazni te se njemu pridružuje vrijednost iz roditeljske klase (Programski kod 4.6). Nakon toga komponenta dijeta pristupa proslijeđenoj vrijednosti

preko ulaznog atributam u ovom primjeru `iskustveniBodoviInput`.

```
<!-- roditelj.componente.ts -->
...
export class KarticaKorisnikaKomponenta{
    iskustveniBodoviRoditelj:number = 430;
}
```

Programski kod 4.6: Primjer klase roditelja

```
<!-- roditelj.componente.html -->
<app-kartica-korisnik [iskustveniBodoviInput]="iskustveniBodoviRoditelj"></
   app-kartica-korisnik>
```

Programski kod 4.7: Primjer povezivanja atributa roditelja prema dijetetu

```
<!-- app-kartica-korisnik.componente.ts -->
...
export class KarticaKorisnikaKomponenta{
    @Input() iskustveniBodoviInput!: number;
}
```

Programski kod 4.8: Primjer dekoratora `@Input()`

Sličan, ali malo kompliciraniji postupak je za obrnuti smjer, odnosno kada dijete želi komunicirati s roditeljom. U komponenti djeteta potrebno je željeni atribut dekorirati s dekoratrom `@Output()`. Željenom atributu potrebno je dodijeliti vrijednost instance klase `EventEmitter`. Objekt te klase ima metodu `.emit()` kojoj se prosljeđuje željena vrijednost [9, str. 436]. Metodom `emit` se izvršava na neki događaj čime djete šalje podatke roditelju. Na primjeru Koda 4.9 događaj bi bio poziv funkcije `odaberiKorisnika`. Kada roditelj očekuje vrijednost od djeteta, pri postavljanju predloška (Programski kod 4.10) potrebno je unutar običnih zagrada staviti vrijednost koju dijete šalje te joj pridružiti metodu klase roditelja (Programski kod 4.11) koja će obraditi taj događaj odnosno prosljeđenu vrijednost.

```
<!-- app-kartica-korisnik.componente.ts -->
...
export class KarticaKorisnikaKomponenta{
    @Output() korisnikOdabranOutput = new Event Emitter<number>();

    odaberiKorisnika():void{
        this.korisnikOdabranOutput.emit(1);
    }
```

```
}
```

Programski kod 4.9: Primjer dekoratora `@Output()`

```
<!-- roditelj.componente.html -->
<app-kartica-korisnik (korisnikOdabranOutput)="obradiOdabranogKorisnika(
    $event)"></app-kartica-korisnik>
```

Programski kod 4.10: Primjer primanja vrijednosti dijeteta

```
<!-- roditelj.componente.ts -->
export class RoditeljComponent {
  obradiOdabirKorisnika(korisnikId: number): void {
    ...
  }
}
```

Programski kod 4.11: Primjer obrade proslijeđene vrijednosti

4.3.2. Angular servisi

U prijašnim poglavljima se govorilo i o servisima te kako oni služe za upravljanje poslovnom logikom, a u Angularu su servisi klase anotirane sa `@Injectable`. U servisima se centralizira komunikacija sa REST servisima te se time rasterećuju komponente koje više ne moraju voditi brigu o toj komunikaciji. Takav centralizirani pristup je dobar jer svaka zasebna komponenta više ne mora definirati svoj način komunikacije, nego se pozove metoda iz servisa koja komponenti vraća odogovor [11, Dependency Injection].

4.3.3. Spajanje dijelova sustava

Kako bi se servisi mogli koristiti unutar komponenti, Angular mora znati kako ih stvoriti i gdje ih treba dostaviti da se mogu koristiti. Servisi su obično TypeScript klase označene dekoratorom `@Injectable`. Ovim dekoratorom klase postaju dostupne za sustav ubrizgavanja/injektiranja ovisnosti (eng. Dependency Injection, DI). Inače bi svaka komponenta trebala ručno instancirati novu instancu servisa kada joj je potrebana, no Angular to radi centralizirano i sam upravlja stvaranjem i ubrizgavanjem u klase koje trebaju te servise. Tako komponenta mora samo navesti ovisnost koju želi koristiti. Ovisnost se u servisu registrira unutar dekoratora preko `providedIn:'root'`, a kada komponenta treba neki servis on se navodi preko

`providers` atributa unutar `@Component` dekoratora. `Providers` Prisma niz tako da se može koristiti više od jedne ovisnosti [9, str. 279-288] [11, Dependency Injection].

4.3.4. Jednostranične aplikacije i navigacija

Angular omogućuje izgradnju jednostraničnih aplikacija (eng. Single Page Applications). Njihova karakteristika je da krajnji korisnik ima osjećaj kao da je uvijek jednoj stranici, a sadržaj stranica se mijenja. To je moguće tako da se inicijalno dohvaća početna stranica, te su sve naredne promijene jednostavne manipulacije stablom elemenata (eng. Document Objekt Model, DOM). Neki elementi nestaju, a novi se stvaraju no stranica se nikad ne osvježava te se time stvara osjećaj da je korisnik uvijek na jednoj stranici, no u pozadini aplikacija ima više stranica [9, str. 10] [11, Routing]. Način na koji Angular to radi jest tako da na poziv neke rute (eng. route) ne vraća novi HTML dokument nego samo prikazuje odgovarajuću komponentu unutar aplikacije. Te rute nisu obične rute, nego se one odnose na objekte koji definiraju komponente koji se moraju prikazati za specifičnu URL putanju ili uzorak. Te rute se mogu definirati pomoću modula `app-routing-module.ts` koji omogućuje `RouterModule`. Unutar tog modula definira se ruta i za nju se definira koja se komponenta mora prikazati (Programski kod 4.12). Za prijelaz s jedna "stranice", odnosno rute, na drugu više se ne koristi HTML atribute `href` nego atribut `routerLink` kojem se proslijedi putanja (eng. path) definirana u modulu (Programski kod 4.13). U konačnici potrebno je definirati na kojem mjestu će se komponenta prikazati. Za to služi posebni Angular HTML element `router-outlet` unutar kojeg će se prikazati komponenta za navedenu rutu (Programski kod 4.13) [9, Poglavlje 23. Angular Routing] [11, Routing].

```
<!-- app-routing-module.ts -->
const routes: Routes = [
  { path: 'pocetna', component: PocetnaComponent },
  { path: 'izazovi', component: IzazoviComponent },
  { path: 'profil', component: ProfilComponent }
];
```

Programski kod 4.12: Primjer definiranja ruta

```
<nav>
  <a routerLink="/pocetna">Početna</a>
  <a routerLink="/izazovi">Izazovi</a>
  <a routerLink="/profil">Profil korisnika</a>
</nav>

```

```
7 <router-outlet></router-outlet>
```

Programski kod 4.13: Primjer korištenja atributa routerLink

4.3.5. Grupiranje elemenata

Do sada su se spomenuli mnogi elementi poput komponenata, servisa, spremišta stanja i drugih. Dodavam nove funkcionalnosti ovi svi elementi bi se trebali napraviti ponovno, što dovodi do množenja elementa koji se moraju pratiti unutar strukture. Angular riješava taj problem tako da omogućuje grupiranje i organizaciju elemenata u logičke i funkcionalne cjeline pomoću `NgModule`. Marla u svojoj knjizi *A Journey to Angular Development* [9, str. 481] `NgModule` opisuje kao učahuriavnje (eng. encapsulation) komponenata. Ovakav pristup omogućava smanjno mijenjašanje odgovornosti i omogućava ponovnu iskorisitvnost zajedničkih dijelova. Na primjer, sve funkcionalnosti vezane uz izazove nalaze se unutar jednog modula, vezane uz lokacije unutar zasebnog te u konačnici zajedničke komponente poput gumba i kartica se nalaze u zajedničkom modulu.

4.4. Arhitektura i organizacija koda na strani poslužitelja

Do sada se govorilo o samo jednom dijelu aplikacije, a to je dio koji je zadužen za prikaz podataka korisniku i ovo kojega se ovaj završni rad bazira te se ta strana aplikacija naziva klijentska strana. S druge strane nalazi se poslužiteljska strana (eng. back-end) kojoj je cilj pružiti informacije koje će klijentska strana prikazati. Fokus ovog rada nije na poslužiteljskoj strani, no potrebno ju je spomenuti i opisati kako u osnovi ona radi.

4.4.1. Izvođenje JavaScript koda van preglednika

JavaScript je napravljen prvenstveno za korištenje na klijentskoj strani, odnosno izvođenju u web pregledniku. Razvojem interneta i novim zahtjevima razvijeno je izvršno okruženje (eng. run-time environment) koji omogućuje izvršavanje JavaScript koda van web preglednika. To je otvorilo nova vrata u razvoju tako da se sada mogu razvijati i desktop, odnosno izvorne (eng. native) aplikacije u JavaScriptu kao i serveri koji mogu odgovoriti na HTTP zahtjeve. To okruženje zove se Node.js [9, str. 13-17].

Kako bi izrada servera bilo što jednostavnija, razvio se je web okvir Express koji olakšava izradu web servera. Ovaj okvir pojednostavljuje rad s Node.js modulima čime definiranje

ruta, obrada HTTP zahtjeva i odgovora postaje jednostavnija.

4.4.2. NestJS

Express je fleksibilan okvir koji programeru daje puno mogućnosti i slobode u radu, no kao što je prije bilo spomenuto, rastom aplikacija teško je držati dosljednost i konzistentnost pogotovo kada više ljudi radi na istoj stvari. Potrebnu strukturu za velike projekte, na strani klijenta, dali su okviri Angular, React, Vue i ostali. Na strani poslužitelja razvili su se posebni okviri za svaki od navedenih okvira na strani klijenta. Tako se za Angular razvio okvir NestJS na strani klijenta, za React se razvio NextJS, a za Vue NuxtJS. Pošto se za dio aplikacije na strani klijenta odabrao Angular, za stranu poslužitelja se odabrao njegov okvir NestJS. Ovakav pristup dovodi do dosljednosti unutar aplikacije zato što su koncepti NestJS-a slični Angularovim konceptima. Tako se NestJS bazira na TypeScriptu, arhitektura se temelji na modulima (eng. Modules) za grupiranje, servisima (eng. Providers) za definiranje poslovne logike i kontrolerima (eng. Controller) za definiranje ruta API-a te koristi isti koncept ubacivanja ovisnosti. Iz tih razloga se NestJS koristi kao rješenje za izradu API-a (eng. Application Programming Interface) koji služi za komunikaciju između dva dijela na poslužiteljskom dijelu aplikacije.

5. Oblikovanje i organizacija stilova

Kao što je u poglavlju *2.1.2: Tehnologije u web razvoju* spomenuto, za oblikovanje sadržaja postavljenog preko HTML-a koristi se deklarativni jezik CSS (eng. Cascading Style Sheets). Za razliku od imperativnih jezika kojima se korak po korak opisuje kako se nešto treba napraviti, pomoću deklarativnih jezika se opisuje što se želi postići. Tako se preko CSS-a nekom elementu može dodati boja, razmak ili obrub. Najnovija verzija CSS jest CSS3 te on ne predstavlja jednu specifikaciju, nego je modularan i postepeno se nadograđuje. Peter Gasston, autor knjige *Moderni Web : responzivni Web dizajn uz HTML5, CSS3 i JavaScript* [12], CSS3 opisuje kao kraticu za sve najnovije od CSS 2.1 [12, str.17]. Između ostalog, CSS3 je uveo mogućnost izrade responzivnih stranica pomoću medija uputa (eng. media queries) i definiranje rasporeda elemenata uzimajuću u obzir mogućnosti korisničkog agenta u kojem se prikazuju [12, str.17].

Glavna karakteristika CSS-a jest kaskadnost. Hrvatski prijevod punog imena *Cascading Style Sheets* jest kaskadni stilski listovi. Ključna riječ ovdje je kaskada, koja označuje prelijevanje. Hrvatska enciklopedija (`https://www.enciklopedija.hr/clanak/kaskada`) definira kaskadu kao: "kaskada, slap, vodopad koji pada stepenasto". U kontekstu CSS-a kaskada opisuje način primjenjivanja stilova na element. Na jedan element mogu utjecati više stilova, od kojih se jedan može nalaziti u CSS datoteci, a drugi može biti definiran kao stilski (eng. style) atribut unutar HTML elementa. Način na koji preglednik određuje koji će se stil primijeniti jest prema pravilima kaskade kojima se određuje važnost, izvor, specifičnost selektora te redoslijed u kojem se pojavljuje. Bitno je za napomenuti da se ne primjenjuje samo jedno stilsko pravilo, a da se ostala zanemaruju, nego preglednik trenira svako CSS svojstvo zasebno i zasebno ga primjenjuje. Pravilo najvišeg prioriteta jest pravilo važnosti, a to je svojstvo koje je označeno kao *!important*. Time se gotovo garantira da će se to svojstvo primijeniti. Ako takav stil nije definiran, preglednik gleda specifičnost selektora. Najveću vrijednost imaju stilovi pisani kao atribut unutar HTML elementa (eng. inline style) te je njihova vrijednost tisuću (1000). Zatim slijede identifikatori (unutar HTML elementa označeni kao `id` atribut) koji vrijede sto (100), pa zatim slijede klase (unutar HTML elementa označeni kao `class` atribut) s deset (10) te na posljetku dolazi selektor elementa/tipa s vrijednošću jedan (1) [8, str. 67-70]. Selektori se dalje mogu kombinirati tako da se spajaju razmakom, znakovima plus (+), tilda (~) i znakom "više od" (>).

Kombinacijom selektora vrijednosti se zbrajaju, tako da bi vrijednost za ovaj selektor `#kartica .kartica-sadržaj .tekst span` (Programski kod 5.2) bila: `100 + 10 + 10 + 1 = 121`, a vrijednost za `span#crvena-boja` (Programski kod 5.2) bila bi: `1 + 100`

= 101. Zato što je vrijednost 121 veća od 101, tekst CSS unutar span (Programski kod 5.1) oznaka bit će zelene boje, ali bit će i podcrtan jer je i to pravilo za ovaj tekst koje nije nadjačano.

```
<div id="kartica">
  <div class="kartica-sadrzaj">
    <p class="tekst">
      Ovo je primjer zbrajanja <span id="crvena-boja">CSS</span> selektora.
    </p>
  </div>
</div>
```

Programski kod 5.1: Primjer zbrajanja selektora - HTML

```
#kartica .kartica-sadrzaj .tekst span{
  color: green;
}
span#crvena-boja{
  color: red;
  text-decoration: underline;
}
```

Programski kod 5.2: Primjer zbrajanja selektora - CSS

U konačnici ukoliko su vrijednost stilova iste prednost dobiva stil koji je definiran kasnije unutar .css datoteke. [8, Poglavlje 3. Order Of Importance]

5.1. Principi pisanja održivih stilova

5.1.1. Problemi kaskade

Iako kaskada daje mnoge dobre i korisne mogućnost, kriva primjena tih mogućnosti može prouzrokovati drastične probleme od kojih je najveći eskalacija specifičnosti. Primjer te eskalacije jest Programski kod 5.2 u kojem ima puno kombiniranja selektora, odnosno oni postaju duboko ugniježđeni. Dobiva se velika specifičnost koju je teško nadjačati kada je to stvarno potrebno, te je teško pratiti put kojim je ona primijenjena. Zatim se poseže za jednostavnim i brzim rješenjima tako da se koristi !important, koji je dalje još teže nadjačati. Takav način razvoja generira veliki tehnički dug, kod postaje teško razumljiv, nepredvidljiv i svaka promjena postaje sve teža za primijeniti.

Iz tih razloga bitno se pridržavati pravila kojima se zaobilaze problemi kaskade. Jedno

od pravila jest da primjena stilova ne ovisi o lokaciji elementa unutar stabla, odnosno situaciju u kojoj isti element dobiva potpuno drugačije stilove ovisno o tome gdje se nalazi unutar hijerarhije elemenata (*DOM*) (kasnije se spominje kao razdvajanje spremnika/kontejnera od sadržaja, poglavlje 5.1.2) ili ako se nalazi na drugim stranicama. Cilj je pisati konkretne stilove za koje programer zna što može očekivati uz minimalan utjecaj nadjačavanja. Također je bolje dodavati više kompleksnosti unutar HTML-a pošto je on sam po sebi jednostavniji i intuitivniji za razumjeti. Cilj povećavanjem kompleksnosti HTML-a jest smanjenje kompleksnosti CSS tako da se mogu pisati konkretniji stilovi. To bi obuhvaćalo da nazivi klasa budu jasniji, da se umjesto jedno velike klase koriste više manjih čime se povećava razumijevanje. Više o toj temi navedeno je u poglavlju *5.1.3 BEM konvencija imenovanja*. Bolje je napraviti novu klasu, nego se oslanjati na kombinaciju selektora kako bi se dohvatio određeni element.

Dobra praksa je izbjegavanje korištenja identifikacijskih selektora (`id`) za potrebe oblikovanja i pisanja stilova. Oni imaju veliku specifičnost, teško ih je nadjačati te dovode do tzv. "ratova specifičnosti". Identifikatori imaju svoju svrhu unutar skripti i navigacije pomoću sidra (eng. anchor), te bi se oblikovanje trebalo prepustiti klasama. Također bi se trebala izbjegavati oznaka važnosti `!important` zato što je onda nepredvidljiva. Na primjer ako se želi neko svojstvo nadjačati, programer napravi klasu sa svojstvom crvene boje teksta i primjeni klasu na element. Na stranici tekst je ostao zelene boje zato što je 300 linija iznad te klase svojstvo za zelenu boju tog teksta označeno s oznakom `!important`.

Primjenom samo ovih pravila, stilske datoteke postaju jasne, čišće, preglednije te lakše za održavati.

5.1.2. Objektno orijentirani CSS: OOCSS

Iz prije definiranih pojmova objektno orijentirani CSS može zvučati čudno pa možda i nemoguće. Objektno orijentirano programiranje se bazira oko objekata i klasa, dok u CSS ne postoje ni objekti ni klase. Objektno orijentiran CSS odnosi se primjenu koncepata iz objektno orijentiranog dizajna tako da se CSS-u doda struktura, a objektnom se smatra ponavljajuću vizualni uzorak koji se može apstrahirati u zasebni isječak HTML-a, CSS-a i JavaScript-a. Taj se objekt tada može koristiti više puta unutar aplikacije [8, str. 209]. Cilj je isporučiti fleksibilne, modularne i zamjenjive komponente, a OOCSS je to objedinio kroz dva pravila: odvojiti strukturu od izgleda (eng. separate structure from skin) i odvojiti spremnik od sadržaja (eng. separate container from content).

Kod prvog pravila, struktura (eng. structure) ili raspored (eng. layout) se odnosi na lo-

kaciju elementa na stranici. To su CSS svojstva koja utječu na veličinu i poziciju elementa, kao što su: `height`, `width`, `margin`, `padding` i sl. Izgled (eng. skin) ili tema (eng. theme) odnosi se na vizualni aspekt elementa kao što je boja, obrub (eng. border), providnost (eng. opacity), sijena (eng. box-shadow) i sl. Razdvajanje ova dva elementa označava razdvajanje jedna klase na više njih. Struktura bi trebala ići u jedna klasu, izgled u drugu [8, str. 211].

U Programskom kodu 5.3, u prvom primjeru može se vidjeti klasa koja miješa strukturu i izgled. Time se smanjuje mogućnost ponavljanje klase, jer ako bi se on htjela primijeniti uvjeti za struktura i izgled trebaju biti zadovoljeni, odnosno jednu je strukturu teško koristiti s drugim izgledima i obrnuto. U drugom primjeru struktura i izgled su izdvojene u zasebne klase te se time izgled i struktura mogu primjenjivati neovisno jedan o drugome.

```
/* Kršenje razdvjanje struture od izgleda */
.kartica {
  /* struktura */
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  padding: 24px;
  width: 320px;

  /* izgled */
  background-color: #f5f5f5;
  font-family: Arial, sans-serif;
  color: #222222;
}

/* Ispravna primjena */
/* struktura */
.kartica-struktura {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  padding: 24px;
  width: 320px;
}

/* izgled */
.kartica-izgled {
```

```
28    background-color: #f5f5f5;
29    font-family: Arial, sans-serif;
30    color: #222222;
31  }
```

Programski kod 5.3: Primjer odvjanje strukture od izgleda

Drugo pravilo, razdvajanje spremnika od sadržaja, ističe da stilovi ne bi trebali ovisiti o lokaciji elementa, nego o njegovim atributima. Primjer Programskog koda 5.4 preuzet je iz knjige *Architecting CSS* [8, str. 212, Listing 9-3. OOCSS Separate Container from Content] zato što jako dobro prikazuje prednosti tog pravila. Autori Martine i Michael Dowden [8, str. 212] ističu da ovo pravilo dovodi do bolje konzistentnosti i održivosti zato što:

- gumb izgleda isto neovisno o lokaciji (konkretno nalazi li se u unutar roditelja s klasom `.hero`)
- svi elementi s klasom `poziv-na-akciju` izgledat će isto bez obzira na lokaciju ili vrstu elementa (eng. tag)
- gledajući HTML, lako je identificirati koja CSS svojstva utječu na taj element
- specifičnost ostaje mala što olakšava nadjačavanje kada je to potrebno

Neko će možda istaknuti da je za neke situacije nepotrebno napraviti novu klasu, što može biti slučaj, no kako je navedeno u *5.1.1 Problemi kaskada* bolje je dodati više kompleksnosti u HTML, u ovom slučaju više klasa na jedan element zato što time CSS postaje razumljiviji, semantika klasa postaje jasnije što u konačnici dovodi do bolje održivosti.

```
1  /* Zadana vrijednost */
2  button{
3    background-color: lightblue;
4  }
5
6  /* Dobra primjena OOCSS */
7  .poziv-na-akciju{
8    background-color: lightgreen;
9    }
10
11 /* Loša primjena */
12 .hero button{
```

```
13    background-color: lightgreen;
14  }
```

Programski kod 5.4: Primjer odvjanje spremnika od sadržaja

5.1.3. BEM konvencija imenovanja

BEM je skraćenica za blok (eng. block), element (eng. element) i modifikator (eng. modifier) te predstavlja konvenciju imenovanja klasa. U knjizi *Architecting CSS* [8, str. 215] ističe se da je cilj ove konvencije izravnati strukturu unutar blok elementa tako da se CSS ne mora mijenjati ako se HTML struktura promijeni. Još jedan cilj je klase učiti predvidljivima, konzistentnijima i lakšima za razumjeti tako da se daju opisna imena. Blok predstavlja samostalnu, ponovno iskoristivu komponentu, dok je element dio bloka. U knjizi, razlika između bloka i elementa svodi se na:

- može li ponovno iskoristiti, ako da onda je blok
- ako se dio koda ne može koristiti izvan roditeljskog spremnika, onda se radi o elementu.

Modifikator označava varijantu ili stanje elementa kome je pridodan [8, str. 212-215]. Dijelovi konvencije spajaju se tako da se ispred elementa stavi dvostruka dolja crta (eng. underline), a ispred modifikator dva znaka minus (eng. dash). Imenovanje izgleda ovako:

- `.blok`
- `.blok__element`
- `.blok--modifikator`
- `.blok__element--modifikator`

Dodatna prednost ove konvencije je u kombinaciji sa SCSS-om zato što se dijelovi mogu spajati i prikazivati u hijerarhiji pomoću znaka `&`.

S primjera Programskih kodova 5.5 i 5.6 može se vidjeti primjer primjene BEM konvencije imenovanja. Kao blok identificiran je navigacija zato što se ona može ponovno iskoristiti na više stranica. Element je pojedina poveznica unutar navigacije zato što je onda dio navigacije i nema ju prevelike logike izdvajati van nje. U konačnici modifikator `--aktiva` dodijeljena je poveznici trenutačne stranice na kojoj se korisnik nalazi. Na kraju tog primjera prikazan je zapis

kako bi izgledao unutar pretprocesora SCSS korištenjem znaka `&`. Ovaj znak je ključan zašto što se prilikom kompajliranja SCSS-a u CSS takav zapis pretvara u čistu BEM notaciju. Ako se ovaj znak ne bi koristio, tražilo bi se element s klasom `__link` unutar elementa s klasom `navigacija`, a ne element s klasom `navigacija__link`.

```
<nav class="navigacija">
  <a class="navigacija__link navigacija__link--aktivna" href="#">Početna</a
   >
  <a class="navigacija__link" href="#">O nama</a>
</nav>
```

Programski kod 5.5: Primjer BEM konvencije imenovanja klasa - HTML

```
.navigacija {
  display: flex;
  gap: 1rem;
}
.navigacija__link {
  color: black;
  text-decoration: none;
  padding: 1rem;
}
.navigacija__link--aktivna {
  color: red;
  text-decoration: underline;
}

// Ugniježđivanje pomoću pretprocesora (npr. SCSS):
.navigacija {
  display: flex;
  gap: 1rem;

  &__link { //Kompajnirani CSS: .navigacija__link
    color: black;
    text-decoration: none;
    padding: 1rem;

    &--aktivna { //Kompajnirani CSS: .navigacija__link--aktivna
      color: red;
      text-decoration: underline;
```

```
28      }
29    }
30  }
```

Programski kod 5.6: Primjer BEM konvencije imenovanja klasa - CSS

5.1.4. Pretprocesori

CSS pretprocesori su nadogradnja na obični (eng. vanilla) CSS. Proširuju CSS tako da dodaju funkcionalnosti, nova pravila ugnježđivanja te imaju svoju sintaksu koja se pri kompajliranju pretvara u obični CSS. Iako pretprocesori imaju očite prednosti, Martine i Michael Dowden ističu da pretprocesori dodaju novu razinu kompleksnosti te da njihovo uvođenje treba biti opravdano stvarnim potrebama projekta [8, str. 165]. Za praktični dio rada odabran je pretprocesor SCSS zato što je on jedan od najpopularnijih, ima sličnu sintaksu kao običan CSS, a istovremeno omogućuje korištenje varijabli, ugnježđivanje selektora te razdvajanje CSS koda u više datoteka.

SCSS ugnježđivanje omogućuje jasniju vizualnu reprezentaciju hijerarhije elemenata tako da se dijete ne roditelja dohvaća preko kombinatora "razmak", nego se unutar selektora roditelja dohvaća selektor djeteta. Iz primjera Programskog koda 5.7 može se vidjeti da se u SCSS dohvaćanje naslova druge razine (`h2`) radi unutar selektora roditeljskog elementa (*kartica*). Kako bi se u ovom slučaju pseudoklasa (ili pseudoelement) referirala na neki element koristi se znak `&` te ako se mišem prijeđe preko elementa s klasom `kartica`, taj element će primijeniti pozadinsku boju u `#eee`. Napisani SCSS kod kompajlira se u običan CSS koji se može vidjeti istu u primjeru Programskog kod 5.7. [8, str. 167-168]

```
1  /* Običan CSS */
2  .kartica {
3    padding: 1rem;
4  }
5  .kartica h2 {
6    font-size: 1.5rem;
7  }
8  .kartica:hover {
9    background-color: #eee;
10 }
11
12 /* SCSS */
```

```
.kartica {
  padding: 1rem;
  h2 {
    font-size: 1.5rem;
  }
  &:hover {
    background-color: #eee;
  }
}
```

Programski kod 5.7: Primjer ugnježđivanja selektora unutar SCSS-a

`Mixins` u SCSS-u predstavljaju setove CSS svojstva i vrijednosti koji se mogu više puta iskoristiti. U Programskom kodu 5.8 definiran je `Mixin` koji unutar sebe ima opća svojstva gumba, tako da kada se dodaju stilovi gumbu, svi opći stilovi se ne moraju ponovno pisati nego se `Mixin` uključi pomoću direktive `@include`. Ovaj `Mixin` ima definirano da Prisma parametar `$pozadinska-boja` koja se koristi postavljenje te konkretne boje. Prilikom kompajliranja na mjestu direktive prepisuju se svojstva iz `Mixin-a`. [8, str. 171-176]

```
@mixin stil-gumb($pozadinska-boja) {
  padding: 0.5rem 1rem;
  border: none;
  background-color: $pozadinska-boja;
}
.gumb-primarni {
  @include stil-gumb(blue);
}
.gumb-sekundarni {
  @include stil-gumb(red);
}
```

Programski kod 5.8: Primjer SCSS Mixins

Jedan problem koji se javlja pri korištenju `@include` dekoratora jest dupliciranje koda, odnosno svaku puta kada se `@include` koristi, isti se CSS stilovi ponovno pišu. Rješenje se pronalazi u korištenju `@extend` dekoratora koji se koristi kada jedan selektora (klasa) treba preuzeti pravila drugog selektora (klase). U primjeru Programskog koda 5.9 klase `gumb-primarni` i `gumb-sekundarni` preuzimaju osnovna svojstva klase `button`, te na to dodaju vlastitu boju pozadine. U knjizi *Architecting CSS* ističe se prednost ovog načina u pogledu održivosti zato što su stilovi kontrolirani unutar CSS-a, odnosno ako proširenje više nije

potrebno, potrebno je samo maknuti dekorator `@extend`. Ovaj način pomaže u održavanju pravila odavanje odgovornosti (eng. separation of concerns). [8, str. 176-178]

```
/* SCSS */
.gumb {
  padding: 0.5rem 1rem;
  border: none;
}
.gumb-primarni {
  @extend .button;
  background-color: blue;
}
.gumb-sekundarni {
  @extend .button;
  background-color: red;
}

/* Kompajlirano u CSS */
.gumb, .gumb-primarni, .gumb-sekundarni {
  padding: 0.5rem 1rem;
  border: none;
}
.gumb-primarni {
  background-color: blue;
}
.gumb-sekundarni {
  background-color: red;
}
```

Programski kod 5.9: Primjer SCSS dekoratora @extend

U konačnici potrebno je spomenuti dekorator `@import` koji omogućuje uključivanje dijelova CSS datoteka u druge datoteke. Ovo će biti temelj za izgradnju arhitekture CSS datoteka (Poglavlje *5.3 Organizacija stilskih datoteka*). [8, str. 179-180]

5.2. Responzivnost primjenom suvremenog CSS-a

Razvojem tehnologija, originalno mjesto web stranica se promijenilo. Web stranicama se sve manje pristupa preko stolnih i prijenosnih računala, a sve više preko mobilnih uređaja.

Uz to koriste se tableti i televizori koji sada nude aktivnost pretraživanje sadržaja [12, str. 2-7]. Ova prilagodljivost weba je jedna od njegovih najvećih prednosti. Kako bi tu prednost iskoristio sadržaj se na webu mora prezentirati najrelevantinije u odnosnu na format područja prikaza (eng. viewport) i uređaj preko kojeg mu se pristupa, te je to definicija responzivnosti koju daje Ben Frain, autor knjige *Respnsive Web Design with HTML5 and CSS* [13, str. 3].

5.2.1. Upiti o mediju

Prema mišljenju Peter Gasstona upiti o mediju su najveći pokretač promjena na webu i razlog zašto je CSS postao široko prihvaćen [12, str. 40]. Upiti o mediju su jednostavni logički izrazi koji ako su zadovoljeni primjenjuju stilove unutar izraza. Ovi upiti se pišu na kraju datoteke tako da se prema pravilima kaskade prikažu oni stilovi koji su najkasnije definirani. Tako prema primjeru Programski kod 5.10 ako je ekran širi od `720px`, boja pozadine stranice bit će zelena zato što je to jedini definirani stil za tu širinu. Ako je ekran uži ili jednak `720px` pozadinska boja bit će crvena jer je taj stil definiran kasnije i nadjačava onaj koji definira zelenu pozadinsku boju.

```
1 body{ background-color: green; }
2
3 @media screen and (max-width:720px) { background-color: red; }
```

Programski kod 5.10: Primjer meidjskog upita

Pri pisanju medijskih upita preporučuje se koristiti značajke `min-width` ili `max-width` [12, str. 42]. Za prvu će se primijeniti medijski upit ako je širina ekrana veća ili jednaka od navedene. Ova značajka koristi se kada se radi dizajn najprije za mobilne uređaje (eng. mobile first) [12, str. 57-58] te je bitno da su vrijednosti širine ekrana unutar medijskih upita poredane uzlazno tako da se za veće ekrane prikazuju stilovi namijenjeni za veće ekrane. Suprotno vrijedi za drugu značajku. Ona je najmijenjena kada se dizajn radi prvo za veće ekrane, te se medijski upit primjenjuje kada širina ekrana ne prelazi definiranu vrijednost unutar `max-width`. Upite je potrebno pisati s vrijednostima u silaznom poretku.

Unutar medijskih upita, osim provjere širine ekrana, može se provjeravati orijentacija prostora za prikaz (`orientation: portrait|landscape`), proporcije prostora za prikaz (`device-aspect-ratio: `), rezolucija (`resolution: `). Isto tako više provjera se mogu spojiti logičkim operatorom `and` ili koristeći zarez koji predstavlja logički operator `ili`[12, str. 40-51].

5.2.2. CSS jedinice

Jedna vrlo jednostavna stvar koja jako utječe na responzivnost sučelja jesu mjerne jedinice koje se koriste. U modernom CSS-u nema jedne jedinice za sve, nego je potrebno koristiti i kombinirati više njih. Najčešće spominjana jedinica u računarstvu jest piksel (eng. pixel, px). Ona predstavlja malu točku na ekranu, te se ta jedinica koristi da definiranje vrlo preciznih detalja poput rubova (eng. border), veličina ikona, no nije savršena za tipografiju. Za tipografiju preporučuje se koristi relativnu mjernu jedinica *rem* (eng. relative em) koja se računa prema veličini fonta korijenskog (*:root*) elementa [12, str. 56-57]. Ova se mjerna jedinica koristi za definiranje većih vrijednosti poput već spomenute veličine teksta, razmaka i sl. Prednost ove jedinice je u tome što je veličina relativna u odnosu na korijenski element, što znači ako korisnik u pregledniku ima podešenu veličinu teksta veću za 2 piksela, sve vrijednosti izražene u jedinici *rem* će se prilagoditi i skalirati prema novoj veličini teksta korijenskog elementa. To bi značilo da se tekst veličine `font-size: 1rem` neće prikazivati u veličini od 16 piksela (što je zadana (eng. default) vrijednost), nego kao tekst veličine 18 piksela.

Za definiranje veličine u odnosnu na roditeljski element, koristi se mjerna jedinica "postotak" (`%`) koja predstavlja koliko će prostora u odnosu na roditelja zauzeti dijete [12, str. 53-54]. Ako je potrebno odrediti visinu, širinu ili bilo koje drugo svojstvo u odnosu na veličinu prostora prikaza (eng. viewport) koriste se mjerne jedinice `vh` (eng. viewport height) i `vw` (eng. viewport width). Skaliranjem prostora prikaza, mijenjaju se ove vrijednosti te širina i/ili visina elementa na kojem su one primjenjuju [13, str. 175-176] [12, str. 56]. Preporučuje se korištenje `svh`(small viewport height) i `svw`(small viewport width) zato što one dodatno uzimaju u obzir visinu/širinu alatne trake preglednika, što je posebno bitno na mobilnim uređajima.

Što se tiče korisničkog iskustva (eng. user experience, ex), ne preporučuje se koristiti tekst dulji od 75 znakova u jednom redu. Ako se to ne poštuje, tekst postaje težak i zamoran za čitanje. Umjesto pogađanja "magičnih brojeva" (eng. magic numbers) da tekst ne prelazi 75 znakova, CSS omogućuje mjernu jedinicu jednog znaka (eng. character) te je samo potrebno postaviti širinu teksta na 75 znakova (`width: 75ch`). [8, str. 50]

5.2.3. Fluidne vrijednosti i CSS funkcije

Iako CSS ne dopušta korisniku da sam definira funkcije, CSS omogućuje korištenje preddefiniranih funkcija koje obavljaju različite zadatke. Tako postoje funkcije `rgb()`, `hsl()` za definiranje boja, `scale()`, `rotateX()` za transformaciju elementa i još mnogo njih [8,

str. 50-52]. Tako jedna od korisnih funkcija za pisanje responzivnog koda jest matematička funkcija `calc()` koja kao argument Prisma matematički izraz [13, str. 176-177]. Pomoću te funkcije moguće je zbrajati, oduzimati, množiti i dijeliti mjerne jedinice. Na primjer, moguće je i kombinirati mjerne jedinice. Moguće je izračunati visinu početne sekcije web stranice tako da se od pune visine prikaznog prostora (*100svh*) oduzme visina navigacije (*200px*) (Programski kod 5.11).

```
/* CSS */
.pocetna-sekcija{
  height: calc(100vh - 200px);
  ...
}
```

Programski kod 5.11: Primjer korištenja CSS funckije calc()

Uz funkciju `calc()`, CSS podržava i matematičke funkcije `min()` i `max()`. Funkcija `min()` odabire manju od ponuđenih vrijednosti, dok `max()` odabire veći od ponuđenih. Bitno je da jedan od argumenata bude fluidna vrijednost, odnosno ona koja se mijenja, a druga vrijednost da bude fiksna, na primjer zadana u pikselima, *rem* mjernoj jedinici i slično. Primjer primjene funkcije `min()` bio bi da je cilj napraviti da je maksimalna širina spremnika u kojem se prikazuje sadržaj 1300 piksela, a ako je širina ekrana manja od ove vrijednosti onda poPrisma najveću dostupnu širinu. To je moguće napraviti pomoću medijskih upita (Programski kod 5.12) ili pomoću funkcije `min()` (Programski kod 5.13). Iz ovih programskih primjera može se vidjeti koliko je kod pregledniji i jednostavniji kada se koristi funkcija `min()`.

```
/* CSS */
.spremnik{
  width: 100%
}
@media screen and (min-width: 1300px){
  width: 1300px;
}
```

Programski kod 5.12: Primjer ograničavanja širine spremnika - Media upit

```
/* CSS */
.spremnik{
  width: min(100%, 1300px);
}
```

Programski kod 5.13: Primjer ograničavanja širine spremnika - funkcija min()

Još jedna korisna funkcija za definiranje responzivne veličine jest funkcija `clamp()`. Funkcija Prisma 3 parametra: minimalna veličina, preferirana veličina i maksimalna veličina. Prednost ove funkcije je da omogućuje definiranje responzivnosti unutar određenih granica. Granice se prosljeđuju kao prvi i treći argument, te one trebaju biti izražene u apsolutnim, odnosno fiksnim mjernim jedinicama (*px, rem, em*), dok se preferirana vrijednost izražava kroz relativne, odnosno fluidne mjerne jedinica (*%, vh, vw*). Tako se preferirana veličina skalira unutar zadanih granica. Najčešći primjer ove funkcije je pri definiranju veličine teksta. To bi konkretno na Programskom kodu 5.14 značilo da je minimalan veličina teksta *1rem*, maksimalna *2rem* te da se stvarna veličina teksta mijenja u ovisnosti od *2vw*.

```
1 /* CSS */
2 .naslov-kartice{
3   font-size: clamp(1rem, 2vw, 2rem);
4 }
```

Programski kod 5.14: Primjer funkcije clamp()

5.2.4. Fluidni raspored elemenata

Viša razina responzivnog dizajna jest fluidni dizajn. Peter Gasston fluidni dizajn opisuje kao ključ responzivnog dizajna [12, str. 53]. Fluidni dizajn je dio responzivnog dizajna zato što omogućuje promjenu rasporeda u odnosu na širinu ekrana. Do sada su se spominjali medijski upiti (eng. media queries) koji su ima fiksne točke prijeloma. Karakteristika fluidnosti jest da nema fiksnih točaka nego se elementi kontinuirano prilagođavaju dostupnom prostoru. Time se olakšava posao izrade dizajna stranice zato što programer više ne mora određivati dizajn za točke prijeloma i kako će elementi biti posloženi, nego sada to umjesto njega radi preglednik koji automatski pozicionira elemente.

Jedan od najvažnijih CSS alata za ostvarenje fluidnog dizajna jest *Flexbox* [13, str. 78]. On omogućuje da se veličina i raspored elemenata mijenja kako bi se najbolje pristupilo dostupnom prostoru [12, str.70]. Da bi se koristio, potrebno je roditeljski element pretvoriti u fleksibilni kontejner preko CSS svojstva `display: flex`. Sve stavke roditeljskog elementa postaju fleksibilne stavke. Svojstvom `flex-warp: warp|no-warp` može se definirati prelamanje fleksibilnih stavki u novi red ako je širina ekrana premala. Preko svojstva `flex-basis` određuje se preferirana širina, a sa svojstvima `flex-shrink` i `flex-grow` određuje se hoće li fleksibilni element rasti i se smanjivati te u kolikoj proporciji. Preko roditeljskog elementa može se utjecati na redoslijed stavki (`flex-direction`) i njihov raspored u odnosu na glavnu

(*x*) ili poprečnu (*y*) os. Elementi se mogu postaviti na početak, sredini, ravnomjerno se rasporediti po širini preko svojstva `justify-content` i na početak, sredinu ili kraj po visini preko `align-items` [12, str.73-77] [13, str. 94-102]. *Flexbox* je idealan za manipulaciju stavkama u jednom smjeru, odnosno jednodimenzionalno (red ili stupac), te Peter Gasston ističe da je *Flexbox* posebno namijenjen za izradu korisničkih sučelja [12, str.70].

Drugi važan oblik rasporeda elementa jest u obliku rešetke. Za razliku od *Flexbox-a* koji omogućuje upravljanje elementima u jednom smjeru, CSS rešetke omogućavaju kreiranje dvodimenzionalnog rasporeda elemenata [13, str. 122]. Rešetka radi na principu da se roditeljskom elementu za svojstvo `display` postavi vrijednost `grid`. Automatski se svi elementi unutar roditeljskog raspoređuju u jedan stupac. Više stupaca moguće je definirati preko svojstva `grid-template-columns`, koja na primjer Prisma vrijednosti `10px 10px 10px` čime se stvaraju 3 stupca širine 10 piksela. Isti je princip za definiranje redova samo što se za njih koristi svojstvo `grid-template-rows`.

Elementi se unutar rešetke prikazuju redoslijedom kojim se definirani unutar HTML-a. Grid uvodi novu jedinicu `fr` koja predstavlja slobodno mjesto. Tako ako je definiran jedan stupac s vrijednosti `1ft` on će zauzeti cijelu širinu. Ako je roditelj širine `900px` i preko rešetke se definiraju dva stupaca: jedan širine 10 piksela, a drugi širine `1fr`, prvi će stupac biti širine od 10 piksela, a drugi 890 piksela jer je toliko preostalo. Isto tako rešetka može koristiti funkciju `repeat()` koja služi za definiranje većeg broja stupaca/redova. Kao prvi argument Prisma koliko stupaca/redova se želi definirati, a kao drugi argument Prisma njihovu širinu/visinu. Tako umjesto definiranja 3 stupca od 10 piksela kako je napravljeno u prethodnom odlomku, to se može napraviti preko ove funkcije ovako `grid-template-columns: repeat(3, 10px)`. [13, str. 135]

Iz navedenog opisa i primjera može se primijetiti da CSS rešetka Prisma konkretno definirane vrijednosti, odnosno broj stupaca ili redova, čiji se broj može mijenjati preko medijskih upita, no to onda više ne pripada fluidnom dizajnu. Unatoč tome, CSS rešetke omogućavaju fluidan dizajn korištenjem svojstva `auto-fit` i `auto-fill` koja kao prvi parametar funkcije `repeat()`. Ove vrijednosti govore pregledniku da kreira što je moguće više stupaca ili redaka širine ili visine definirane drugim argument. No moguće je i definirati fluidnu širinu/visinu elementa pomoću CSS funkcije `minmax()` kojoj se prosljeđuje minimalna i maksimalna vrijednost, te zatim preglednik sam izračuna širinu ili visinu u odnosnu na slobodno mjesto. [13, str. 145-148]

Programski kod 5.15 primjer je fluidne CSS rešetke, te ovaj kod govori pregledniku da

stvori što je više moguće stupaca u jednom redu, tako da kao je širina ekrana veća od 400 piksela, stupci ne prelaze širinu od 400 piksela. Ako je širina ekrana manja od 400 piksela, funkcija `min` uzima vrijednost `100%` te elementi poprimaju punu širinu ekrana. Funkcija `minmax` osigurava da svaki stupac ima minimalnu i maksimalnu vrijednost. Ako ima viška prostora u jednom redu, stupci ga ravnomjerno podjele zahvaljujući svojstvu `auto-fit`.

```
.fluidna-resetka{
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(400px, 100%), 1fr));
}
```

Programski kod 5.15: Primjer potpuno fluidne CSS rešetke

5.2.5. Tokeni i varijable

Moderni CSS omogućuje korištenje koncepta varijabli. Taj koncept služi kako bi se spremila neka informacija unutar stilova koja se može ponovno koristiti, ali i unutar JavaScripta. Takva spremišta se unutar CSS naziva prilagođena svojstva (eng. custom properties), ali se i nazivaju CSS varijable. Najčešće se definiraju unutar `:root` pseudoklase tako da se mogu dohvatiti unutar cijelog dokumenta. No mogu se definirati i unutar pojedine klase pa su te varijable dostupne samo unutar te klase i djeci koja imaju roditelja s tom klasom. Prilagođena svojstva definiraju se prefiksom dvostrukog znaka minus (`-`), zatim proizvoljno ime koji se dohvaća vrijednosti te zatim dvotočka iza koje se definira vrijednost. Vrijednost se dohvaća tako da se ime varijable proslijedi kao argument funkcije `var`. Pošto te vrijednosti može JavaScript mijenjati, moguće je da se postavi neka neispravna vrijednosti ili se ona u potpunosti izbriše, zato je moguće postaviti zamjenske vrijednosti (eng. fallback) koje se prikazuju ako varijablu nije moguće. Zamjenske vrijednosti prosljeđuju se kao drugi argument funkcije `var()` (Programski kod 5.16). [13, str. 177-180]

```
/* CSS */
:root{
  --primarna-boja: blue;
}
body{
  background-color: var(--primarna-boja, red);
}
```

Programski kod 5.16: Primjer prilagođenog svojstva

Primjena prilagođenih svojstva omogućava da se na jednom mjestu definiraju sve fiksne vrijednosti poput boja, razmaka, sijena, veličina teksta i sl. tako da se kasnije mogu ponovno koristiti, te u slučaju da se neka vrijednost treba promijeniti to se radi samo na jednom mjestu i promjena se primjenjuje svugdje. Time se smanjuje ponavljanje koda, olakšava održivost i omogućuje brža promjena.

Važno je razlikovati SCSS varijable i CSS prilagođene vrijednosti. SCSS varijable definiraju se znakom `$` te se nakon kompajliranja koda pretvaraju u obične CSS vrijednosti pa ih zato preglednik ne može mijenjati. S druge strane, CSS prilagođena svojstva se definiraju pomoći dva znaka minus i ostaju prisutna unutar CSS-a te njima zato može manipulirati pomoći JavaScripta.

Do sada su se spominjale mnoge nove funkcionalnosti i obrasci koje mogu biti teže za razumjeti i možda su nepoznate razvojnom timu te to može predstavljati problem za održivost. Pomoću CSS prilagođenih svojstva moguće je izbjeći taj problem tako da se ključne vrijednosti izdvoje u lokalna prilagođena svojstva, te se u slučaju neke promjene programer mora samo promijeniti vrijednost svojstva bez da zna kako određene CSS funkcija radi. Uzmemo li Programski kod 5.17 kao primjer koda u kojem se koriste CSS funkcije, gledajući samo ovaj CSS kod teško je razumjeti što koja vrijednost znači i čemu služi.

```
1 /* CSS */
2 .spremnik{
3   width: min(calc(100% - 2rem), 1300px);
4   margin: 0 auto;
5   font-size: clamp(1rem, 2vw, 2rem);
6 }
```

Programski kod 5.17: Primjer korištenja CSS funkcija bez CSS varijabli

Programski kod 5.18 predstavlja refaktorirani prethodni primjer. On se može činiti kompleksnijim, ali sve što se napravilo jest da su se ključne vrijednosti izdvojile u lokalne CSS varijable te su im dodijeljena imena. Ovaj pristup je sličan načelu skrivanja implementacije, odnosno učahurivanjem (eng. encapsulation) u objektno orijentiranom programiranju. CSS varijable imaju ulogu sučelja (eng. interface) preko kojeg programer mijenja ponašanje stilova, dok samu implementaciju CSS funkcija ne mora detaljno znati ako ju ne treba izravno mijenjati. Ako je potrebno promijeniti maksimalnu širinu spremnika na 1500 piksela, potrebno je samo primijeniti vrijednost CSS varijable `-max-sirina-spremnika` bez potrebe da zna što se događa u pozadini.

```css
/* CSS */
.spremnik{
  /* "Sučelje" */
  --razmak-od-ruba-ekrana: 1rem;
  --max-sirina-spremnika: 1300px;
  --min-velicina-fonta: 1rem;
  --preferirana-velicina-fonta: 2vw;
  --max-velicina-fonta: 2rem;

  /* Implementacija */
  --ukupan-razmak-od-ruba-ekrana: 2 * var(--razmak-od-ruba-ekrana);
  width: min(calc(100% - var(--ukupan-razmak-od-ruba-ekrana)), var(
    --max-sirina-spremnika));
  margin: 0 auto;
  font-size: clamp(var(--min-velicina-fonta), var(
    --preferirana-velicina-fonta), var(--max-velicina-fonta));
}
```

Programski kod 5.18: Primjer korištenja CSS funkcija s CSS varijabli

5.3. Organizacija stilskih datoteka

U praktičnom dijelu rada stilske datoteke će se organizirati prema metodologije obrnutog trokuta CSS-a (eng. Inverted Triangle CSS) poznatija pod skraćenicom ITCSS. Ovu metodologiju osmislio je Harry Roberts s ciljem rješavanja problema specifikacije selektora tako da se prioritet odabira pravila oslanja na prirodni prioritet CSS-a [8, str. 219]. Taj cilj se ostvaruje razdvajanjem listova stilova (eng. style sheets) u slojeve na temelju ciljeva određenog sloja. Time se postiže jasna struktura, predvidljivost i kontrola nad stilovima, odnosno kaskadom.

Ova metodologija je odabrana za organizaciju stilova jer Angular daje jasnu strukturu za opću arhitekturu aplikacije kroz module, komponente i servise dok se stilove pišu na razini komponenta te to dovodi do dupliciranje stilova i nekonzistentnosti u njihovom pisanju. Stilovi se pišu "usput", uvijek se stavljaju na kraj datoteke kako bi nadjačali prethodne te se time gubi organizacije, dosljednost, teško je predvidjeti posljedice promjene što drastično otežava održavanje. ITCSS ima ulogu standardizirati način na koji se stilovi pišu, gdje ih se treba smjestiti i koja ima je uloga. Po samo prirodi metodologije potrebno je poštivati pravila koja sprječavaju tzv. "ratove specifičnosti".

Naziv obrnuti ili invertirani trokut (eng. Inverted Triangle) dolazi od vizualnog prikaza arhitekture (Slika 5). Na vrhu trokuta nalaze se stilovi s najvećim dosegom i najmanjom specifičnošću. Doseg je prema slici prikazan koliko je širok prostor unutar trokuta, dok se specifičnost prikazana položajem po visini. Što je sloj niže smješteni to su njegovi stilovi više specifični i eksplicitni, odnosno imaju uzak slučaj upotrebe (eng. use case). Dosegom se označava nad kolikim dijelom aplikacije pojedini stilovi imaju utjecaj. Cilj je da doseg i eksplicitnost budu obrnuto proporcionalni. Veći doseg predstavlja veću razinu apstrakcije i širi spektar primjene, dok su eksplicitniji stilovi vezani uz konkretan dio aplikacije i kontekst primjene.

5.3.1. Slojevi ITCSS-a

Najviši sloj u obrnutoj piramidi je sloj koji se zove "postavke" (eng. settings). U ovom sloju se ne definiraju nikakva CSS pravila i stilovi, nego osnovne vrijednosti koje se koriste u projektu. Te se vrijednosti spremaju u SCSS varijable, a te vrijednosti mogu biti boje, tipografija, razmaci, točke prijeloma, trajanje animacija, sjene i slično. Ove vrijednosti koriste se kroz cijeli projekt te zato imaju najveći doseg. Ovaj sloj ne bi trebao generirati nikakav CSS rezultat te je namijenjen za pretprocesore. [8, str. 219]

Zatim dolazi sloj pod nazivom "alati" (eng. tools). U ovom se sloju isto tako ne pišu tipični stilovi, nego pomoćne funkcije koji služe za jednostavnije pisanje stilova, ali unutar sebe ne definiraju izgled. To su najčešće *mixins*, funkcije i pomoćnici. Ovaj sloj je primarno namijenjen za pretprocesore i ne bi smio proizvoditi CSS rezultat. [8, str. 220]

Nakon njega dolazi sloj u kojem se definiraju opći/generički stilovi. To su stilovi koji ima vrlo širok doseg te nisu direktno vezani uz dizajn projekta. U to spadaju tzv. ponovno postavljanje CSS-a i normaliziranje CSS-a, CSS varijable, osnovne postavke modela kutije. Ponovno postavljanje CSS-a je skup stilova čija je zadaća ukloniti ili izjednačiti zadane stilove HTML elemenata koje imaju preglednici. Resetiranje CSS-a pomaže da početno stanje bude konzistentno. Drugi pristup je normaliziranje CSS-a. Cilj normaliziranja nije obrisati sve stilove, nego uskladiti razlike između preglednika. U glavnom se na oba koriste kao polazni skup pravila za dosljedan CSS. [8, str. 130-131, 220]

U praktičnom dijelu rada, SCSS varijable koristit će se za spremanje vrijednosti koje su potrebne tijekom izgradnje/kompajliranja CSS-a kao što su točke prijeloma, postavke CSS rešetki, putanje i ostale elementi koji tijekom rada aplikacije ostaju nepromijenjeni. U CSS varijable spremat će se vrijednosti koje moraju biti dostupne preglednicima i koje su podložne promjenama. Tako se, na primjer, varijabla boja pozadine mijenja u odnosu koristi li korisnik

svijetlu ili tamnu temu. Ista tako spremat će se vrijednosti poput razmaka, zaobljenja rubova, sjena i slično.

Četvrti sloj po redu koji se koristi unutar ITCSS jest sloj "elementi" (eng. elements). Unutar njega definiraju se stilovi za standardne HTML elemente bez korištenja klasa [8, str.220]. Koriste se selektori elementa odnosno tipa poput: *h1*, *p*, *a*, *button*. Preko ovih selektora cilj je postaviti osnovnu tipografiju, boje, osnovno ponašanje i izgled gumbova, linkova i sl. Pravila trebaju biti jednostavna i apstraktna jer utječu na elemente unutar cijele aplikacije.

Zatim dolazi sloj "objekta" (eng. objects) u kojem se pišu selektori klasa kako bi se definirali ponovno iskoristiv okvir objekta [8, str.220]. Definiraju se strukturni obrasci kojima se definira struktura i raspored elemenata, a ne detaljan izgled tako da se ti obrasci mogu više puta iskoristiti. Tipični su za definiranje rasporedu unutar spremnika, definiraju rešetku, media upite, centriranja i slično.

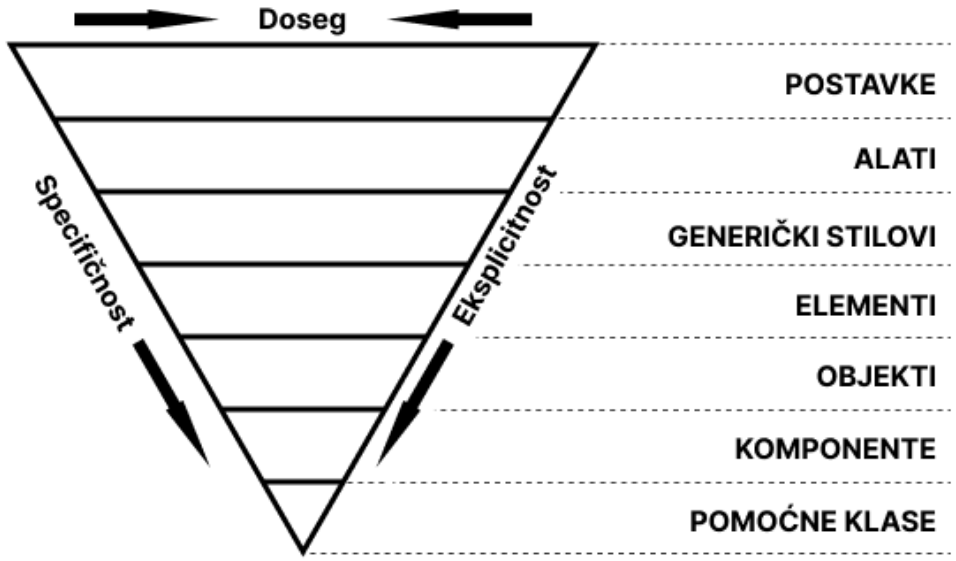
Šesti sloj naziva se sloj "komponenti" (eng. component). U ovom se sloju pišu konkretni stilovi za korisničko sučelje te se u sloju objekta i komponenti nalazi najviše stilova [8, str.220]. Pišu se konkretni stilovi za kartice, navigaciju, modalni dijalog, gumbe, forme i slično.

Posljednji sloj koji je naveden na slici jest sloj "pomoćne klase" (eng. utilities/helpers). U taj se sloj smještaju pravila koja su jako specifična i imaju usko područje primjene te moraju moći nadjačati ostala pravila kada je to potrebno [8, str.220]. Unutar tog sloja nalaze se klase koje prema BEM konvenciji imenovanja imaju modifikator poput `...-hidden` ili `...-active`. One osiguravaju da se element pokaže u željenom stanju kada je stanje zadano.

ITCSS nije okvir koji strogo definira pravila i slojeve. Slojevi mogu biti fleksibilni, spajati se, izolirati i proširivati ovisno o veličini i potrebama projekta. Na primjer, ako se ne koristi SCSS ili neki drugi pretprocesor nema potrebne imati sloj "Alati" koji sadrži proširenja pretprocesora (funkcije, mixins i sl.), zato što te funkcionalnosti ne postoje unutar običnog CSS-a. Harry Roberts spominje da nije uvijek moguće pisati savršene stilove koji imaju jasno definiran sloj i zadaću. Zbog vremenskih ograničenja, ponekad je potrebno uvesti privremena CSS pravila koja nisu idealno strukturirana. Zato se ponekad uvodi sloj koji se naziva "sloj za privremene stilove" ili "sloj tehničkog duga" ili kako ga Harry Roberts naziva "sramotni/posramljeni sloj" (eng. shame layer). On se uvodi kao najniži sloj i služi kako bi se mjesto tehničkog duga izoliralo i učilo vidljivijim. [14]

Na kraju važno je spomenuti da se ITCSS zasniva na principu uključivanja datoteka putem `@import` ili modernije `@use/@forward` u SCSS-u. Princip se temelji na tome da slojevi niže razine uključuju slojeve više razine tako da se osigura da se temeljna pravila definiraju

ranije, a da kasniji slojevi mogu ciljano nadjačati prethodna pravila.



Slika 5: Prikaz arhitekture obrnutog trokuta CSS-a (Izvor: slika je rekreirana prema slici sa `https://www.xfive.co/blog/itcss-scalable-maintainable-css-architecture`, pristupljeno: 26.6.2026.)

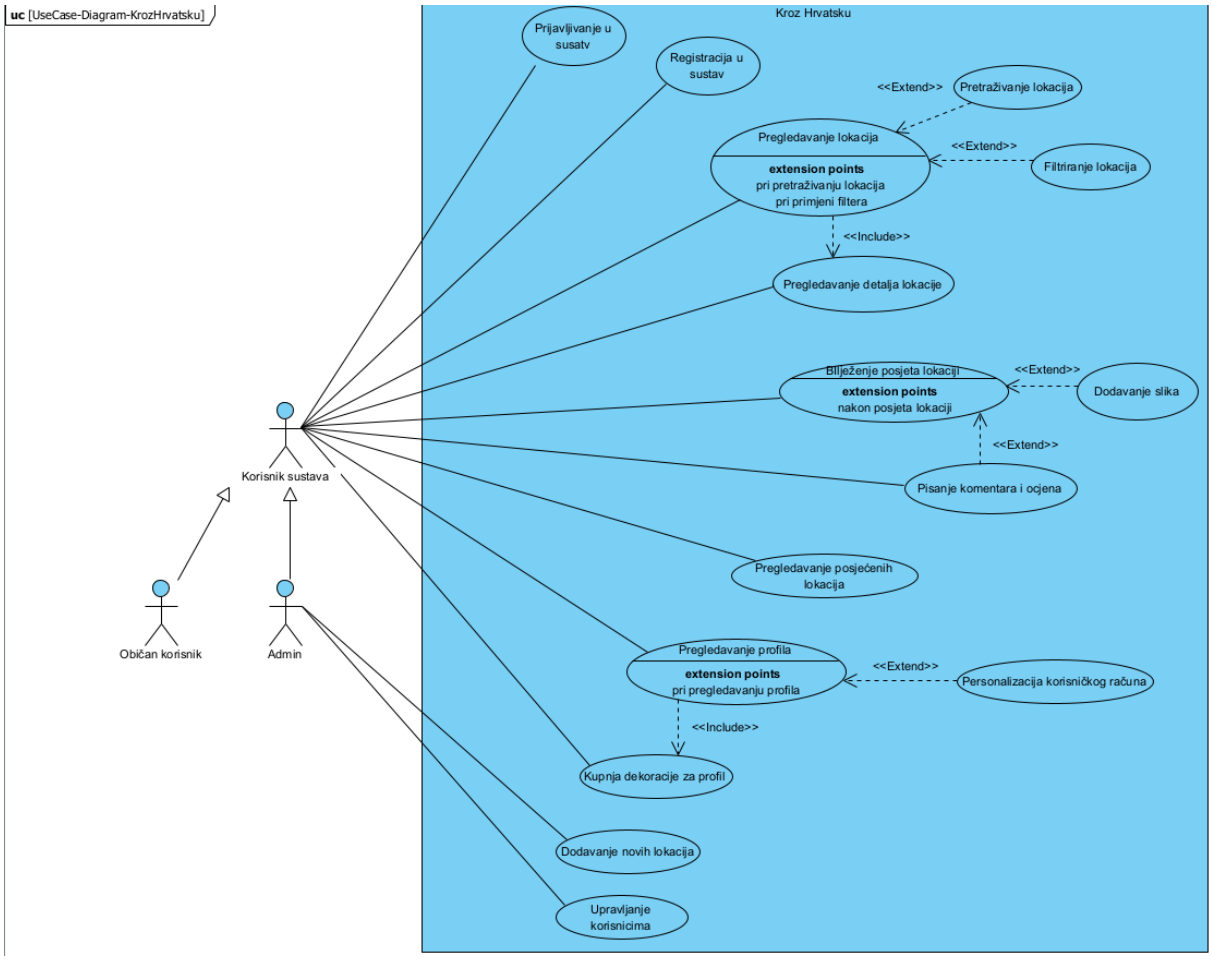
6. Praktični dio rada: Web aplikacija Kroz Hrvatsku

6.1. Opis teme aplikacije

Kao dio praktičnog dijela završnog rada izrađena je aplikacija Kroz Hrvatsku. Aplikacija je web rješenje koje korisnicima omogućuje istraživanje prirodnih, kulturnih i turističkih lokaliteta Hrvatske na interaktivan i pristupačan način. Glavna karakteristika aplikacija jest istraživanje Hrvatske u obliku igre tako nakon što korisnik napravi profil može rješavati lokacije poput izazova. Prilikom rješavanja izazova korisnik može ostaviti vlastiti komentar i slike s lokacije koju je posjetio tako da njegove uspomene traju duže. Uz to, rješavanjem izazova korisnik dobiva iskustvene bodove (eng. Experience Points, XP) čime se prati njegov napredak u obliku razine koju je dostigao tako što svakih 100 osvojenih iskustvenih bodova korisnik otključava novu razinu. Uz iskustvene bodove korisnik dobiva virtualni novac kojim može kupovati dekoracije i bedževe kojima može dekorirati svoj profil unutar aplikacije. U konačnici korisnik svoje iskustvo može podijeliti s drugima tako da lokaciji da ocjenu i staviti svoje mišljenje o lokaciji u obliku komentara.

Osnovna ideja aplikacije temelji se na povezivanju informativnog i zabavnog aspekta putovanja i istraživanja Hrvatske. Korisniku se ne nude samo podaci o određenoj lokaciji, već mu se nudi dodatni poticaj da posjeti lokaciju, zabilježi dolazak, ostaviti svoje mišljenje i osvoji nagrade.

Ova tema aplikacije je odabrana ne samo radi funkcionalnosti nego i zbog zahtjeva implementacije. Korisnika treba potaknuti da putuje i istražuje Hrvatsku te je zato potrebno privlačno korisničko sučelje koje će motivirati korisnika na sljedeći korak. Zbog aspekta putovanja, potrebno je da aplikacije bude responzivna pošto će joj korisnik, kada je na lokaciji, pristupati preko mobilnih uređaja. Aplikacija definira jasno definirane strukture koje se često ponavljaju, te tu dolazi do naglaska održivosti i ponovna iskoristivost komponenti i programskog koda općenito. Zbog velikog broja komponenti i funkcionalnosti potrebno je kvalitetno organizirati datoteke i postaviti arhitekturu koja će omogućavati i olakšati održavanje, rast i razvoj aplikacije.



Slika 6: Prikaz dijagrama slučaja korištenja (eng. use case) za aplikaciju Kroz Hrvatsku

6.2. Arhitektura poslužiteljskog dijela aplikacije

6.2.1. Opća strukutra aplikacije

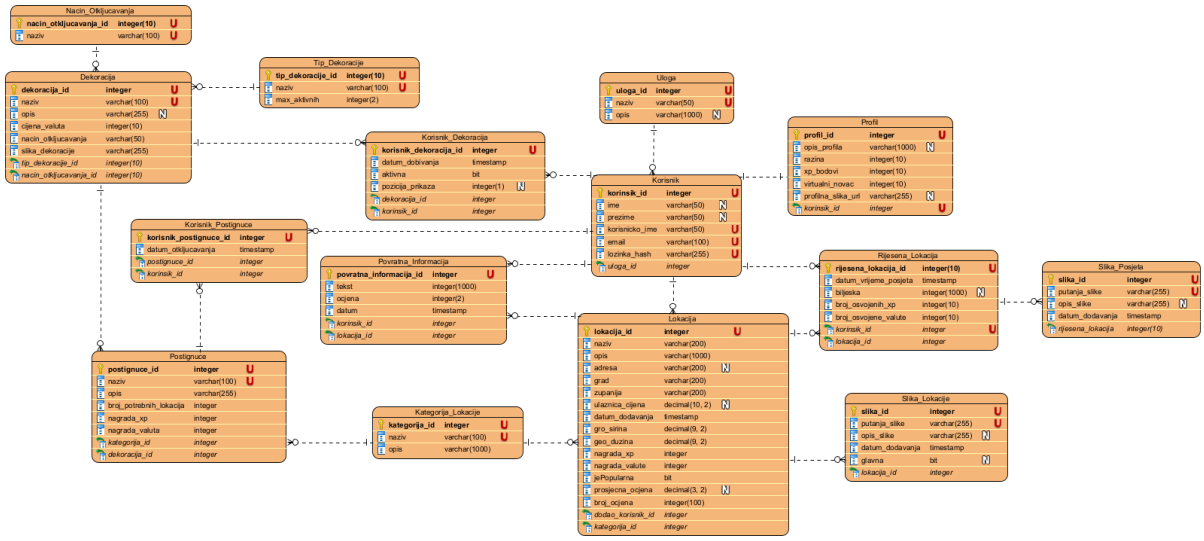
Aplikacija se sastoji od poslužiteljskog i korisničkog dijela te se ta podjela može vidjeti i u strukturi aplikacije tako što se nalaze dvije mape. Prva mapa predstavlja poslužiteljski dio aplikacije te se zove `backend`, koja će se kasije detaljnije analizirati, i druga mapa koja predstavlja klijentski ili korisničku stranicu te se zove `frontend`. Klijentski dio napravljen je u okviru Angular, kako je navedeno u teorijskom dijelu rada, te ima tipičnu strukturu svakog Angular projekta.

6.2.2. Struktura poslužiteljskog dijela aplikacije

Iako poslužiteljska strana nije tema ovog završnog rada, ona je ključna jer se kroz nju definiraju osnovni podaci koji određuju način prikaz i izrade korisničkog dijela aplikaci-

je. Slika 7 prikazuje dijagram entiteta (eng. Entity Relationship Attribute Diagram, ERA dijagram/model) koji definira način i organizaciju podataka unutar aplikacije. Glavni entiteti modela su: `Korisnik/Profil`, `Lokacija`, `Dekoracija` i `Postignuce`. Makar su entiteti `Korisnik` i `Profil` mogli biti jedna tablica, što se vidi iz veze jedan naprema jedan (1:1) oni su odvojeni entiteti kako bi se odvojili korisnikovi podaci iz stvarnog života i podaci vezani uz aplikaciju. Neki korisnik može imati ulogu admina ili običnog korisnika te se iz dijagrama mogu primijeniti 3 veze između entiteta `Korisnik` i `Lokacija`. Jedna veza je jedna naprema više (1:n) koja predstavlja mogućnost administratora da dodaje više lokacije, a da je jednu lokaciju dodao isključivo jedan korisnik. Druge dvije veze su više naprema više (n:n) od kojih jedna predstavlja ostavljanje komentara pri čemu jedan korisnik može ostaviti više komentara na lokaciji te na jednoj lokaciji komentar može ostaviti više korisnika. Druga veza više naprema više predstavlja rješavanje lokacija pri čemu jedan korisnik može rješavati više lokacija i jedna lokacija može biti riješena od strane više korisnika. Za lokacije i slike posjeta mogu biti dodane više od jedne slike te su iz tog razloga slike izdvojene u zasebnu tablicu.

Unutar aplikacije postaje dekoracije više vrsta. Korisnik može imati više dekoracija, a jednu dekoraciju može imati više korisnika te je zato dodana tablica `Korisnik_Dekoracija` koja unutar se sadrži atribut `aktivna` koja određuje hoće li dekoracija biti prikazana na profilu. Neke dekoracije korisnik može kupiti odmah, ako ima dovoljno virtualnih novaca, no za neke dekoracije korisnik mora riješiti određeni broj izazova. Tako postoji tablica `Postignuce` unutar kojeg se definira taj kriterij. Kriterij može biti da korisnik mora riješiti barem 3 lokacije s kategorijom `dvorac`. To se postignuće povezuje s dekoracijom te služi kao kriterij otključavanja, te se određeno postignuće može dodijeliti korisniku. Korisnik može imati otključano više postignuća, te jedno postignuće može otključati više korisnika (n:n) te je zato potrebna tablica `Korisnik_Postignuce`. Ako korisnik ima riješeno postignuće, dekoracija mu se otključava za kupnju.



Slika 7: Prikaz dijagrama entiteta za aplikaciju Kroz Hrvatsku

6.2.3. Komunikacija bazom podataka

Baza podataka napravljena je na temelje ovog dijagrama entitet te je korištena PostgreSQL baza. Pozadinski dio napravljen je u NestJSu, a za komunikaciju s bazom korišten je Prisma ORM. ORM predstavlja objektno-relacijsko mapiranje (eng. Object-Relational Mapping), odnosno preslikavanje podataka koji su bazi spremljeni u relacijskom obliku u oblik objekta koji se koristi programskim jezicima. Prisma ORM je alat razvijen za Node.js i TypeScript aplikacije tako da se idealno uklapa u ovu aplikaciju te omogućuje pisanje upita u obliku rad s objektima, a ne u obliku tipičnih SQL upita. U programskom kodu 6.1 može se vidjeti primjer upita za dohvaćanje podataka iz baze pomoću Prisma ORM. U servis je prvo potrebno ubaciti (injektirait) Prisma servis preko kojeg se pristupa bazi. Zatim se u funkciji `dohvatiSveLokacije()` vraćaju vrijednosti iz tablice `lokacija` kojoj se pristupa preko instance Prisma servisa, te se preko metode `findMany` specificira dohvat više podataka. U ovom slučaju vraćaju se sve lokacije, zato što nije definiran uvjet, te se uz atribute u tablice `lokacija`, vraćaju svi atributi iz tablica `kategorija` i `slikeLokacije` koji su preko vanjskog ključa povezani s određenom lokacijom.

```
1 @Injectable()
2 export class LokacijeService {
3   constructor(private readonly prisma: PrismaService) {}
4   dohvatiSveLokacije() {
5     return this.prisma.lokacija.findMany({
6       include: {
7         kategorija: true,
```

```
        slikeLokacije: true,
      },
    });
  }
}
```

Programski kod 6.1: Primjena prisma upita na bazu

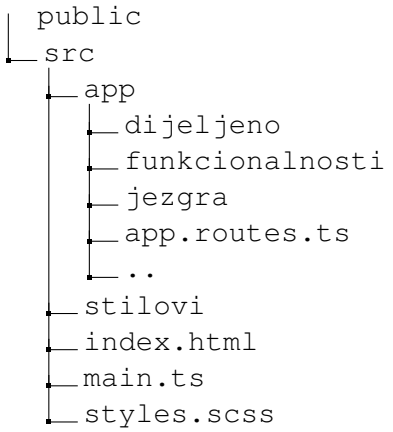
Zatim se u kontroleru za lokacije, na poziv rute `'lokacije'` sa `Get()` metodom poziva metoda `dohvatiSveLokacije()` iz servisa koji je prethodno injektiran.

```
@Injectable()
@Controller('lokacije')
export class LokacijeController {
  constructor(private readonly lokacijeService: LokacijeService) {}
  @Get()
  dohvatiSveLokacije() {
    return this.lokacijeService.dohvatiSveLokacije();
  }
}
```

Programski kod 6.2: Poziv funkcije dohvatiSveLokacije() na poziv rute

6.3. Arhitektura korisničkog dijela aplikacije

Na sam vrhu stabla nalaze se mape `public` i `src`. U mapu `public` spremljene su slike koje se neće mijenjati te su trajne, poput nekih ikona, slika na početnoj stranici koje služe kao dekoracija te ikona web stranice poznatija kao `favicon`. U drugoj mapi nalazi se kod aplikacije. `Index.html`, `main.ts` i `styles.scss` su osnovne datoteke u Angularu iz kojih se učitava sav ostali kad. U mapi `app` spremnjene su komponente i sav kod stranica. U konačnici dodana je mapa `stilovi` unutar koje je realizirana arhitekturu stilova invertiranim CSS trokutom (engl. ITCSS).



Slika 8: Organizacija direktorija Angular aplikacije

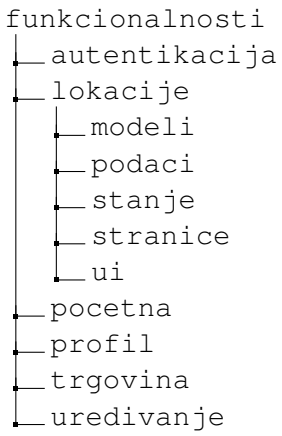
U mapi `app` napravljena je podjela datoteka u odnosu svrhu. U mapi `dijeljeno` smještene su komponente koje se koriste u svim dijelovima aplikacije poput trake napretka iskustvenih bodova. Isto tako, većina ikona spremljene su kao komponente u obliku skalabilne vektorske grafike (SVG) tako da postoji više fleksibilnosti prilikom korištenja. Tako na primjer komponenti koja prikazuje ikonu strelice mogu poslati ulazni atributi poput veličine, boje ili rotacije tako da postoji potpunu kontrolu nad izgledom. Zatim u mapi `jezgra` smješteni su osnovni dijelovi aplikacije. Tako se u toj mapi nalaze: zaštitnici (eng. guards) ruta koji provjeravaju ima li osoba dopuštenje za pristup ruti, presretač koji prilikom svakog zahtjeva na poslužiteljski dio dodaje, kao već nije dodan, JWT token preko kojeg se autenticira korisnik koji šalje zahtjev, konfiguracijske datoteke poput putanje na rutu poslužitelja i API ključ za Google mape. Na kraju se još nalaze osnovne komponente navigacije i podnožja koji se učitavaju na svim stranicama. Također u datoteci gdje se učitavaju navigacija i podnožje, učitava se i `<router-outlet />` komponenta koja prikazuje komponentu ovisno o ruti na kojoj se korisnik nalazi.

6.3.1. Funkcionalnosti i njihova podjela

Mapa `funkcionalnosti` sadrži module i komponente koji prikazuju glavne funkcionalnosti aplikacije. Svaka funkcionalnost, ili više njih grupiranih u jednu krovnu funkcionalnost, organizirane su u zasebnu podmapu čime se potiče načelo razdvajanje odgovornosti.

Glavne funkcionalnosti aplikacije su: pregled i filtriranje lokacija, prikaz detalja lokacije, interaktivni prikaz županija na početnoj stranici, registracija i prijava korisnika, prikaz korisničkog profila, bilježenje dolaska na lokaciju, dodavanje slika i bilješke posjeta, komentiranje i ocjenjivanje lokacija, prikaz napretka korisnika kroz razine, XP bodove i virtualni novac, kupnja i prikaz dekoracija profila te administracijsko upravljanje lokacijama. Za ovih 11 osnovnih funk-

cionalnosti odabrane su 6 cjelina aplikacije, odnosno 11 navedenih funkcionalnosti spadaju u jednu od 6 krovnih. 6 cjelina su: autentikacija, lokacije, početna stranica, profil, trgovina i uređivanje. Svaka od njih ima vlastite stranice, prezentacijske komponente, modele, servise (mapa podaci sa slike 9) i stanje.



Slika 9: Organizacija mape funkcionalnosti

6.3.2. Dohvat i prikaz podataka

U aplikaciji je korišteno razdvajanje kontejnerskih i prezentacijskih komponenti. Kontejnerske komponente povezane su sa rutama i služe za izvršavanje logike unutar web stranice tako da komuniciraju sa stanjem i manipuliraju podacima dok su prezentacijske komponente fokusirane samo na prezentaciju podatke koje su dobili od kontejnerske komponente. Tako se kontejnerske komponente nalaze u podmapi `stranice`. Ove kontejnerske komponente dalje uključuju prezentacijske komponente koje se nalaze u mapi `ui`. Te komponente su na primjer kartica lokacije, filter lokacija, sekcija komentara i sl. Ovime se ostvaruje načelo razdvajanje odgovornosti čime se jasno određuje zadaća svake komponente i jednostavno se može predvidjeti ponašanje.

Za upravljanje lokalnim i dijeljenim stanjima korišteni su Angular signali. Oni omogućuju jasno praćenje promjena podataka i jednostavno povezivanja stanja podataka s predloškom unutar kojeg se prikazuju. Signali se unutar aplikacija koriste za čuvanje podataka je li korisnik prijavljen, dohvat i osvježivanje korisničkog profila i sl. Takav pristup omogućuje smanjenu potrebnu da se isti podaci prosljeđuju između komponenti, već u stanju postoji jedan podatak kao jedan izvor istine koji se koristi kroz aplikaciju.

Komunikacija s poslužiteljskim dijelom aplikacije izdvojena je u zasebni podatkovni sloj. Angular komponente ne šalju HTTP zahtjev izravno prema API-u, nego korisničke akcije daju-

signal kontejnerima koji prema spremištu stanja šalju zahtjev za podacima i komunicira sa sučeljima servisa. Svaka od funkcionalnosti ima zaseban servis za komunikaciju s poslužiteljskim dijelom. Kako bi se smanjila ovisnost ostatka aplikacije o konkretnim HTTP servisima, uvedena su sučelja servisa koja su apstraktne klase koje definiraju metode. Određena implementacija sučelja definira svoj način dohvata podatka od poslužitelja, te u koliko se implementacija promijeni ostatak aplikacije ostaje isti. Time se osigurava modularnost, skalabilnost i održivost aplikacije. Za komunikaciju odabran je Angularov `HttpClient` koji učahuruje API rute, HTTP metode i tipove podataka koji se šalju čime se izbjegava ponavljanje API logike i dobiva se jasnije mjesto komunikacije.

6.3.3. Stilovi

Mapa stilova sastoji se od 7 podmapa od kojih svaka predstavlja jedan sloj u ITCSSu (Slika 10). Svaka mapa unutar sebe ima stilove grupirane prema svrsi. Tako u mapi `genericki-stilovi` postoje dvije *.scss* datoteke. Jedna za definiranje osnovnih vrijednosti u obliku CSS varijabli te jedna datoteka koja predstavlja ponovno postavljanje (*css reset*) ili za sloj objekta su definirani stilovi za fleksibilni kontejner, za CSS rešetku, sekcije i ostali stilovi. Ove sve datoteke se uključuju u svoj `_index.scss` unutar podmape (Kod 6.3).

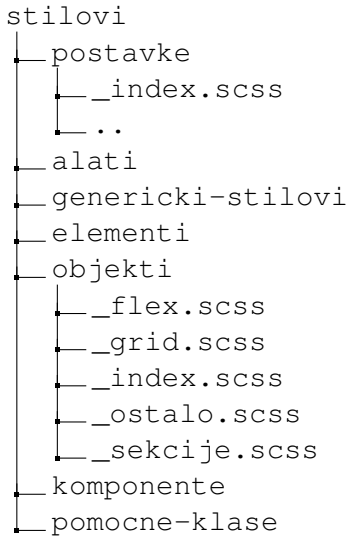
```
1  @forward "flex";
2  @forward "grid";
3  @forward "sekcije";
4  @forward "ostalo";
```

Programski kod 6.3: Uključivanje zasebnih scss datoteka u _index.scss

Zatim da bi svi stilovi bili dostupni u svim stilskim datoteka, moraju se uključiti preko `@use` u glavnu stilsku datoteku Angular projekta `styles.scss` (Kod 6.4)

```
1 @use "stilovi/postavke";
2 @use "stilovi/alati";
3 @use "stilovi/genericki-stilovi";
4 @use "stilovi/elementi";
5 @use "stilovi/objekti";
6 @use "stilovi/komponente";
```

Programski kod 6.4: Uključivanje _index.scss datoteka slojeva u glavnu stilsku datoteku



Slika 10: Organizacija stilova prema ITCSS arhitekturi

Način na koji je ITCSS realiziran za fontove, tako je realiziran i za ostale dijelove aplikacije, jest ovaj. U podmapi `postavke` u datoteci `_tipografijac.scss` definirane su SCSS varijable kojima su pridružene vrijednosti koji se koriste u definiraju tipografije unutar aplikacije. Definirane su vrijednosti obitelji fonta za običan tekst i za naslove. Naziv tih varijabli ima prefiks `ff-` koji označava `font-family`. Isto tako je napravljen za vrijednost prored s prefiksom `lh-` (eng. line-height) i debljinom fonta s prefiksom `fw-` (eng. font-weight) (Kod 6.5)

```
1  $ff-osnovni: "Manrope", sans-serif;
2  $ff-naslovi: "Lora", serif;
3
4  $lh-osnovni: 1.15;
5  $lh-naslovi: 1.3;
6
7  $fw-regulat: 400;
8  $fw-medium: 500;
9  $fw-semibold: 600;
10 $fw-bold: 700;
```

Programski kod 6.5: Definiranje vrijednosti tipografije

Zatim se vrijednosti svih definiranih postavki centralizirano prosljeđuju preko `_index.scss` datoteke u podmapi `postavke` do ostalih dijelova aplikacije kako bi se vrijednosti mogle koristiti (Kod 6.6).

```
1  @forward "tipografija";
2  @forward "razmaci";
```

```
@forward "tocke-prijeloma";
@forward "z-index";
@forward "tranzicije";
```

Programski kod 6.6: Proslijeđivanje dijelova sloja preko `_index.scss`

Zatim se u nižim slojevima ove varijable koriste kako bi se primijenila vrijednost iz jednog centraliziranog mjesta. Kod 6.7 prikazuje primjenu vrijednosti varijabli u ITCSS sloju `elementi` u kojem se definiraju osnovni stilovi za HTML elemente.

```
@use "../postavke" as postavke;
h1, h2, h3, h4 {
  font-family: postavke.$ff-naslovi;
  line-height: postavke.$lh-naslovi;
  color: var(--clr-text);
  font-weight: postavke.$fw-bold;
}
```

Programski kod 6.7: Primjena definiraih varijabli

Prema istom principu se ostale varijable mogu koristiti kroz cijelu aplikaciju.

U ITCSS sloju `objekti` definirani su osnovni stilovi struktura koji će se koristiti kroz aplikaciju, te je taj sloj jedan od temelja za primjenu načela razdvajanja strukture od izgleda. Između ostalog, definirani su stilovi poput: `o-stog` i `o-klaster` koji su temelj za izradu strukture komponente. Prva klasa definira raspored elementa jedan ispod drugoga, dok druga definira raspored elemenata jedan pokraj drugoga s dodanim modifikatorima. Način rasporeda može biti u sredini ili razdvojeno (eng. space-between). Osim toga, razmak se može definira preko CSS varijable unutar elementa u kojem se ta klasa primjenjuje, s tim da se postavljaju sa zamjenskim (eng. fallback) vrijednosti koje se primjenjuju ako vrijednost varijable nije definirana (Kod 6.8). Ta klasa omogućuje da se izgled elementa u potpunost definira unutar zasebne klase elementa, a da se raspored definira unutar zasebne klase.

```
.o-klaster {
  display: flex;
  align-items: center;
  gap: var(--klaster-gap, 1rem);
  &--between {
    justify-content: space-between;
  }
  &--center {
```

```
    justify-content: center;
  }
  &--wrap {
    flex-wrap: wrap;
  }
}
```

Programski kod 6.8: Definiranje klase za raspored elemenata jedan pored drugog

U programskom kodu 6.9 primijenjena je klasa `o-klaster` tako da komponenta `app-ocjena-zvijezdice` i `div` spremnik budu jedna pored drugog. Uz tu klasu dodijeljena je klasa `lokacija-hero__ocjena` u kojoj se definira izgled elementa. U definiranju klasa koristi se okomita crta `|` (eng. pipeline) kao vizualni razdjelnik klasa koje definiraju strukturu od klasa koje definiraju izgled. U definiranju stilova (Kod 6.10) unutar klase `lokacija-hero__ocjena` definira se izgled, u ovom slučaju boja teksta te se dodatno definira razmak između elemenata s CSS varijablom `-klaster-gap`.

```
<div class="o-klaster | lokacija-hero__ocjena">
  <app-ocjena-zvijezdice
    [ocjena]="lokacija().prosjecnaOcjena ?? 0"
    [velicina]="18"
  />
  <div>
    ...
  </div>
</div>
```

Programski kod 6.9: Primjerna klase objekta i razdvjanje strukture od izgleda - HTML

```
.lokacija-hero {
  &__ocjena {
    --klaster-gap: 0.5rem;
    color: white;
    }
}
```

Programski kod 6.10: Primjerna klase objekta i razdvjanje strukture od izgleda - CSS

6.4. Analiza implementiranih stranica

U ovom poglavlju objasnit će se primjena i razlog korištena osnovnih koncepata održivosti programskog koda koji su bili navedeni i objašnjeni u teorijskom dijelu rada.

6.4.1. Analize navigacije

Makar navigacija nije konkretna stranica, ona je prisutna na svim stranicama. Navigacija se ne definira ponovno na svim stranicama, nego je napravljena komponenta glavnog rasporeda aplikacija čiji predložak prikazuje navigaciju neovisno o adresi/URL-u. Navigacija je definirana u zaglavlju nakon koje dolazi glavni sadržaj stranice koji se prikazuje ovisno o adresi. Sadržaj koji će se prikazati određuje Angularov sustav za usmjeravanje (`Angular Router`). Na kraju se nalazi podnožje stranice koje također ne ovisi o adresi te je zato prisutano na svim stranicama.

```
<header class="zaglavlje-stranice">
  <app-navigacija />
</header>

<main class="sadrzaj-stranice">
  <router-outlet />
</main>

<footer>
  <app-podnozje />
</footer>
```

Programski kod 6.11: Definiranje glavnog rasporeda stranica

Sama navigacija je izdvojena u zasebnu komponentu koja se uključuje preko `<app-navigacija />`. Navigacija koja se prikazuje na većim ekranima sastoji se od 3 dijela. Prvi dio jest logotip aplikacije, u sredini se nalaze glavni putanje prema ostalim web stranicama, te s desne stranice se prikazuje gumb za prijavu ili korisnički podaci ovisno o tome je li korisnik prijavljen. Isto tako, ako korisnik nije prijavljen u aplikaciju, na sredini vidi samo poveznicu na lokacije (Slika 11), ako je prijavljen, što se provjerava preko Angularove kontrole strukture `@if` tako da se gleda vrijednost signala `jePrijavljen()` u spremištu stanja za autentikaciju, prikazuje se poveznica na stranicu trgovine. Ako je korisnik prijavljen a je "administrator" (u programskom kodu je to označeno identifikatorom uloge u bazi koji je 2) administratoru se

prikazuje poveznica za upravljanje lokacija što obuhvaća izvršavanje CRUD operacija (Slika 12).

Za prelaz između stranica koristiti koristi se Angularava direktiva `routerLink`, koja u odnosu na uobičajen HTML atribut `href`, ne uzrokuje ponovno učitavanje cijele strance, nego se samo učitava nova komponenta na mjestu `<router-outlet />`. Time se zadržava ponašanje jednostranične aplikacije.

```
<nav class="o-klaster o-klaster--between | navigacija">
  <div class="navigacija__logo-spremnik">
    <a routerLink="/">
      <app-kroz-hrvatsku-logo-ikona sirinaLogo="150" [jeInverz]="true" />
    </a>
  </div>

  <div class="navigacija__linkovi-spremnik">
    <ul class="o-klaster o-lista-pocetno | navigacija__linkovi-lista">
      <li class="navigacija__stavka">
        <a
          routerLink="/lokacije"
          routerLinkActive="active"
          class="navigacija__link"
          >Lokacije</a
        >
      </li>

      @if (authStanje.prijavljen()) {
        <li class="navigacija__stavka">
          <a
            routerLink="/trgovina"
            routerLinkActive="active"
            class="navigacija__link"
            >Trgovina</a
          >
        </li>
      }

      @if (profilStanje.profil(); as profil) {
        @if (profil.korisnik.ulogaId === 2) {
          <li class="navigacija__stavka">
```

```
            <a
              routerLink="/uredivanje/lokacije"
              routerLinkActive="active"
              class="navigacija__link"
              >Uređivanje lokacija</a
            >
          </li>
        }
      }
    </ul>
  </div>

  <div class="navigacija__autentikacija-spremnik">
    @if (authStanje.prijavljen() && profilStanje.profil(); as profil) {
      <!-- prikaz podataka o korisniku -->
    } @else {
      <a
        routerLink="/autentikacija"
        routerLinkActive="active"
        class="c-button c-button--primarna-inverz"
        >Prijava</a
      >
    }
  </div>
  <button
    type="button"
    class="navigacija__hamburger"
    (click)="otvoriMobilniIzbornik()"
  >
    <app-hamburger-ikona boja="#e7f9f3" velicina="48" />
  </button>
</nav>

<app-mobilni-izbornik
  [otvoren]="mobilniIzbornikOtvoren()"
  [prijavljen]="authStanje.prijavljen()"
  [profil]="profilStanje.profil()"
  (zatvori)="zatovriMobilniIzbornik()"
  (odjava)="odjava()"
```

```
/>
```

Programski kod 6.12: Predložak navigacije



Slika 11: Izgled navigacije kada korisnik nije prijavljen



Slika 12: Izgled navigacije kada je korisnik prijavljen

Na manjim ekranima napravljeno je da se srednji i desni dio navigacije ne prikazuju već da se umjesto njih prikazuje hamburger ikona koja je zapravo gumb (Programski kod 6.13). Pritiskom na ovu ikonu, preko Angularovom povezivanja događaja (eng. event binding), poziva se funkcija `otvoriMobilniIzbornik()` koja u klasi komponente mijenja vrijednost signala `mobilniIzbornikOtvoren` na istinu (Programski kod 6.14). Uz to na tijelo stranice (*body*) dodaje se klasa `body-bez-scrolla` koja tijelu dodaje svojstvo `overflow:hidden` što sprječava korisnika da pomiče tijelo stranice.

```
@include alati.ekranManjiOd(56.25rem) {
  .navigacija__linkovi-spremnik,
  .navigacija__autentikacija-spremnik {
    display: none;
  }

  .navigacija__hamburger {
    display: block;
  }
}
```

Programski kod 6.13: Sakrivanje sadržaja navigacije na navigacije na manjim ekranima i dodavlje hamburder ikone

```
export class NavigacijaComponent implements OnInit {
  mobilniIzbornikOtvoren = signal(false);

  otovriMobilniIzbornik(): void {
    this.mobilniIzbornikOtvoren.set(true);
    this.document.body.classList.add('body--bez-scrolla');
  }

```

```
9     zatovriMobilniIzbornik(): void {
10      this.mobilniIzbornikOtvoren.set(false);
11      this.document.body.classList.remove('body--bez-scrolla');
12    }
13  }
```

Programski kod 6.14: Primjena zastavice za prikaza navigacije na manjim ekranima

Na kraj programskog koda 6.12 ubačena je komponenta `<app-mobilni-izbornik />`. Ta komponenta je navigacija na manjima ekrana, koja između ostalog Prisma ulazni parametar `[otvoren]`. Promjenom zastavice u programskom kodu 6.14, nova vrijednost se šalje toj komponenti. Prilikom slanja vrijednosti istine (eng. true), predložak mobilnog izbornika/navigacije (Programski kod 6.15) povezuje logičku vrijednost ulaznog signala `otvoren()` s primjenom klase `mobilni-izbornik` s modifikatorom `- -otvoren`. Navigacija je u početnom stanju pomaknuta izvan desnog ruba stranice pomoću CSS svojstva `transform: translateX(100%)`. Klasa s modifikatorom `- -otvoren` pomiče poziciju navigacija na `0%` tako da je u potpunosti vidljiva na ekranu. Ako korisnik klikne na gumb za zatvaranje navigacije ili klikne van njenog prostora, komponenta `<app-mobilni-izbornik />` emitira signal prema roditeljskoj komponenti navigacije koja mijenja stanje `otvoren` na ne istinu te se time miče klasa sa modifikatorom i mobilni izbornik se zatvara.

Razlog iz kojeg se radila posebna navigacija za manje ekrane jest da navigacija u ovoj aplikaciji sadrži jako puno informacija. Te informacije teško je dostojno prikazati na manjim ekranima, te se je zato koristi potpuno novi izbornik za manje ekrane.

```
1 <div class="mobilni-izbornik" [class.mobilni-izbornik--otvoren]="otvoren()"
      >
2   <!-- predložak navigacije na manjim ekranima -->
3 </div>
```

Programski kod 6.15: Predložak mobilne navigacije



Slika 13: Izgled navigacije na manjim ekranima

6.4.2. Analiza početne stranice

Početna stranica služi za privlačenje korisnika i poticanje na početne akcije prijave i pregleda lokacija. Zato početna stranica u velikoj većini ima isključivo dekorativne komponente koje nemaju previše logike, nego samo sadržaj i stilove koje privlače korisnika. Početna stranica sastoji se od 5 dijelova, odnosno 5 sekcija koja su odvojene u zasebne komponente, te time komponenta početne stranice ima ulogu spremnika ili kontejnerske komponente. Te komponente poredane su jedna ispod druge, pa se zato koristi klasa objektnog sloja ITCSS `o-stog` (Programski kod 6.16).

```
<div class="o-stog | pocetna-stranica-spremnik">
  <app-hero-sekcija [jePrijavljen]="authStanje.prijavljen()" />

  <app-lokacije-sekcija
    [odabranaZupanija]="lokacijaStanje.odabranaZupanija()"
    [lokacijeZupanije]="lokacijaStanje.topLokacijeZupanije()"
    [ucitavanjeLokacija]="lokacijaStanje.ucitavanjeTopLokacijaZupanije()"
    (zupanijaOdabrana)="odaberiZupaniju($event)"
  />

  <app-posjet-sekcija />

  <app-tvoja-prica />

```

```
  <app-zapocni-putovanje />
</div>
```

Programski kod 6.16: Struktura početne stranice

Jedna sekcija koja ima logike jest sekcija `app-lokacija-sekcija` unutar koje prikazuje kartica Hrvatske sa županijama. Korisnik može kliknuti na neku od županije te se time korisniku prikazuje najbolje ocijenjena lokacija te županije. Pri prvom dolasku na stranicu niti jedna županije nije odabrana te korisnik samo vidi tekst koji ga upućuje na to da klikne na neku od županija. Slika 14 prikazuje to stanje te sekcije.



Slika 14: Izgled sekcije lokacija unutar početne stranice

Komponenta `<app-lokacija-sekcija />` sastoji se od lijevog dijela koji predstavlja sadržaj koji se prikazuje (div spremnik s klasom `sekcija-lokacije__sadrzaj`) i desnog dijela koji uključuje komponente karte . Unutar spremnika sadržaja koristi se kontrolna struktura `if` unutar koje se provjerava je li signal `odabranaZupanija()` postavljen. Ako jest prikazuje se kartica lokacija, a ako nije korisniku se prikazuje tekst da klikne na županiju.

```
<section class="sekcija-lokacije">
  <div class="sekcija-lokacije__zaglavlje">
    <div class="sekcija-lokacije__sadrzaj">
      <h2>Otkrij najbolje lokacije u Hrvatskoj</h2>
      @if (odabranaZupanija(); as zupanija) {
        <div class="sekcija-lokacije__rezultati">
          <app-kartica-lokacije [lokacija]="lokacija" />
          ...
        </div>
      } @else {
```

```
11          <span
12            >Kliknite na željenu županiju i vidite najpopularnije lokacije<
    /span
13          >
14        }
15      </div>
16
17      <app-karta-hrvatske (zupanijaOdabrana)="zupanijaOdabrana.emit($event)"
    />
18    </div>
19  </section>
```

Programski kod 6.17: Struktura predloška sekcije lokacija

Komponenta `<app-karta-hrvatske />` sastoji se od predloška u kojem se nalazi karta Hrvatske u formatu skalabilne vektorske grafije (svg). Svakoj putanji, odnosno liku županije, dodan je Angular atribut `(click)` koji omogućuje korisniku da klikne na taj lik i da se dogodi neka akcija (Programski kod 6.18).

```
1  <path
2    id="Varaždinska"
3    class="karta-hrvatske__zupanija"
4    role="button"
5    (click)="odaberiZupaniju('Varaždinska', $event)"
6    ...
7    d="M236.239 46.904L235.77 ..."
8    />
```

Programski kod 6.18: Programski kod putanje za link jedne županije

Korisnikovim pritiskom na određenu putanju na grafici, poziva se metoda `odaberi Zupaniju()` definirana unutar klase komponente. Ova metoda Prisma naziv županije i sam element na koji je korisnik pritisnuo. Taj element se sprema u atribut klase `aktivnaZupanija Element` tako da se kasnije može maknuti klasu koja se dodaje. Ta klasa koja se doda jest klasa `karta-hrvatske__zupanija- -aktivna` koja ima BEM modifikator `- -aktivna`. Između ostalog ova klasa elementu mijenja pozadinsku boju čime korisniku daje jasno informaciju koja je županija odabrana. U konačnici metoda emitira događaj kliknute županije prema njoj nadređenoj komponenti (Programski kod 6.19) koja je `<app-lokacija-sekcija />` i u njenom predlošku se može vidjeti da ta komponenta prima emitirani događaj `zupanijaOdabrana()`

tako da Prisma naziv županije koja joj je proslijeđena.

```
export class KartaHrvatskeComponent {
  @Output() zupanijaOdabrana = new EventEmitter<string>();

  private aktivnaZupanijaElement: Element | null = null;

  odaberiZupaniju(zupanija: string, event: Event): void {
    this.aktivnaZupanijaElement?.classList.remove(
      'karta-hrvatske__zupanija--aktivna',
    );

    const element = event.currentTarget as Element;
    element.classList.add('karta-hrvatske__zupanija--aktivna');
    this.aktivnaZupanijaElement = element;

    this.zupanijaOdabrana.emit(zupanija);
  }
}
```

Programski kod 6.19: Programski kod klase komponente `app-karta-hrvatske`

Dalje predložak `<app-lokacija-sekcija />` emitira taj isti događaj prema svojoj nadređenoj klasi (Programski kod 6.20) zato što ona sama nije kontejnerska te iz tog razloga ne smije komunicirati s ostatkom aplikacije, nego je obična komponenta kojoj je zadatak prikaz sadržaja, te zato emitira događaj prema komponenti početne stranice koja je kontejnerska te ona komunicira s ostatkom aplikacije kako bi dobila potrebne informacije

```
<app-karta-hrvatske (zupanijaOdabrana)="zupanijaOdabrana.emit($event)" />
```

Programski kod 6.20: Emitiranje naziva županije prema iz `app-lokacija-sekcija` komponente prema komponenti početne stranice

Zatim predložak početne stranice Prisma emitirani sadržaj (Programski kod 6.16) iz komponente `<app-lokacije-sekcija />` te taj sadržaj prosljeđuje metodi `odaberiZupaniju()`. Ta metoda nalazi se u klasi komponente te Prisma naziv županije. Klasa komponente je prethodno injektirala servis stanja za lokacije koja ima metodu `ucitajTopLokacijePoZupaniji()` i Prisma naziv županije. Servis stanja nije injektiran klasično preko konstruktora nego funkcijski.

```
export class PocetnaComponent {
  readonly lokacijaStanje = inject(LokacijaStanjeService);
```

```
3
4     odaberiZupaniju(zupanija: string): void {
5       this.lokacijaStanje.ucitajTopLokacijePoZupaniji(zupanija);
6     }
7   }
```

Programski kod 6.21: Poziv spremišta za dohvat podataka

Spremište stanja prvo injektira sučelje servisa `LokacijeIServis` unutar kojeg su definirane metode za dohvat podataka. Također, definira se signal `topLokacijeZupanije` koji je niz tipa podataka `Lokacija` definiranog u modelu (Programski kod 6.22). Metoda `ucitajTopLokacijePoZupaniji()` poziva metodu `dohvatiSveLokacije()` sučelja servisa za rad s lokacijama te joj prosljeđuje naziv županije. Zato što se koristi Angularov HttpClient, Http poziv vraća vrijednost `Observable` te da bi se do konačne vrijednosti došlo potrebno se pretplatiti kako bi se ostatak metode mogao izvršiti kada dođe odogovor iz baze. Ako je odogovor zahtjeva uspješan, izvršava se `next` dio koji poziva funkciju povratnog poziva (eng. callback function) kojoj se prosljeđuju vrijednosti iz baze. Lokacije koje je baza vratila se prvo sortiraju po prosječnoj ocjeni silazno, tako da je prvo lokacija s najboljom ocjenom te se u konačnici uzima samo prva lokacija koja se sprema u signal `topLokacijeZupanije` koji kontejnerska komponenta čita. Ukoliko je došlo do pogreške, poziva se funkcija povratnog poziva definiranog u `error:` dijelu u kojem se signal postavlja na prazni niz i postavlja se poruka greške (Programski kod 6.23).

```
export type Lokacija = {
  lokacijaId: number;
  naziv: string;
  opis?: string | null;
  ...
  slikeLokacije: SlikaLokacije[];
};
```

Programski kod 6.22: Model podataka lokacije koja se dohvaća

```
export class LokacijaStanjeService {
  private readonly lokacijeIServis = inject(LokacijeIServis);
  readonly topLokacijeZupanije = signal<Lokacija[]>([]);

  ucitajTopLokacijePoZupaniji(zupanija: string): void {
    this.lokacijeIServis.dohvatiSveLokacije({ zupanija }).subscribe({
```

```
      next: (lokacije) => {
        this.topLokacijeZupanije.set(
          lokacije
            .sort((a, b) => (b.prosjecnaOcjena ?? 0) - (a.prosjecnaOcjena
    ?? 0))
            .slice(0, 1),
        );
      },
      error: () => {
        this.topLokacijeZupanije.set([]);
        this.greska.set('Došlo je do pogreške pri učitavanju lokacija');
      },
    });
  }
}
```

Programski kod 6.23: Poziv servisa za dohvat podatak iz baze u spremištu

Sučelje servisa `LokacijeIServis` je apstraktna klasa koja ima definirane metode bez njihove implementacije. Sučelje samo navodi koje metode implementacija treba imati, pa tako za ovaj primjer definira apstraktnu metodu `dohvatiSveLokacije()` koja kao argument Prisma filter, odnosno u kontekstu primjena naziv županije, te vraća listu lokacija (Programski kod 6.24).

```
export abstract class LokacijeIServis {
  abstract dohvatiSveLokacije(filter?: FilterLokacija): Observable<Lokacija
    []>;
}
```

Programski kod 6.24: Sučelje servisa za lokacije

Konkretna implementacija ovog sučelja nalazi se u `LokacijeHttpService`. To je servis koji implementni sučelje i implementira njegove metode. Unutar implementacije injektira se HttpClient za komunicira s poslužiteljom. Unutar metode prvo se kreiraju HTTP parametri te ako je u filteru definirana županija, ona se dodaje u te parametre. Nakon toga se dalje šalje `GET` zahtjev prema ruti `/lokacije`. Uz samu rutu prosljeđuju se i parametri u objekta. Unutar rute koristi se globalna varijabla `API_URL` u kojoj je definirana početna putanja na kojoj se nalazi poslužitelje (na primjeru aplikacije to je `http://localhost:3000`) (Programski kod 6.25).

```
export class LokacijeHttpService implements LokacijeIServis {
```

```
  private readonly http = inject(HttpClient);

  dohvatiSveLokacije(filter?: FilterLokacija) {
    let params = new HttpParams();

    if (filter?.zupanija) {
      params = params.set('zupanija', filter.zupanija);
    }
    // + ostali filteri

    return this.http.get<Lokacija[]>(`${API_URL}/lokacije`, { params });
  }
}
```

Programski kod 6.25: Implementacija servisa za dohvat lokacija

Konkretno povezivanje implementacije i sučelja radi se u konfiguracijskoj datoteci Angular projekta: `app.config.ts`. Programski kod 6.26 omogućuje da kad se injektira sučelja da Angular prepozna da mora primijeniti implementaciju iz `LokacijeHttpService`.

```
{
  provide: LokacijeIServis,
  useClass: LokacijeHttpService,
}
```

Programski kod 6.26: Povezivanje sučelja i implementacije servisa

Na poslužitelju zahtjev se obrađuje kroz kontroler i servis. U kontroleru se definira ruta, u ovom primjer `GET lokacije` te Prisma moguće filtere preko parametara upita. Nakon toga se poziva funkcija za određenu rutu koja poziva funkciju iz servisa, koju je prethodno injektirao, kojoj prosljeđuje filter. Servis dalje koristi Prismu kako bi preko *where* klauzule dohvatio lokaciju za određeni parametar.

```
@Controller('lokacije')
export class LokacijeController {
  constructor(private readonly lokacijeServis: LokacijeService) {}

  @Get()
  dohvatiSve(@Query() filter: FiltrirajLokacijeDto) {
    return this.lokacijeService.dohvatiSveLokacije(filter);
  }
```

```
}
```

Programski kod 6.27: Programski kod rute za dohvat lokacija u kontrolenu na poslužitelju

```
dohvatiSveLokacije(filter: FiltrirajLokacijeDto) {
  return this.prisma.lokacija.findMany({
    where: {
      zupanija: filter.zupanija,
    },
      include: {
        kategorija: true,
        slikeLokacije: true,
    },
  });
}
```

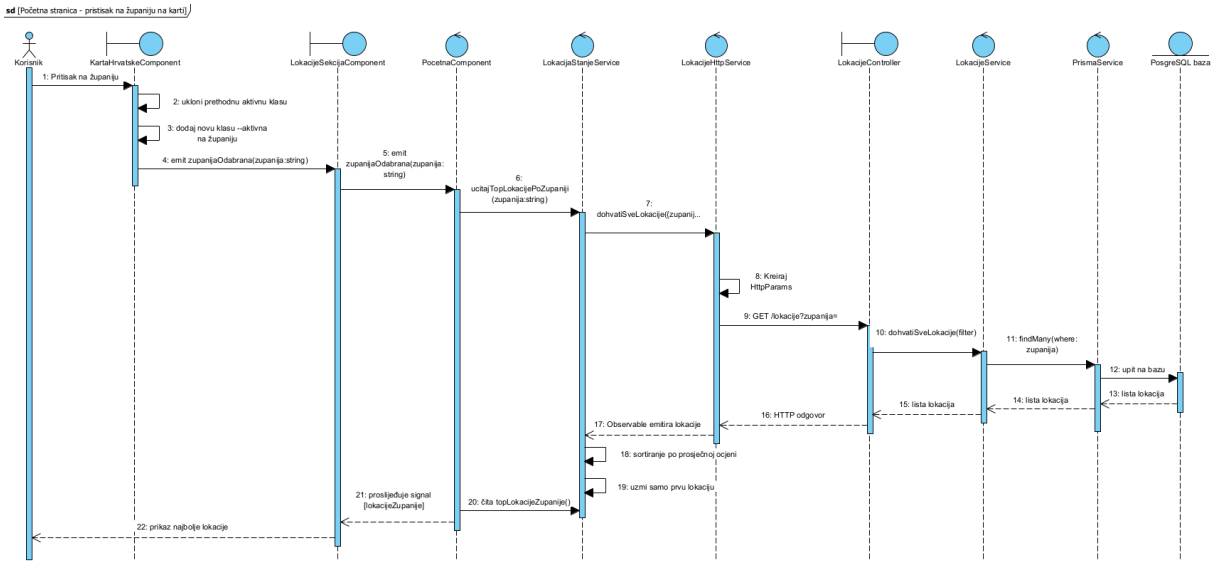
Programski kod 6.28: Programski kod funkcije za dohvat lokacija iz baze na poslužitelju

Nadalje se podaci iz baze propagiraju natrag do spremišta gdje se spremaju u signal `topLokacijeZupanije`. Taj signal čita kontejner komponenta početne stranice (Programski kod 6.16) te tu vrijednost prosljeđuje prema komponenti preko `[odabranaZupanija]="lokacijaStanje.odabranaZupanija()"` ulaznog atributa komponente.

Zatim vrijednost `odabranaZupanija()` iz programskog koda 6.17 postaje stvarna vrijednost te se komponenti kartice županije prosljeđuje vrijednosti lokacije. Konačno na kraju korisnik vidi najbolje ocijenjenu lokaciju županije, te na karti vidi istaknutu županiju koju je odabrao (Slika 15). Slika 16 prikazuje dijagram slijeda za navedeni proces nakon korisnikovog pritiska na županiju na karti. Ovakav slijed izvođenja kroz arhitekturu aplikacije provodi se kroz sve funkcionalnosti aplikacije.



Slika 15: Izgled sekcije lokacija kada je županija odabrana



Slika 16: Dijagram slijeda na pritisak županije na karti

6.4.3. Analiza stranice popisa lokacija

Stranica popisa lokacija služi za pregledavanje svih dostupnih lokacija unutar aplikacije. Ova stranica omogućuje pretraživanje lokacija po nazivu, gradu ili županiji te filtriranje lokacija po kategoriji. Ova stranica, odnosno kontejnerska komponenta sastoji se od forme koja se korsiti za definiranje filtera te popisa lokacija u obliku rešetke (Programski kod 6.29).

```
<section class="o-stog | lokacije-stranica">
  <header class="o-stog | lokacije-stranica__zaglavlje">
    <h1>Lokacije</h1>
  </header>

```

```
  <app-filter-lokacija-forma
    [filter]="filter"
    [kategorije]="kategorije()"
    (primijeni)="primijeniFilter($event)"
    (resetiraj)="resetirajFilter()"
  />

  @if (ucitavanje()) {
    <p>Učitavanje lokacija</p>
  } @else if (greska()) {
    <p>{{ greska() }}</p>
  } @else {
    <div class="lokacije-stranica__resetka">
      @for (lokacija of lokacije(); track lokacija.lokacijaId) {
        <app-kartica-lokacije
          [lokacija]="lokacija"
          [rijesena]="dolasci.jeLokacijaRijesena(lokacija.lokacijaId)"
        />
      }
    </div>
  }
</section>
```

Programski kod 6.29: Kontejnerska komponenta stranice popisa lokacija

Forma za filtriranje korisniku omogućuje unos naziva lokacije, grada ili županija. Naziv ne mora biti u cijelosti napisan, te je moguće pretraživanje i ako se unesu samo prva slova. Odabir kategorije županije radi se preko padajućeg izbornika. Nakon što korisnik unesene željene vrijednosti u formu filtera i primjeni ga, vrijednosti se ne koriste lokalno unutar komponente, nego se zapisuju u URL kao parametri upita. Time se omogućuje da primijenjeni filter bude vidljiv u adresi stranice, pa korisnik pri osvježavanju ili dijeljenju poveznice ne ostaje bez njega. Također, za filtriranje nije potrebno koristiti formu, nego je moguće ručno unijeti filter u adresnu traku te će se za ispravan unos prikazati filtrirane lokacije.

```
  /lokacije?pretraziNaziv=Slje&grad=Zagreb&kategorija=Planina
```

Programski kod 6.30: Primjer URL-a s parametrima za filtriranje lokacija

U ovom primjeru (Programski kod 6.30) `lokacije` je osnovna ruta. Nakon nje dolazi znak upitnik (`?`) koji označava da nakon njega dolaze parametri upita. U ovom primjer svaki

parametar predstavlja jedan filter koji se šalje prema poslužitelju. Parametri su odvojeni znakom ampersand (`&`), a svaki se sastoji od naziva parametra i njegove vrijednosti. Pa se tako traži lokacija koja u nazivu ima "*Slje*", grad "*Zagreb*" te da je kategorije planina.

Čitanje parametara upita preko URL-a stranice moguće je preko Angularovog *ActivatedRoute* servisa. Kada se stranica učita, definirano unutar `ngOnInit`, komponenta pročita postojeće parametre iz URL-a i prema njima postavi početne vrijednosti forme. Nakon toga se poziva spremište stanja koje dohvaćanja lokacije prema zadanim filterima. Time se povezuje stanje URL-a, stanje forme i podaci koji se prikazuju korisniku (Programski kod 6.31).

```
export class ListaLokacijaComponent {
  private readonly lokacijaStanje = inject(LokacijaStanjeService);
  private readonly route = inject(ActivatedRoute);
  filter: FilterLokacija = {
    pretraziNaziv: '',
    zupanija: '',
    grad: '',
    kategorija: '',
  };
  this.route.queryParams.subscribe((params) => {
      this.filter = {
        pretraziNaziv: params['pretraziNaziv'] ?? '',
        zupanija: params['zupanija'] ?? '',
        grad: params['grad'] ?? '',
        kategorija: params['kategorija'] ?? '',
      };

      this.lokacijaStanje.ucitajLokacije(this.ocistiFilter(this.filter));
    });
}
```

Programski kod 6.31: Postavljanje vrijednosti filtera i dohvaćanje lokacije prema zadanom filteru

Kada korisnik promijeni vrijednosti u formi i klikne na gumb za primjenu filtera, komponenta ne mora izravno dohvaćati podatke, nego najprije ažurira parametre upita u ruti te ta promjena ponovno pokreće dohvat lokacija prema novim parametrima. Tako URL postaje jedini izvor informacija o aktivnim filterima. Funkcija `ocistiFilter()` služi kako bi se iz objekta filtera uklonili filteri bez vrijednosti, tako u URL-u programskog koda 6.30 nema parametra za županiju, nego opće nije poslan jer nije opće definiran.

```
primijeniFilter(filter: FilterLokacija): void {
  const ocisceniFilter = this.ocistiFilter(filter);

  this.router.navigate([], {
    relativeTo: this.route,
    queryParams: ocisceniFilter,
  });
}
private ocistiFilter(filter: FilterLokacija): FilterLokacija {
  return Object.fromEntries(
    Object.entries(filter).filter(([, vrijednost]) => vrijednost),
  ) as FilterLokacija;
}
```

Programski kod 6.32: Funkcija za primjenu filtera

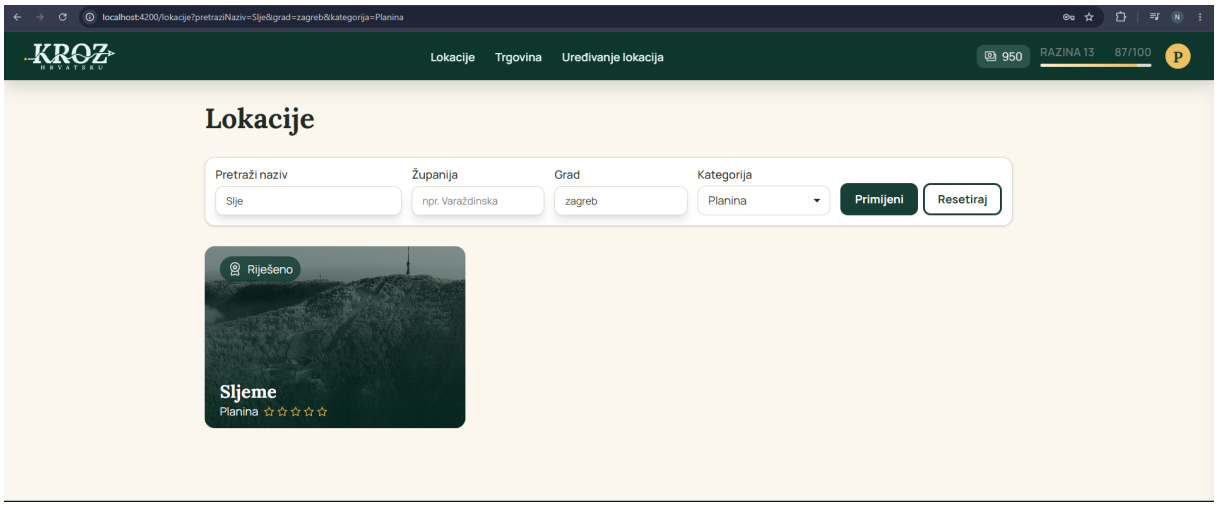
Filtriranje se dalje prenosi prema poslužitelju preko servisnog sloja koji objekt filtera pretvara u parametre upita HTTP zahtjeva.

```
dohvatiSveLokacije(filter?: FilterLokacija) {
  let params = new HttpParams();

  if (filter?.pretraziNaziv) {
    params = params.set('pretraziNaziv', filter.pretraziNaziv);
  }
  if (filter?.zupanija) {
    params = params.set('zupanija', filter.zupanija);
  }
  if (filter?.grad) {
    params = params.set('grad', filter.grad);
  }
  if (filter?.kategorija) {
    params = params.set('kategorija', filter.kategorija);
  }

  return this.http.get<Lokacija[]>(`${API_URL}/lokacije`, { params });
}
```

Programski kod 6.33: Definiranje parametara upita za rutu poslužitelja



Slika 17: Primjena filtera iz programskog kod 6.30

Drugi važan aspekt stranice je prikaz lokacija u obliku kartica koje su raspoređene pomoći CSS rešetke. Kartica lokacija izdvojena je u zasebnu komponentu kako bi se mogla ponovno koristiti na više mjesta u aplikaciji. Tako se komponenta kartice koristi na navedenoj stranici popisa lokacija, na početnoj stranici kod prikaza najbolje lokacije za odabranu županiju (Slika 15) te na profilu korisnika kod prikaza riješenih lokacija. Time se smanjuje dupliciranje HTML-a i SCSS-a, a izgled lokacije ostaje dosljedan kroz cijelu aplikaciju.

Kartica lokacija (Programski kod 6.29, `<app-kartica-lokacije />`) prima podatke kroz ulazni atribut te on ne dohvaća podatke samostalno, pošto je ona isključivo prezentacijska komponenta. Ona samo prikazuje podatke koje dobije od roditeljske komponente. Kartica može primiti dvije ulazne vrijednosti od koje je ulazna vrijednost `[lokacija]` obavezna. Kroz ovaj atribut šalje se objekt lokacije s podacima koji se prikazuju, te ukoliko nema podataka, nema potrebne prikazivati karticu. Također može primati neobavezni ulazni atribut `[rijesena]` koja predstavlja logičku vrijednost. Ako je vrijednost "istina", korisnik vidi karticu na kojoj ima bedž "Riješeno" (Druga kartica na slici 18). Ako se ova vrijednost ne proslijedi ili se proslijedi vrijednost "laž" korisniku se prikazuje kartica s bedževima iskustvenih bodova i novca (Prva kartica na slici 18).



Slika 18: Prikaz dizajna kartica lokacije kada ona nije riješena i kada jest

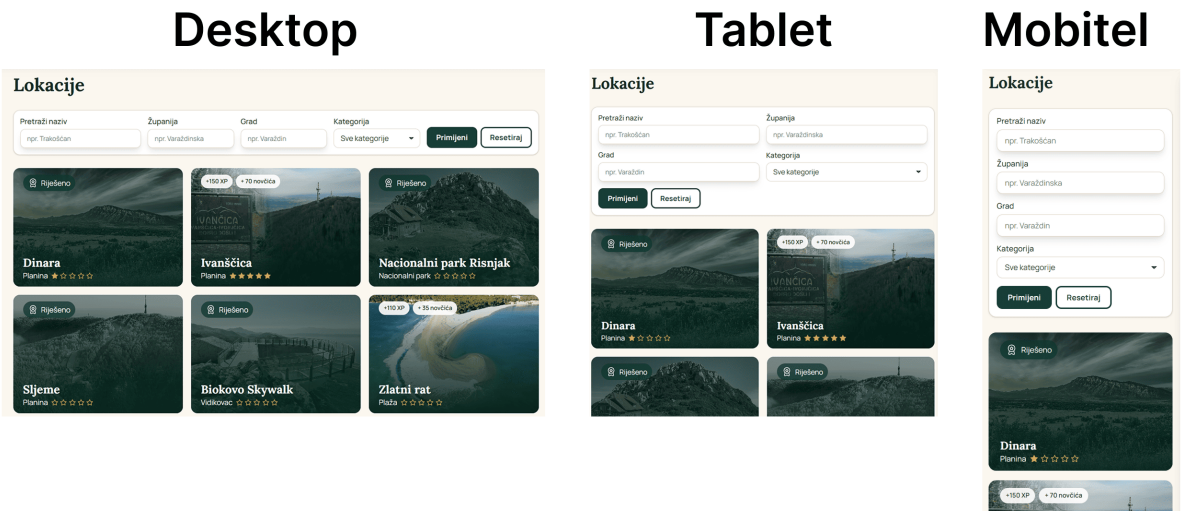
Kako je prethodno bilo spomenuto, popis lokacija kartica se na stranici prikazuje u obliku CSS rešetke. Prikaz više lokaciju unutar rešetke radi se preko Angularove kontrole strukture `for()` (Programski kod 6.29). Za responzivan raspored koristi se svojstvo `grid-template-columns` s funkcijama `repeat()`, `auto-fill` i `minmax()` objašnjenim u poglavlju 5.2.4 Fluidni raspored elemenata. Glavni spremnik lokacija dodatno je ograničen na 80 rem jedinica (1280px) na većim ekranima te je s `margin-inline: auto` horizontalno centriran, dok je na ekranima manjim od navedene vrijednosti on maksimalne širine. Preko `minmax` funkcije definirano je da svaki stupac ima minimalnu širinu od 20 rem jedinica (320px), dok se njegova širina može povećati i ravnomjerno zauzeti preostali raspoloživi prostor. Ako nema prostora za više stupaca, širina tog jednog određena je s `1fr` (Slika 19).

```
.lokacije-stranica {
  --stog-gap: 2rem;

  width: min(100% - 2rem, 80rem);
  margin-inline: auto;
  padding-block: 2rem;

  &__resetka {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(20rem, 1fr));
    gap: 1.25rem;
  }
}
```

Programski kod 6.34: Definicija rešetke za popis kartica lokacija



Slika 19: Raspored kartica lokacija ovisno o širini ekrana

6.4.4. Analiza stranice detalja

Stranica detalja lokacije je najsloženija stranica unutar aplikacije jer objedinjuje više funkcionalnosti vezanih uz jednu lokaciju. Na toj se stranici prikazuju osnovni podaci o lokaciji, glavna slika, galerija slika, karta, informacije o nagradama, status riješenosti izazova, komentari korisnika, prosječna ocjena lokacija... Zbog tolike količine sadržaja i različitih vrsta interakcija, stranica je podijeljena na više manjih komponenti, pri čemu je glavna komponenta stranice detalja lokacije kontejnerska komponenta. Stranica je podijeljena na hero sekciju lokacije, informacije o lokaciji, opis lokacije, galeriju slika lokacije, karticu izazova i sekciju komentara (Slika 20). Ova raspored i podjela omogućuju da svaka komponenta ima jasnu odgovornost.

```
<div class="detalji-lokacije__spremnik">
  @if (lokacija(); as lokacija) {
    <div class="detalji-lokacije__zaglavlje-spremik">
      <app-lokacija-hero
        [lokacija]="lokacija"
        (slikaKliknuta)="otvoriGaleriju($event)"
      />

      <section class="lokacija-informacije">
        <app-lokacija-informacije
          [lokacija]="lokacija"
          [rijeseno]="!!dolasciStanje.mojDolazak()"
        />
      </section>

```

```
      <section class="lokacija-sadrzaj">
        <div class="lokacija-sadrzaj__spremnik">
          <div class="lokacija-sadrzaj__glavni-stupac">
            <app-lokacija-opis [lokacija]="lokacija" />

            <app-karta-lokacije [lokacija]="lokacija" />

            <app-lokacija-galerija
              [lokacija]="lokacija"
              (slikaKliknuta)="otvoriGaleriju($event)"
            />

            <app-komentari-sekcija [lokacijaId]="lokacija.lokacijaId" />
          </div>

          <aside class="lokacija-sadrzaj__bocni-stupac">
            <app-kartica-izazova-lokacije
              class="lokacija-sadrzaj__kartica-izazov"
              [lokacija]="lokacija"
              [rijesena]="!!dolasciStanje.mojDolazak()"
              [spremanje]="dolasciStanje.spremanjeDolaska()"
              (zabiljeziDolazak)="dolasciStanje.otvoriModelDolaska()"
              (pogledajUspomene)="dolasciStanje.otvoriModelUspomena()"
            />
          </aside>
        </div>
      </section>
    </div>
  } @else {
    <p>lokacija nije pronađena</p>
  }
</div>
```

Programski kod 6.35: Predložak kontejnerske komponente stranice detalja lokacije

Važan dio stranice i aplikacije jesu komentari, ocjene, a time i prosječna ocjena lokacije. Prosječna ocjena prikazuje se na više mjesta unutar stranice, na primjer u hero sekciji i sekciji komentara. Isto tako korisnik može dodati novi komentar te je zatim potrebno na svim mjestima ažurirati prosječnu ocjenu. Zbog toga nije dobro da svaka komponenta zasebno računa i

spremu svoju verziju prosječne ocjene jer može doći do nekonzistentnosti u prikazu. Umjesto toga koristi se princip jednog izvora istine. Za ovaj primjer, jedan izvor istine je spremište stanja lokacije u kojoj se nalazi signal u koji se sprema trenutačna lokacija s njezinim podacima uključujući i prosječnu ocjenu. Kada korisnik doda, obriše ili uredi ocjenu, samo se u spremištu stanja ažurira prosječna ocjena, a komponente samo čitaju i prikazuju novu izračunatu vrijednost.



Dvorac Trakošćan

★★★★☆ 4.2(6 glasova)

Adresa
Trakošćan 4, Bednja

Cijena ulaznice
10.50€ po osobi

Kategorija
Dvorac

Status
Riješeno

Osnovne informacije o Dvorac Trakošćan

Dvorac Trakošćan jedan je od najpoznatijih i najromantičnijih dvoraca u Hrvatskoj. Smješten je u Hrvatskom zagorju, okružen šumom, brežuljcima i jezerom koje dodatno naglašava njegov bajkovit izgled. Današnji izgled dvorac je dobio u 19. stoljeću, kada je obnovljen u neogotičkom stilu, dok njegovi počeci sežu još u srednji vijek. Zbog očuvane arhitekture, bogate povijesti i slikovitog krajolika, Trakošćan je jedna od najprepoznatljivijih kulturno-povijesnih lokacija u Hrvatskoj.
Unutrašnjost dvorca uređena je kao muzej te prikazuje način života plemićke obitelji Drašković, koja je stoljećima bila povezana s ovim prostorom. Posjetitelji mogu razgledati zbirke oružja, namještaja, portreta i drugih povijesnih predmeta, dok šetnica oko jezera pruža miran doživljaj prirode i pogled na dvorac iz različitih perspektiva. Trakošćan je zato idealna lokacija za istraživanje hrvatske povijesti, arhitekture i prirodnih ljepota u jednom posjetu.

IZAZOV ZAVRŠEN
Pogledajte uspomenu

Zabilježi dolazak na lokaciju "Dvorac Trakošćan" tako da posjetiš ovu lokaciju kategorije Povijesni dvorci i utvrde te zatim potvrdi svoj dolazak. Također možeš ostaviti slike lokacije kako bi tvoje uspomene još dugo trajale

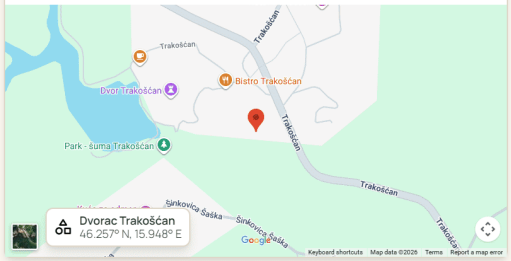
Iskustvo +120XP

Nagrade +50 novčića

Pogledaj uspomene

Mapa lokacije

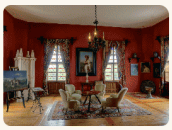
Kako doći



Galerija

Prikaži svih 5 fotografija/e







Ocjene posjetitelja

4.2
★★★★☆
6 komentara

5
4
3
2
1

Ostavite Vaš doživljaj

VAŠA OCJENA
☆☆☆☆☆

VAŠ KOMENTAR

Podijelite svoje iskustvo s drugima

Objavi

Pero 08.08.2026. ★★★☆☆
asdasdasdasdasd

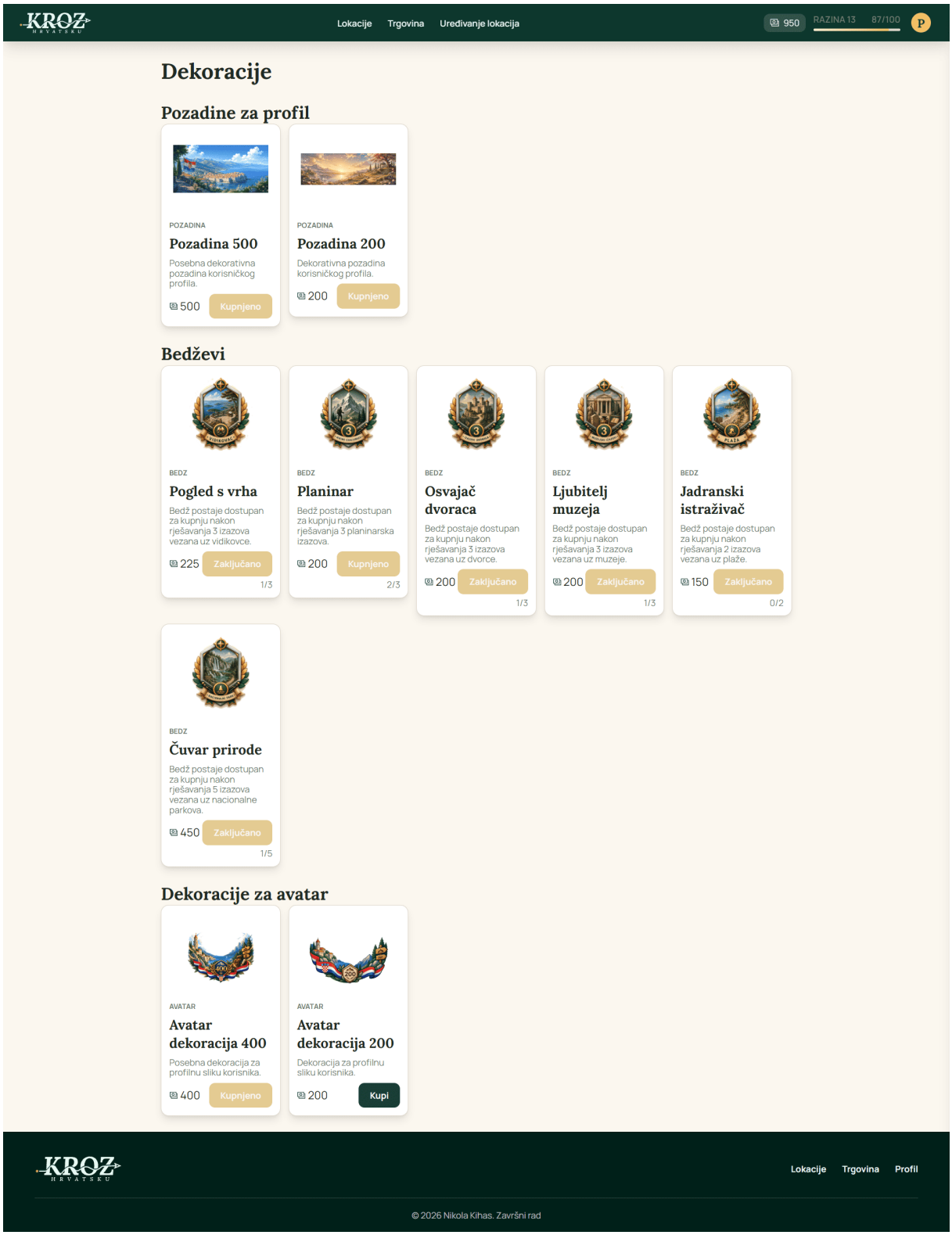
Ivo 04.08.2026. ★★★★★
Prekrasna lokacija, sve je uredno i dobro označeno. Svakako preporučujem posjet.

Ivo 04.08.2026. ★★★★★
Odlično mjesto za izlet. Pogled je prekrasan, a pristup lokaciji je jednostavan.

Prikaži više (3/6)

6.4.5. Analiza stranice trgovine

Stranica trgovine prikazuje dekoracije koje korisnik može kupiti pomoću virtualnog novca osvojenog rješavanjem izazova unutar aplikacije. Dekoracije se dohvaćaju s poslužitelja i grupiraju prema tipu. Dekoracije mogu biti tipa: bedž, pozadine profila ili dekoracije avatara. Gumb na kartica mijenja svoje stanje ovisno ima li korisnik već kupljenu dekoraciju ("Kupljeno"), ako uvjeti za kupnju dekoracije nisu ispunjeni ("Zaključano") te ako je dekoracija dostupna za kupnju prikazuje se tekst "Kupi". Nakon uspješne kupnje ažurira se stanje trgovine i ponovno se dohvaća profila korisnika kako bi mogao primijeniti kupljenu dekoraciju. Time se osvježava i broj novaca unutar aplikacije koji je prikazan u navigaciji. Za ovu funkcionalnost najvažnija je poslovna logika koja se definira na poslužitelju, no pošto to nije tema ovog završnog rada neće se detaljnije opisivati. Korisnički dio služi za prikaz informacija o dekoracijama u obliku kartica koje na klik emitiraju događaja do kontejnerske komponente trgovine koja dalje usmjerava rad sprema spremištu, servisu i poslužitelju. Dekoracije su unutar svojih zasebnih tipova prikazane u rasporedu rešetke te se responzivnost realizira kroz CSS funkcije `repeat`, `auto-fill` i `minmax` isto kao je objašnjeno u poglavlju Analiza stranice popisa lokacija 6.4.3.



Slika 21: Izgled stranice trgovina

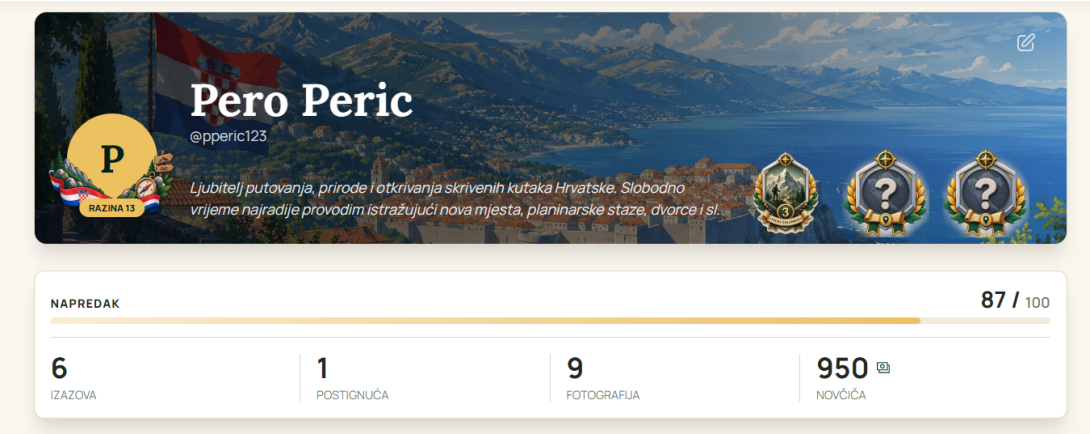
6.4.6. Analiza stranice profil

Stranica profil namijenjena je da korisnik na jednom mjestu može pronaći sve podatke povezane s njegovim profilom. U to spadaju osobni podaci poput imena i prezimena, koris-

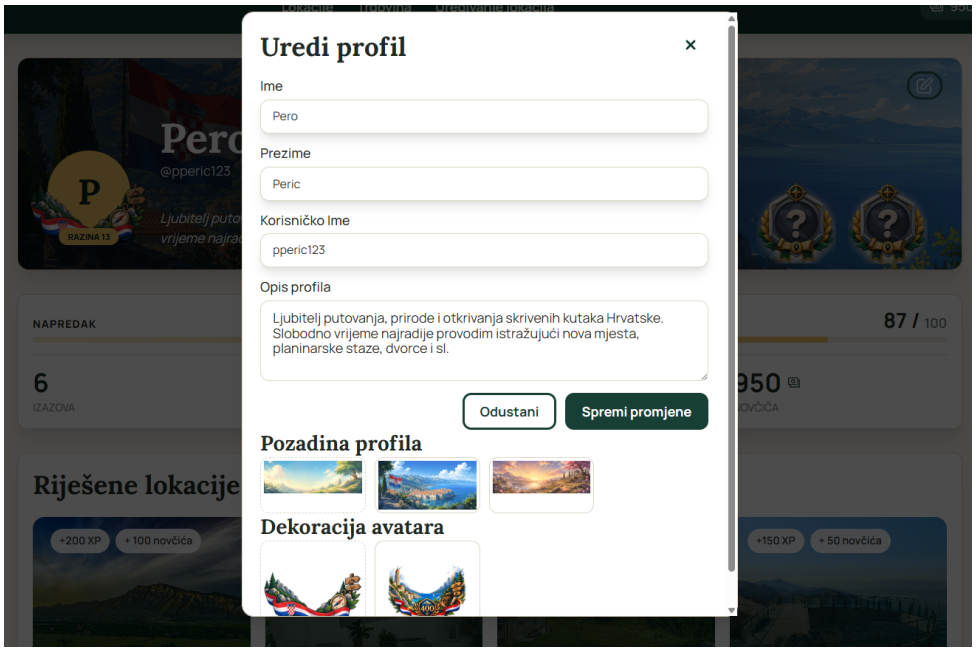
ničkog imena, opisa profila, pa zatim podatke vezane uz aplikaciju kao što su broj riješenih izazova, postignuća, objavljenih fotografija, novaca, razina profila, posjećene lokacije i objavljene fotografije.

Osim toga glavna funkcionalnost profila jest personalizacija tako da korisnik može uređivati profil dekoracijama. Kao što je prije bilo navedeno, dekoracije se kupuju unutar stranice trgovine te one mogu biti tipa pozadina profila, dekoracija avatara ili bedževi. Ove dekoracije se mogu vidjeti u zaglavlju stranice na slici 22. Također korisnik može uređivati podatke profila, mijenjati profilnu sliku ili primijeniti odabranu dekoraciju (Slika 23).

Kako bi se ovo sve povezalo u jednu cjelinu, ključna je primjena spremišta stanja profila koji upravlja radom podacima, poziva funkcije u servisu koji radi upite prema poslužitelju te se kroz kartice lokacije prikazuje ponovna iskoristivost i korisnost razdvajanja sustava na komponente.



Slika 22: Izgled zaglavlja stranice profila

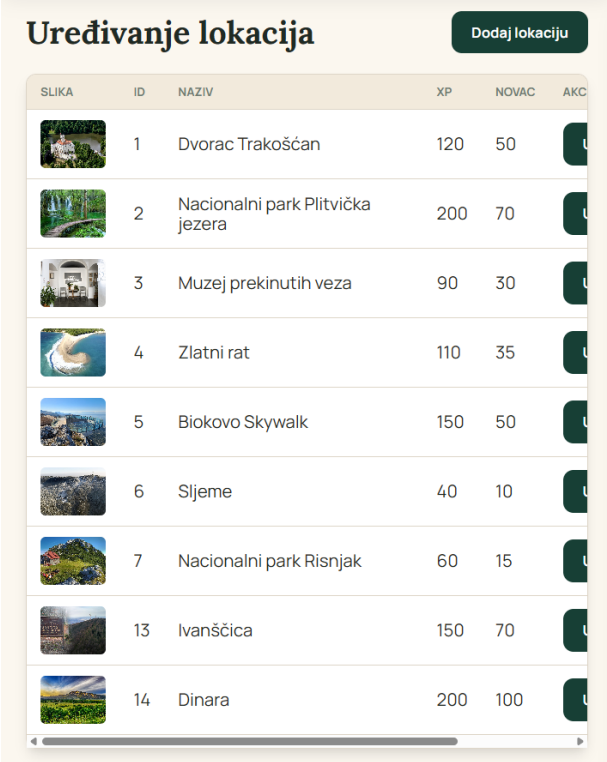


Slika 23: Izgled forme za uređivanje podataka profila i odabira dekoracija avatar i pozadinske slike

6.4.7. Analiza stranice uređivanja lokacije

Stranica uređivanje lokacija predstavlja administratorski dio aplikacije i dostupan je samo korisnicima s odgovarajućom ulogom, odnosno ulogom administratora. Na toj stranici prikazuje se tablica sa svim lokacijama iz baze s najvažnijim podacima za svaku lokacija. Na kraju stupca prikazuje se akcije za brisanje i uređivanje lokacije. Klikom na gumb za uređivanje, korisnika se dovodi na stranicu za uređivanje lokacije s formom, dok na pritisak na gumb za brisanje, korisnika se preko dijaloga traži potvrda brisanja lokacije. Ako potvrdi brisanje na poslužitelja se šalje zahtjev za brisanje lokacije. Ovaj način je odabran da administrator dobiva jednostavan pregled postojećih lokacija i može brzo pristupiti njihovoj izmjeni. Isto tako na stranici se nalazi gumb za dodavanje lokacije koji korisnika odvodi na formu za dodavanje lokacije.

Zato što je ovo administracijska stranica očekuje se da korisnik koji joj pristupa koristi stolno ili prijenosno računalo, tako da responzivnost tablice nije bio primarni cilj. Responzivnost radi tako da se na manjim ekranima dodaje horizontalni klizač na tablicu tako da, ako je baš potrebno, administrator može pristupiti ostalim opcijama (Slika 24).



Slika 24: Izgled tablice svih lokacija na manjim ekranima

7. Zaključak

Razvoj web aplikacija postao je važan dio suvremenog digitalnog okruženja jer korisnici sve češće pristupaju informacijama i uslugama putem različitih uređaja. Web se razvio od jednostavnih statičkih stranica u složene aplikacije koje moraju biti brze, responzivne, sigurne i jednostavne za korištenje. Time korisničko sučelje nije više samo vizualni dio sustava, nego važan dio aplikacije koji utječe na korisničko iskustvo, performanse i dugoročnu kvalitetu rješenja.

U teorijskom dijelu rada obrađena je održivost programskog koda i načini pisanja takvog koda. Spomenuta je bila važnost čistog koda, refaktoriranje, smanjenje tehničkog duga, modularnost i razdvajanje odgovornosti. Objašnjene su osnovne tehnologija koje se koriste u praktičnom dijelu rad, a to su Angular i TypeScript te principi komponentnog pristupa, servisi, spremište stanja i organizacija stilova kroz SCSS, BEM i ITCSS. Cilj ovih načela jest postići održivost dosljednom strukturom, ponovnom iskoristivošću i jesnim granicama između prikaza, logike i dohvata podataka.

U praktičnom dijelu rada izrađena je aplikacija Kroz Hrvatsku čiji je cilj korisnicima omogućiti istraživanje prirodnih, kulturnih i turističkih lokacija u Hrvatskoj. Aplikacija omogućuje pregled i filtriranje lokacije, prikaz detalja lokacije, komentiranje i ocjenjivanje, izvršavanje izazova itd. Cilj u izradu aplikacije bio jest primjena teorijskih načela unutar ove Angular aplikacije primjenom kontejnerskih i prezentacijskih komponenti, servisa i spremišta stanja. Poslužiteljski dio izrađen je pomoću NestJSa, te je korištena PostgreSQL baze i Prisma ORM.

Rad pokazuje da održiva web aplikacije nastaje od samog početka, postavljanjem temelja koja za cilj imaju dugoročnost aplikacije. Tehnologije izrade same po sebi ne jamče kvalitetu, nego ona proizlazi od arhitekture, načina organiziranja komponenti, stilova i podataka te ispravne primjene načela i načina pristupa prema projektu.

Popis literature

[1] H. Washizaki, „Software Maintenance," *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*, IEEE Computer Society, 2024., od 7-1 do 7–18. adresa: https://www.swebok.org

[2] H. Bandaranayake. „The Birth of the First Website: How It All Began," pogledano 5. studenoga 2026. adresa: https://medium.com/@acs_usj/the-birth-of-the-first-website-how-it-all-began-9fc1e5c1af50

[3] F. Doglio. „Top 7 Frontend Frameworks to Use in 2026: Pro Advice," pogledano 5. studenoga 2026. adresa: https://roadmap.sh/frontend/frameworks

[4] *ISO/IEC/IEEE 14764:2022(En) Software Engineering — Software Life Cycle Processes — Maintenance*, Standard, 2022. pogledano 16. svibnja 2026. adresa: https://www.iso.org/obp/ui/en/#iso:std:iso-iec-ieee:14764:ed-3:v1:en

[5] R. C. Martin i R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design* (Robert C. Martin Series). London, England: Prentice Hall, 2018., 404 str., ISBN: 978-0-13-449416-6.

[6] R. C. Martin, ur., *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall, 2009., 431 str., ISBN: 978-0-13-235088-4.

[7] B. Plavljanić. „Vibe coding: Revolucija u razvoju softvera ili prolazni trend?" Pogledano 16. svibnja 2026. adresa: https://pcchip.hr/softver/korisni/vibe-coding-revolucija-u-razvoju-softvera-ili-prolazni-trend/

[8] M. Dowden, *Architecting CSS: The Programmer's Guide to Effective Style Sheets*. Berkeley, CA: Apress L. P, 2020., 1 str., ISBN: 978-1-4842-5749-4 978-1-4842-5750-0.

[9] S. Marla, *A Journey to Angular Development*. Delhi: BPB Publications, 2021., 1 str., ISBN: 978-93-91030-14-8 978-93-91030-15-5.

[10] A. Kagoshima. „UI Architecture Patterns. MVC and MVVM," pogledano 18. lipnja 2026. adresa: https://medium.com/@a.kago1988/mvc-vs-mvvm-understanding-the-pros-and-cons-in-real-world-projects-efdb2544d450

[11] *Angular Documentation*, Documentation. pogledano 19. lipnja 2026. adresa: https://angular.dev/overview

[12] P. Gasston, *Moderni Web : Responzivni Web Dizajn Uz HTML5, CSS3 i JavaScript*. Zagreb:Dobar plan, 2013., ISBN: 978-953-7398-41-5.

[13] B. Frain, *Responsive Web Design with HTML5 and CSS: Develop Future-Proof Responsive Websites Using the Latest HTML5 and CSS Techniques, 3rd Edition*, 3rd ed. Place of publication not identified: Packt Publishing, 2020., 1 str., ISBN: 978-1-83921-156-0 978-1-83921-979-5.

[14] H. Roberts. „shame.css," csswizardry, pogledano 26. lipnja 2026. adresa: https://csswizardry.com/2013/04/shame-css/

Popis slika

1. Vodopadni model (Izvor: slika je rekreirana prema slici sa https://artoftesting.com/waterfall-model, pristupljeno: 16.6.2026.) 9
2. Agilni model (Izvor: slika je rekreirana prema slici sa https://www.code-brew.com/agile-methodology/, pristupljeno: 16.6.2026.) 10
3. Vizualni prikaz arhitekture korisničkog sučelja (Izvor: vlastita izrada) 26
4. Stablo Angular komponenata (Izvor: vlastita izrada) 28
5. Prikaz arhitekture obrnutog trokuta CSS-a (Izvor: slika je rekreirana prema slici sa https://www.xfive.co/blog/itcss-scalable-maintainable-css-architecture, pristupljeno: 26.6.2026.) 54
6. Prikaz dijagrama slučaja korištenja (eng. use case) za aplikaciju Kroz Hrvatsku . 56
7. Prikaz dijagrama entiteta za aplikaciju Kroz Hrvatsku 58
8. Organizacija direktorija Angular aplikacije 60
9. Organizacija mape funkcionalnosti 61
10. Organizacija stilova prema ITCSS arhitekturi 63
11. Izgled navigacije kada korisnik nije prijavljen 69
12. Izgled navigacije kada je korisnik prijavljen 69
13. Izgled navigacije na manjim ekranima 71
14. Izgled sekcije lokacija unutar početne stranice 72
15. Izgled sekcije lokacija kada je županija odabrana 79
16. Dijagram slijeda na pritisak županije na karti 79

17. Primjena filtera iz programskog kod 6.30 83
18. Prikaz dizajna kartica lokacije kada ona nije riješena i kada jest 84
19. Raspored kartica lokacija ovisno o širini ekrana 85
20. Izgled stranice detalja lokacije 88
21. Izgled stranice trgovina 90
22. Izgled zaglavlja stranice profila 91
23. Izgled forme za uređivanje podataka profila i odabira dekoracija avatar i pozadinske slike 92
24. Izgled tablice svih lokacija na manjim ekranima 93

Popis programskih kodova

2.1. Devina notacija 5
2.2. Zmijina notacija 5
2.3. Notacija - mađarska notacija 5
3.1. KISS primjer - grupiranje ternarnih operatora 14
3.2. KISS primjer - preduboko ognježđivanje 14
4.1. Primjer osnovne Angular komponente 22
4.2. Primjer JSON objekta odgovora REST API-a 25
4.3. Primjer modela podataka unutar aplikacije 25
4.4. Primjer izvoza funkcije iz modula 27
4.5. Primjer uvoza funkcije iz modula 28
4.6. Primjer klase roditelja 29
4.7. Primjer povezivanja atributa roditelja prema dijetetu 29
4.8. Primjer dekoratora `@Input()` 29
4.9. Primjer dekoratora `@Output()` 29
4.10. Primjer primanja vrijednosti dijeteta 30
4.11. Primjer obrade proslijeđene vrijednosti 30
4.12. Primjer definiranja ruta 31
4.13. Primjer korištenja atributa routerLink 31
5.1. Primjer zbrajanja selektora - HTML 35
5.2. Primjer zbrajanja selektora - CSS 35
5.3. Primjer odvjanje strukture od izgleda 37
5.4. Primjer odvjanje spremnika od sadržaja 38
5.5. Primjer BEM konvencije imenovanja klasa - HTML 40
5.6. Primjer BEM konvencije imenovanja klasa - CSS 40
5.7. Primjer ugnježđivanja selektora unutar SCSS-a 41
5.8. Primjer SCSS Mixins 42
5.9. Primjer SCSS dekoratora @extend 43
5.10. Primjer meidjskog upita 44
5.11. Primjer korištenja CSS funckije calc() 46
5.12. Primjer ograničavanja širine spremnika - Media upit 46
5.13. Primjer ograničavanja širine spremnika - funkcija min() 46
5.14. Primjer funkcije clamp() 47
5.15. Primjer potpuno fluidne CSS rešetke 49
5.16. Primjer prilagođenog svojstva 49
5.17. Primjer korištenja CSS funkcija bez CSS varijabli 50

5.18. Primjer korištenja CSS funkcija s CSS varijabli 50
6.1. Primjena prisma upita na bazu 58
6.2. Poziv funkcije dohvatiSveLokacije() na poziv rute 59
6.3. Uključivanje zasebnih scss datoteka u _index.scss 62
6.4. Uključivanje _index.scss datoteka slojeva u glavnu stilsku datoteku 62
6.5. Definiranje vrijednosti tipografije 63
6.6. Proslijeđivanje dijelova sloja preko `_index.scss` 63
6.7. Primjena definiraih varijabli 64
6.8. Definiranje klase za raspored elemenata jedan pored drugog 64
6.9. Primjerna klase objekta i razdvjanje strukture od izgleda - HTML 65
6.10. Primjerna klase objekta i razdvjanje strukture od izgleda - CSS 65
6.11. Definiranje glavnog rasporeda stranica 66
6.12. Predložak navigacije 67
6.13. Sakrivanje sadržaja navigacije na navigacije na manjim ekranima i dodavlje hamburder ikone 69
6.14. Primjena zastavice za prikaza navigacije na manjim ekranima 69
6.15. Predložak mobilne navigacije 70
6.16. Struktura početne stranice 71
6.17. Struktura predloška sekcije lokacija 72
6.18. Programski kod putanje za link jedne županije 73
6.19. Programski kod klase komponente `app-karta-hrvatske` 74
6.20. Emitiranje naziva županije prema iz `app-lokacija-sekcija` komponente prema komponenti početne stranice 74
6.21. Poziv spremišta za dohvat podataka 74
6.22. Model podataka lokacije koja se dohvaća 75
6.23. Poziv servisa za dohvat podatak iz baze u spremištu 75
6.24. Sučelje servisa za lokacije 76
6.25. Implementacija servisa za dohvat lokacija 76
6.26. Povezivanje sučelja i implementacije servisa 77
6.27. Programski kod rute za dohvat lokacija u kontrolenu na poslužitelju 77
6.28. Programski kod funkcije za dohvat lokacija iz baze na poslužitelju 78
6.29. Kontejnerska komponenta stranice popisa lokacija 79
6.30. Primjer URL-a s parametrima za filtriranje lokacija 80
6.31. Postavljanje vrijednosti filtera i dohvaćanje lokacije prema zadanom filteru 81
6.32. Funkcija za primjenu filtera 81
6.33. Definiranje parametara upita za rutu poslužitelja 82
6.34. Definicija rešetke za popis kartica lokacija 84
6.35. Predložak kontejnerske komponente stranice detalja lokacije 85